

Efficient Threshold ML-DSA

Sofía Celi ^{1,2}, Rafael del Pino ³, Thomas Espitau ³, Guilhem Niot ^{3,4}, Thomas Prest ³

¹ Brave Research ² University of Bristol ³ PQShield ⁴ Univ Rennes, CNRS, IRISA

Abstract

Threshold signature schemes allow a group of users to jointly generate a digital signature, providing resilience against faults and enhancing decentralization. With the advent of post-quantum cryptography, lattice-based threshold signatures have gained attention as viable PQ-threshold solutions. Nevertheless, existing constructions are limited in terms of their scalability, robustness. Worse, none is compatible with standardized schemes, particularly with the NIST-selected and standardized Module-Lattice-based Digital Signature Algorithm (ML-DSA) algorithm.

In this work, we present the first threshold signature scheme that is *fully compatible* with ML-DSA, supporting secure and efficient signing among up to six parties. Our construction leverages advanced short secret sharing techniques and integrates optimized rejection sampling to achieve a favorable balance between communication efficiency and correctness in distributed environments. We implement our construction in Go and evaluate its performance across local, LAN, and WAN network settings. Our benchmarks demonstrate that our threshold ML-DSA scheme is not only practically deployable but also well-suited for real-world applications, including multi-device cryptocurrency wallets, threshold-based TLS authentication, and for Tor’s directory authorities.

1 Introduction

As organizations migrate to post-quantum cryptography, they are adopting the Module-Lattice-based Digital Signature Algorithm (ML-DSA), recently finalized by NIST [49]. However, this transition creates a critical security gap: modern systems rely on threshold signature schemes (TSS) to eliminate single points of failure and distribute trust, but no practical threshold construction for ML-DSA exists. This forces a dangerous trade-off between achieving quantum resistance and maintaining the operational resilience provided by mature classical

threshold schemes for RSA, Schnorr, and ECDSA. Systems that migrate to ML-DSA today must forfeit the threshold guarantees they depend on.

The absence of a threshold ML-DSA scheme is not an oversight but a deep technical challenge. The core of ML-DSA’s design—the *Fiat-Shamir-with-aborts* paradigm—is fundamentally at odds with multi-party computation. This paradigm relies on a rejection sampling procedure where a signature attempt is “aborted” and retried. Rejected candidates must not be leaked and thereof, in a threshold setting, this requires multiple parties to collaboratively decide whether to abort without revealing the candidate to each other, nor to an adversary. Naïve protocols for this either statistically leak or incur catastrophic performance overhead. Even the two-party case has remained an open problem, as the abort decision depends on the final signature, which cannot be revealed until all parties agree the signature is valid.

1.1 Our Contribution: The First Practical Threshold ML-DSA

We introduce the first practical threshold signature scheme for the ML-DSA standard. Our protocol allows an existing ML-DSA key to be securely distributed among multiple parties (a posteriori sharing). It also includes a fully distributed key generation (DKG) protocol that eliminates any trusted dealer. It supports signing up to 6 parties. While a threshold of six may appear small at first glance, it represents a significant step forward and is already more than sufficient for most industrial applications (which requires usually 2-out-of-2 and 2-out-of-3 settings).

Our design overcomes the fundamental conflict between ML-DSA’s abort-and-retry procedure and multi-party computation. We achieve this by tailoring the core ideas of short secret sharing and per-party rejection from del Pino and Niot [20] to ML-DSA’s unique constraints.

Technical Innovations. Our key contribution is a distributed protocol described in Section 3.1. It is a novel combination of (i) short replicated secret sharing, and (ii) optimized per-party rejection sampling, specifically designed for ML-DSA’s unique constraints. Our protocol includes a two-stage rejection sampling: a per party rejection, to ensure the good shape of all partial signatures, and that their reveal does not leak any secret; and a global one, at the time of combination, which ensures that the final combined signature fits in the sizes of ML-DSA signatures. Note that while the first one acts as in ML-DSA as is required for security, the second one is only there for size compatibility. Even though this idea seems simple, it requires several new pieces of technology to make it work altogether:

- **Unbalanced Hyperball Rejection Sampling.** We replace the rejection sampling technique from ML-DSA with one based on hyperballs. This significantly reduces abort probability when multiple parties must simultaneously satisfy the rejection criteria. Further, we fine-tune this new rejection sampling by using simultaneously different norms.
- **Optimized Share Reconstruction (Section B).** The probability of signing success is directly tied to the size of the secret shares. We designed a tailored reconstruction procedure that minimizes this size by modeling the problem as a variant of the B-matching problem, which we solve efficiently using a maximum flow algorithm.
- **Parallel Protocol Instances.** To ensure high signing success rates at reconstruction time, our protocol runs multiple instances in parallel. This approach guarantees fast and reliable signing without compromising security.

For key generation, we cover two complementary scenarios:

- **A Posteriori Key Sharing (Section 3.3).** For settings where the signing key is fixed a priori (e.g., for backward compatibility), we show how it can be securely shared among multiple users, while crucially preserving the original verification key.
- **Distributed Key Generation.** We design a fully decentralized key generation protocol that removes the need for a trusted dealer.

Formal Security (Section 3.2) We provide a full formal analysis of our scheme, showing that it preserves the security and correctness guarantees of ML-DSA under standard MLWE assumptions.

Practical Deployment (Section 4) We implement our scheme in Go and benchmark its performance across local, LAN, and WAN settings. Our experiments show that a signing attempt completes in milliseconds in local deployments, and under one second in wide-area settings across continents. Communication costs range from 10.5 kB to 525 kB per party depending on the threshold (T, N) , which is competitive with established classical TSS. Crucially, the output signatures are fully compliant with the standard ML-DSA format, enabling drop-in integration with existing systems.

1.2 Related Work

Recent years have witnessed a surge of interest in constructing TSS that are secure against quantum adversaries. While classical threshold signatures have been deployed and extensively studied, extending them to the post-quantum setting introduces significant challenges, especially for schemes compatible with NIST-designated standards and candidates.

Threshold Signature Schemes. Classical TSS constructions include schemes based on RSA [32], Schnorr [44, 14], and ECDSA [8, 9], many of which offer advanced properties such as identifiable aborts. However, the design of post-quantum TSS has proven more difficult, with many proposals relying on assumptions outside of the NIST call for standardization. Proposed post-quantum schemes span a variety of hardness assumptions: lattice-based, isogeny-based [15], and hash-based [41]. A recent multivariate-based TSS by Celi et al. [10] achieves compatibility with the MAYO and UOV signature schemes using generic MPC techniques, potentially offering practical efficiency for specific parameter choices.

Lattice-based Fiat-Shamir threshold schemes. Given the perceived impracticality of directly thresholdizing ML-DSA due to its rejection sampling mechanism, early efforts in the lattice setting focused on Fiat-Shamir-style signatures without rejection sampling, targeting variants of the Raccoon signature scheme [16]. These include threshold schemes based on threshold FHE by Agrawal et al. [1] and Gur et al. [34], and standard tools by del Pino et al. [18] Espitau et al. [25], Katsumata et al. [40] and Chairattana-Apirom et al. [11]. While these schemes typically yield larger signatures, they offer attractive features such as adaptive security [40], robustness [26], identifiable aborts [17, 19], and Beyond Unforgeability Features (BUFF) [31]. These design directions avoid the difficulties associated with ML-DSA’s rejection mechanism but sacrifice compatibility with a standardized scheme.

Table 1: Comparison of approaches to thresholdize ML-DSA.

Scheme	Paradigm	# Parties	# Rounds	Communication (MB)	Computation	Security
This work	Tailored	6	6	0.021 to 1.05	Lightweight	Standard
Bienstock et al. [5]	MPC	Unlimited	96 24	$>1.2^*$ $>2.3^*$	Online lightweight*	Honest majority
Trilithium [23]	MPC	2	60	234 [†]	Heavy	Trusted party [†]
Generic MPC [13]	MPC	Unlimited	High	High	Impractical	Standard

Rounds and communication costs are reported as per-party averages, measured over successful signing attempts. These reflect the impact of ML-DSA’s inherent probabilistic abort mechanism. *Communication and computation costs exclude the offline generation of correlated randomness (via Beaver triples, edaBits, etc.). [†]Requires 234 MB of correlated randomness to be generated by a trusted party.

Direct Thresholdization of ML-DSA. Despite its complexity, several works have pursued direct thresholdization of ML-DSA, primarily through generic MPC techniques that address its rejection sampling behavior (Table 1).

The generic MPC-based approach of Cozzo and Smart [13] is secure in theory but suffers from high computational and communication costs, making practical deployment infeasible. The “Trilithium” scheme by Dufka et al. [23] supports two-party signing, but it assumes a *trusted party* to generate correlated randomness: an assumption that falls outside standard adversarial models and incurs high communication overhead. Moreover, its security analysis lacks a full proof of unforgeability, relying instead on heuristic assumptions about the distribution of rejected ML-DSA transcripts.

Most recently, and concurrently with this work, Bienstock et al. [5] proposed a threshold ML-DSA variant using MPC under an honest-majority assumption. Their construction achieves either 96 rounds with 1.2 MB of communication per party or 24 rounds with 2.3 MB, depending on parameter choices. To which must be added the cost of correlated randomness generation, which is known to be expensive.

Our scheme departs from generic MPC paradigms by tailoring the design to ML-DSA design. It supports up to 6 parties in the strongest adversarial model (dishonest majority), retains compatibility, and achieves practical efficiency without relying on trusted setup or heuristic security assumptions.

Secret Sharing Techniques Recent works by Chairattana-Apirom et al. [11] and del Pino et al. [17] highlighted the benefits of short secret sharing techniques – consisting of a secret sharing with short partial shares – which enable more compact and efficient protocols [7, 11], and support advanced features such as identifiable aborts at no additional cost [17]. Additionally, these techniques

simplify integration with rejection sampling [20], which is crucial for this work.

2 Preliminaries

2.1 General notations

Algorithmic notations. To denote the *assign* operation, we use $y := f(x)$ (resp. $y \leftarrow f(x)$) when f is deterministic (resp. randomized). We note **req** (resp. **assert**) the function that takes a proposition x as input and outputs 0 (resp. $\perp$) if x is false, else it does nothing.

Set and distributions. For an integer $N > 0$, we denote $[N] = \{0, \dots, N - 1\}$. We recall that the union $S \cup T$ of two sets writes $S \sqcup T$ when disjoint. For a set S , we denote by $\mathcal{U}(S)$ the *uniform distribution* over S , and write $x \xleftarrow{\$} S$ as shorthand for $x \leftarrow \mathcal{U}(S)$. For a real parameter $0 \leq p \leq 1$, the *Bernoulli distribution* $\text{Ber}(p)$ outputs 1 with probability p and 0 otherwise. The *Binomial distribution* $\text{Bin}(m, p)$ is defined as the sum of m independent Bernoulli trials sampled from $\text{Ber}(p)$.

Linear algebra. For a fixed power-of-two n , we denote $\mathcal{R} = \mathbb{Z}[x]/(x^n + 1)$ the cyclotomic ring and $\mathcal{R}_q = \mathcal{R}/(q\mathcal{R})$. We also let $\mathcal{R}_{\mathbb{R}} = \mathbb{R}[x]/(x^n + 1)$ be a polynomial ring over real numbers. Given $\mathbf{x} \in \mathcal{R}_{\mathbb{R}}^{\ell}$, we denote $\|\mathbf{x}\|_2$ the Euclidean norm (resp. $\|\mathbf{x}\|_{\infty}$ the infinite norm) of the $(n\ell)$ -dimensional vector of the coefficients of $\mathbf{x}$. Unless specified otherwise, vectors are *column* vectors.

2.2 Threshold signatures

In all this work, we denote the security parameter by κ . Following the syntax of Katsumata et al. [40], a R -round threshold signature (*without* session identifiers) is as a tuple $\text{TSS} = (\text{Setup}, \text{Keygen}, (\text{Sign}_i)_{i \in [R]}, \text{Combine}, \text{Verify})$. $\text{act} \subseteq [N]$ denotes a set of T parties used for

signing. Each signer i maintains a state st_i : short-lived session-specific information.

Setup(κ, N, T) $\rightarrow$ **pp**. Takes as input a security parameter κ , the total number of parties N , the reconstruction threshold $T \leq N$. Outputs a set of public parameters **pp**.

Keygen(**pp**) $\rightarrow$ (**vk**, (sk_i) $_{i \in [N]}$). Takes as input the public parameters. Outputs a verification key **vk**, and partial private keys (sk_i) $_{i \in [N]}$.

Sign_r(**vk**, **act**, **msg**, (pm_{r-1}^j) $_{j \in \text{act}}$, i , sk_i , st_i) $\rightarrow$ (pm_r^i , st_i). For the r -th round ($r \in \{1, \dots, R\}$), takes as input the verification key **vk**, the set of signers **act**, a message **msg**, the protocol messages exchanged during the previous round (pm_{r-1}^j) $_{j \in \text{act}}$, as well as the partial private key sk_i and state st_i of user i . Outputs a new message pm_r^i and updated state st_i . We define $pm_0^i = \perp$.

Note that **msg** and **act** may optionally be provided only from a round later than the first.

Combine(**vk**, **act**, **msg**, (pm_r^i) $_{r \in [R], i \in \text{act}}$) $\rightarrow$ **sig**. Takes as input the verification key **vk**, the set of signers **act**, the message **msg**, all messages exchanged during the protocol (pm_r^i) $_{r \in [R], i \in \text{act}}$. Outputs a signature **sig**.

Verify(**vk**, **msg**, **sig**) $\rightarrow$ 0 or 1. Takes as input a verification key **vk**, a message **msg**, and a signature **sig**. Outputs 1 if **sig** is valid and 0 otherwise.

A key property of threshold signatures is unforgeability, which ensures that an adversary cannot produce valid signatures without the cooperation of honest parties. Several security models exist, differing in whether the adversary chooses corrupted users statically [18] or adaptively [40], and whether the communication channel is controlled by the adversary [18], or requires consensus properties [26, 17], and for which messages a forgery is accepted.

In this work, we consider the unforgeability notion from del Pino et al. [18] adapted to remove the use of session identifiers. It ensures static security – i.e. corrupted users are chosen before executing the protocol, a communication channel fully controlled by the adversary, and forgeries are accepted for messages for which no signing oracle was called. We provide formal definitions of these properties below.

Note that, for small thresholds (T, N), static security already implies adaptive security via the *complexity leveraging* technique, with at most a 5-bit theoretical loss when $N \leq 6$, as is the case for our Threshold ML-DSA construction. This loss appears to stem from limitations of existing proof techniques rather than from any inherent weakness of the scheme itself.

Definition 2.1 (Unforgeability). For a R -round TSS, the advantage of an adversary $\mathcal{A}$ (with oracle access to a random oracle H) against the unforgeability of TSS is defined as:

$$\text{Adv}_{\text{TSS}, \mathcal{A}}^{\text{TS-UF}}(\kappa, N, T) := \Pr \left[\text{Game}_{\text{TSS}, \mathcal{A}}^{\text{TS-UF}}(\kappa, N, T) = 1 \right]$$

where $\text{Game}_{\text{TSS}, \mathcal{A}}^{\text{TS-UF}}$ is described in Figure 1. We say that TSS is unforgeable in the random oracle model, if for all PPT adversary $\mathcal{A}$, $\text{Adv}_{\text{TSS}, \mathcal{A}}^{\text{TS-UF}}(\kappa, N, T) \leq \text{negl}(\kappa)$.

$\text{Game}_{\text{TSS}, \mathcal{A}}^{\text{TS-UF}}(\kappa, N, T)$
1: $Q_{\text{msg}} := \emptyset$ 2: $\text{pp} \leftarrow \text{Setup}(\kappa, N, T)$ 3: $\text{corrupt}, st_{\mathcal{A}} \leftarrow \mathcal{A}(\text{pp})$ 4: assert { $\text{corrupt} \subseteq [N]$ and $ \text{corrupt} < T$ } 5: $\text{honest} := [N] \setminus \text{corrupt}$ 6: (vk , (sk_i) $_{i \in [N]}$) $\leftarrow \text{Sign.Keygen}(\text{pp})$ 7: for { $i \in \text{honest}$ } do { $st_i := \emptyset$ } 8: (msg , sig) $\leftarrow \mathcal{A}^{H, (\text{OSign}_i(\cdot))_{i \in [R]}}(st_{\mathcal{A}}, \text{vk}, (sk_i)_{i \in \text{corrupt}})$ 9: req { $\text{msg} \notin Q_{\text{msg}}$ } 10: return Verify (vk , msg , sig)
$\text{OSign}_r(\text{act}, \text{msg}, i, (pm_{r-1}^j)_{j \in \text{act}})$
1: req { $i \in [N] \wedge i \in \text{act} \cap \text{honest}$ } 2: $Q_{\text{msg}} := Q_{\text{msg}} \cup \{\text{msg}\}$ 3: (pm_r^i , st_i) $\leftarrow \text{ShareSign}_r(\text{vk}, \text{act}, \text{msg}, (pm_{r-1}^j)_{j \in \text{act}}, i, sk_i, st_i)$ 4: return pm_r^i

Figure 1: Unforgeability game for TSS, where H denotes the random oracle. We consider that the OSign_r oracles return $\perp$ if the corresponding ShareSign_r fails to output a protocol message. Finally, $\mathcal{A}$ wins if the game TS-UF returns 1, i.e. if it forged a new signature.

We also wish to ensure *correctness*, i.e. that a valid signature is output when signers behave honestly. We however tolerate a probability of abort to support the *Fiat-Shamir with Aborts* paradigm.

Definition 2.2 (Correctness of TSS with aborts). We say that a R -round TSS with aborts p -terminates if:

$$\Pr \left[\text{Game}_{\text{TSS}}^{\text{TS-corr}}(\kappa, N, T, \text{msg}, \text{act}) \neq \perp \right] \geq p \quad (1)$$

$$\Pr \left[\text{Game}_{\text{TSS}}^{\text{TS-corr}}(\kappa, N, T, \text{msg}, \text{act}) = 0 \right] = \text{negl}(\kappa) \quad (2)$$

where $\text{Game}_{\text{TSS}}^{\text{TS-corr}}$ is defined in Figure 2.

2.3 Modulus Rounding

We recall helper functions over $\mathbb{Z}_q$, used in ML-DSA. These functions are applied coefficient-wise when applied

Game _{TSS} ^{TS-corr} ($\kappa, N, T, \text{msg}, \text{act}$)
<pre> 1: pp ← Setup(κ, N, T) 2: (vk, (sk_i)_{i ∈ [N]}) ← Sign.Keygen(pp) 3: for {i ∈ [N]} do {st_i := ∅} 4: for r ∈ [R] do 5: for i ∈ act do 6: (pm_{r,i}ⁱ, st_i) ← ShareSign_r(vk, act, msg, 7: (pm_{r-1,j}^j)_{j ∈ act, i, sk_i, st_i}) 8: if { ∃ i ∈ act, pm_{r,i}ⁱ = abort } then { return ⊥ } 9: sig := Combine(vk, act, (pm_{r,i}ⁱ)_{r ∈ [R], i ∈ act}) 10: return Verify(vk, msg, sig) </pre>

Figure 2: Correctness game for TSS with aborts.

to polynomials in $\mathcal{R}_q$ to optimize the sizes of keys and signatures.

Power2Round_q(r): rounds an integer $r \in \mathbb{Z}_q$. It outputs a pair (r_0, r_1) , where $r_0 = r \bmod \pm 2^d$ and $r_1 = (r - r_0)/2^d$.

HighBits(r, α): returns the high-order bits r_1 of r , defined as $\text{HighBits}(r, \alpha) = \frac{r - (r \bmod \pm \alpha)}{\alpha}$ if $r - (r \bmod \pm \alpha) \neq q - 1$ and 0 otherwise.

MakeHint_q(z, r, α): produces a binary hint $h \in \{0, 1\}$ indicating that the high bits of z and $z+r$ differ. The hint mechanism ensures the correct reconstruction of high bits during verification.

UseHint_q(h, r, α): takes a hint bit $h \in \{0, 1\}$ and an integer $r \in \mathbb{Z}_q$. Based on the hint, it corrects the computation of $\text{HighBits}(r, 2\gamma_2)$. The goal is to recover $\text{HighBits}(r+z, 2\gamma_2)$. If $h = 1$, a carry occurred, and the high bits need adjustment: the high bits by +1 if $r - \alpha \cdot r_1 > 0$, and otherwise by -1.

Lemma 2.1 ([42]). For $q, \alpha > 0$ with $q > 2\alpha$, $q = 1 \bmod \alpha$ and α is even. Let $\mathbf{r}, \mathbf{z} \in \mathcal{R}_q$ where $\|\mathbf{z}\|_\infty \leq \alpha/2$. Then:

$$\text{UseHint}_q(\text{MakeHint}_q(\mathbf{z}, \mathbf{r}, \alpha), \mathbf{r}, \alpha) = \text{HighBits}_q(\mathbf{r} + \mathbf{z}, \alpha).$$

2.4 Hardness Assumptions

We rely on the MLWE assumption, underlying the security of ML-DSA. Note that we do not explicitly leverage the STMSIS [42] assumption, which also underlies ML-DSA, and instead assume the unforgeability of ML-DSA in our proofs.

Definition 2.3 (MLWE). Let k, ℓ, q be integers, χ be a probability distribution over $\mathcal{R}^{k+\ell}$. The advantage of

an algorithm $\mathcal{A}$ in solving the $\text{MLWE}_{q,k,\ell,\chi}$ problem is defined as:

$$|\Pr[\mathcal{A}(\mathbf{A}, [\mathbf{A} \ \mathbf{I}_k] \cdot \mathbf{s}) = 1] - \Pr[\mathcal{A}(\mathbf{A}, \mathbf{u}) = 1]| \quad (3)$$

where $\mathbf{A} \xleftarrow{\$} \mathcal{R}_q^{k \times \ell}$, $\mathbf{u} \xleftarrow{\$} \mathcal{R}_q$, $\mathbf{s} \leftarrow \chi$. The MLWE assumption states that any efficient adversary $\mathcal{A}$ has a negligible advantage against MLWE.

2.5 The Rényi divergence

Following Devevey et al. [21], we rely on the *Rényi divergence of infinite order* R_∞ as well as the *smooth Rényi divergence* R_∞^ε to measure the distance between two distributions.

Definition 2.4. Let $\mathcal{P}, \mathcal{Q}$ two distributions such that $\mathcal{P}$ is absolutely continuous with respect to $\mathcal{Q}$. The Rényi divergence of infinite order is $R_\infty(\mathcal{P} \parallel \mathcal{Q}) = \text{ess sup}_{\frac{\partial \mathcal{P}}{\partial \mathcal{Q}}}(x)$.

We additionally recall a relaxed version of the Rényi divergence from [21, Def. 2.1], called *smooth Rényi divergence*, where one can remove a few problematic points from the support, including those that may lie in $\text{Supp}(\mathcal{P}) \setminus \text{Supp}(\mathcal{Q})$.

Definition 2.5. Let $\varepsilon > 0$. Let $\mathcal{P}, \mathcal{Q}$ two probability distributions such that $\int_{\text{Supp}(\mathcal{Q})} \mathcal{P}(x) \partial \mu(x) \geq 1 - \varepsilon$ for the measure μ . Their ε -smooth Rényi divergence is $R_\infty^\varepsilon(\mathcal{P} \parallel \mathcal{Q}) = \inf\{M > 0 \mid \Pr_{x \leftarrow \mathcal{P}}(\mathcal{P}(x) \leq M \mathcal{Q}(x)) \geq 1 - \varepsilon\}$.

Lemma 2.2 ([3] and [38]). For two distributions $\mathcal{P}, \mathcal{Q}$, the Rényi divergence satisfies the following properties:

- **Data processing inequality.** $R_\infty(\mathcal{P}^f \parallel \mathcal{Q}^f) \leq R_\infty(\mathcal{P} \parallel \mathcal{Q})$ for any function f , where $\mathcal{P}^f$ (resp. $\mathcal{Q}^f$) is the distribution of $f(y)$ induced by sampling $y \leftarrow \mathcal{P}$ (resp. $y \leftarrow \mathcal{Q}$).
- **Multiplicativity.** Assume that $\mathcal{P}$ and $\mathcal{Q}$ are the joint distributions of random variables $(x_i)_i$. Note $\mathcal{P}_{i, |x_{<i}=v}$, $\mathcal{Q}_{i, |x_{<i}=v}$, the conditional distributions of x_i given $x_{<i} = v_{<i}$. Assume $R_\infty(\mathcal{P}_{i, |x_{<i}=v_{<i}} \parallel \mathcal{Q}_{i, |x_{<i}=v_{<i}}) \leq r_i$ for any possible $v_{<i}$. Then, $R_\infty(\mathcal{P} \parallel \mathcal{Q}) \leq \prod_i r_i$.
- **Probability preservation.** for any event $E \subseteq \text{Supp}(\mathcal{Q})$, we have $\mathcal{Q}(E) \geq \mathcal{P}(E)/R_\infty(\mathcal{P} \parallel \mathcal{Q})$.

2.6 Rejection Sampling

ML-DSA is a Fiat-Shamir with Aborts signature scheme. It heavily relies on the rejection sampling technique introduced by Lyubashevsky [47], which we recall with the same notations as Finally! [20].

$\text{Rej}(\mathbf{v}, \chi_{\mathbf{z}}, \chi_{\mathbf{r}}, M) \rightarrow \mathbf{z} \in \mathcal{R}^{k+\ell} \cup \{\perp\}$	$\text{Ideal}(\chi_{\mathbf{z}}, M) \rightarrow \mathbf{z} \in \mathcal{R}^{k+\ell} \cup \{\perp\}$
1: $\mathbf{r} \leftarrow \chi_{\mathbf{r}}$	1: $\mathbf{z} \leftarrow \chi_{\mathbf{z}}$
2: $\mathbf{z} := \mathbf{v} + \mathbf{r}$	2: $b \leftarrow \text{Ber}(\frac{1}{M})$
3: $b \leftarrow \text{Ber}(\min(\frac{\chi_{\mathbf{z}}(\mathbf{z})}{M\chi_{\mathbf{r}}(\mathbf{r})}, 1))$	3: if $\{b = 0\}$ then $\{\mathbf{z} = \perp\}$
4: if $\{b = 0\}$ then $\{\mathbf{z} = \perp\}$	4: return $\mathbf{z}$
5: return $\mathbf{z}$	

Figure 3: Rejection algorithm and ideal algorithm.

We extend the algorithm Rej to $\text{Rej}(\mathbf{v}, \chi_{\mathbf{z}}, \chi_{\mathbf{r}}, M; \mathbf{r})$ to parse the randomness $\mathbf{r} \leftarrow \chi_{\mathbf{r}}$ as an input. We will also write $(\mathbf{z}|\text{acc})_{\mathbf{v}}$ to denote the distribution of $\mathbf{z} := \mathbf{v} + \mathbf{r}$ conditioned on $b = 1$, and $(\mathbf{z}|\text{rej})_{\mathbf{v}}$ to denote the distribution of $\mathbf{z} := \mathbf{v} + \mathbf{r}$ conditioned on $b = 0$ (note that in that case $\mathbf{z} = \perp$ but the distribution we are interested in is the one of $\mathbf{v} + \mathbf{r}$ so we abuse this notation).

Rejection sampling aims to map a distribution $\mathbf{v} + \chi_{\mathbf{r}}$ – that depends on the sensitive value $\mathbf{v}$ – to a distribution $\chi_{\mathbf{z}}$ that does not. We recall the following lemma that bounds the Rényi divergence between the real and ideal distributions above.

Lemma 2.3 (Lemma 4.1, [21]). *Let $M > 1$ and $\varepsilon < 1$ such that $R_{\infty}^{\varepsilon}(\chi_{\mathbf{z}}\|\mathbf{v} + \chi_{\mathbf{r}}) \leq M$. Then, $R_{\infty}(\text{Rej}\|\text{Ideal}) \leq 1 + \frac{\varepsilon}{M-1}$.*

2.7 Rejection sampling over hyperballs

Devevey et al. [21] showed that hyperball-based rejection sampling achieves the most compact parameters, and outperforms uniforms [47] and polytopes as used by Bambury et al. [4]. Hyperballs achieve the same compactness as using Gaussians, but have the advantage of a simpler rejection condition in the form of a norm two bound check.

Definition 2.6 (Continuous hyperball). *We denote:*

$$\mathcal{B}_{\mathcal{R}, \ell}(r, \mathbf{c}) = \{\mathbf{x} \in \mathcal{R}_{\mathbb{R}}^{\ell} \mid \|\mathbf{x} - \mathbf{c}\|_2 \leq r\}$$

the continuous hyperball with center $\mathbf{c} \in \mathcal{R}_{\mathbb{R}}^{\ell}$ and radius $r > 0$ in dimension $\ell > 0$. When $\mathbf{c} = 0$, we omit it.

To bound the Rényi divergence we rely on the followings.

Definition 2.7 (Regularized Incomplete Beta Function). *The incomplete beta function is defined over $[0, 1] \times \mathbb{R}^+ \times \mathbb{R}^+$ as $B : (x; a, b) \mapsto \int_0^x t^{a-1}(1-t)^{b-1} \partial t$. We also define $I_x(a, b) = B(x; a, b)/B(1; a, b)$.*

Lemma 2.4 ([21, Lemma 5.1]). *Let $\ell \geq 1$ and $\mathbf{v} \in \mathbb{R}^{n\ell}$. Let $\varepsilon \in [0, 1/2)$ and $\phi \geq 1$ be such that $2\varepsilon =$*

$I_{1-1/\phi^2}(\frac{n\ell+1}{2}, \frac{1}{2})$. Let $r, r' > 0$ such that $r'^2 \geq r^2 + \|\mathbf{v}\|_2^2 + 2r\|\mathbf{v}\|_2/\phi$. It holds that:

$$R_{\infty}^{\varepsilon}(\mathcal{U}(\mathcal{B}_{\mathcal{R}, \ell}(r))\|\mathcal{U}(\mathcal{B}_{\mathcal{R}, \ell}(r', \mathbf{v}))) = \left(\frac{r'}{r}\right)^{n\ell} \quad (4)$$

Let $M > 1$. If $r \geq \|\mathbf{v}\|_2 \cdot \frac{\frac{1}{\phi} + \sqrt{\frac{1}{\phi^2} + M^2/(n\ell) - 1}}{M^2/(n\ell) - 1}$ and $r' = M^{1/(n\ell)}r$, then the value in Eq. (4) is upper-bounded by M .

Lemma 2.5 (from [21, Section A.6]). *For $n > 1, \phi > 1$, we have $I_{1-1/\phi^2}(\frac{n+1}{2}, \frac{1}{2}) < \left(1 - \frac{1}{\phi^2}\right)^{n-1} \cdot n \cdot \left(1 - \frac{1}{\phi}\right)$.*

We describe in Fig. 3 the imbalanced rejection sampling that we will use for Threshold-ML-DSA.

$\text{HRej}(\mathbf{v}, r, r', \nu, M) \rightarrow \mathbf{z} \in \mathcal{R}^{k+\ell} \cup \{\perp\}$
1: $\mathbf{r} \xleftarrow{\$} \mathcal{B}_{\mathcal{R}, \ell+k}(r')$
2: $(\mathbf{v}^{(1)}, \mathbf{v}^{(2)}) := \mathbf{v} \in \mathcal{R}_q^k \times \mathcal{R}_q^{\ell}$
3: $\mathbf{z} := (\mathbf{v}^{(1)} \cdot \nu, \mathbf{v}^{(2)}) + \mathbf{r}$
4: if $\{\ \mathbf{z}\ > r\}$ then $\{\mathbf{z} = \perp\}$
5: $(\mathbf{z}^{(1)}, \mathbf{z}^{(2)}) := \mathbf{z} \in \mathcal{R}_q^k \times \mathcal{R}_q^{\ell}$
6: return $[\mathbf{z}^{(1)} \cdot \nu, \mathbf{z}^{(2)}]$

Figure 4: Imbalanced rejection sampling over hyperballs. Major differences with Rej (Fig. 3) are highlighted.

3 Threshold ML-DSA

This section describes our variant of ML-DSA (Module-Lattice-Based Digital Signature Algorithm) optimized for small threshold settings. We build upon the foundation laid by Finally! [20], which introduced the key innovation of leveraging short replicated secret sharing to distribute rejection sampling in lattice-based signatures, by performing parallel rejection sampling steps in each party. However, Finally! was designed as a custom signature scheme, and adapting its techniques to the ML-DSA standard presents significant challenges due to fundamental differences in their rejection sampling, target norm (Finally! relies on the Euclidean norm, while ML-DSA uses the infinity norm) and rounding procedures.

At a high level, our scheme samples several short secrets $\mathbf{s}_I$ and derives a public key as the sum of their corresponding commitments $\text{vk} = \sum [\mathbf{A} \quad \mathbf{I}] \mathbf{s}_I$. These secrets are shared among all parties such that any subset of at least T parties jointly knows all the $\mathbf{s}_I$'s. Signing proceeds via parallel rejection sampling performed independently by each party.

Table 2: Parameters used in Threshold ML-DSA.

ML-DSA	
$\mathcal{R}_q$	Polynomial ring $\mathcal{R}_q = \mathbb{Z}[X]/(q, X^n + 1)$
(k, ℓ)	Dimension of the public matrix $\mathbf{A} \in \mathcal{R}_q^{k \times \ell}$
τ	Number of ± 1 's in challenge c
d	Amount of bits dropped from the public key $\mathbf{t}$
η	Secret key range: $\chi_s = \mathcal{U}([- \eta, \eta]^{n(\ell+k)})$
$(\gamma_1, \gamma_2, \beta)$	Ranges for the signature generation and verification ($\beta = \tau \cdot \eta$)
ω	Number of 1's in the hint h
Threshold ML-DSA	
K	Number of parallel protocol repetitions
ν	Expansion factor for the first ℓ coordinates of the randomness
r'	Randomness ball rad. $\chi_r = \{ \lfloor \nu \mathbf{x}_1, \mathbf{x}_2 \rfloor \mid (\mathbf{x}_1, \mathbf{x}_2) \xleftarrow{\$} \mathcal{B}_{\mathcal{R}, \ell+k}(r') \}$
r	Target ball rad. $\chi_s = \{ \lfloor \nu \mathbf{x}_1, \mathbf{x}_2 \rfloor \mid (\mathbf{x}_1, \mathbf{x}_2) \xleftarrow{\$} \mathcal{B}_{\mathcal{R}, \ell+k}(r) \}$
M	Rejection parameter (per party) $M = \left(\frac{r'}{r}\right)^{n(k+\ell)}$

While performing rejection sampling over T parties has a negative impact on the overall acceptance probability, we mitigate this by using tighter distributions over *hyperballs*, which is particularly effective in small-threshold regimes. Hence, our thresholdization exploits the non-tightness in ML-DSA's use of uniform sampling. We exploit the slack between rejection sampling over hyperballs (as noted in Section 2.7) and uniform distributions, allowing us to retain compact signature sizes even in the distributed setting.

As the *hint* technique used in ML-DSA only targets the second portion $\mathbf{z}^{(2)}$ of the signature, it introduce a much stricter verification condition on $\mathbf{z}^{(2)}$ than $\mathbf{z}^{(1)}$. We optimize the success rate of this check by using imbalanced rejection sampling in which $\mathbf{z}^{(2)}$ rejects towards a smaller hyperball than $\mathbf{z}^{(1)}$. The imbalance is parametrized by a factor ν . See HRej (Fig. 4) for a concrete description.

To amplify the success probability p , we run $K > 1$ iterations of the protocol in parallel. That is, during the signing procedure, each party i produces K commitments $(\mathbf{w}_{i,j})_{j \in [K]}$, and a response $\mathbf{z}_{i,j}$ for each aggregated commitment $\mathbf{w}_j = \sum_{i \in \text{act}} \mathbf{w}_{i,j}$. Concretely, taking $K = \lceil 1/p \rceil$ guarantees a correctness probability $\geq 1 - e^{-1} \approx 0.632$.

3.1 Construction

As defined in Section 2, threshold signature schemes primarily consist of key generation, signing, combining, and verifying procedures. To formalize these procedures for Threshold ML-DSA, we first outline the parameters used in the single-party ML-DSA scheme, along with the additional hyperball parameters $r_{N,T}$ and $r'_{N,T}$ required for our threshold variant. We collect all of them in Table 2.

Key Generation. Our construction relies on the *short secret sharing* notion introduced by del Pino et al. [17]

(RSS), which is particularly fit for lattice-based constructions. These secret sharings are special instances of Linear Secret Sharing Schemes (LSSS) that additionally guarantee shares with *small norm and small reconstruction coefficients*. This is particularly suited for rejection-sampling-based schemes, as it enables each party to perform rejection sampling locally on a *short* partially reconstructed secret. This *short replicated secret sharing* scheme consists in sampling $\binom{N}{N-T+1}$ ML-DSA secrets $(\mathbf{sk}_I)_I$ from χ_s , that is, one for each distinct set I of $N - T + 1$ users. Since there are at most $T - 1$ corrupted parties, at least one of these sets is fully non-corrupted and its secret remains hidden from the corrupted parties.

The final public key is derived by summing all secrets, multiplying by $[\mathbf{A} \ \mathbf{I}]$, and rounding the result as in ML-DSA, yielding a sample from the MLWE distribution. Thanks to at least one secret remaining hidden from corrupted parties, the public key maintains pseudorandomness under the same MLWE assumption as ML-DSA. In addition, any set J of T parties can reconstruct the aggregated secret: by construction, there exists a partition $\mathcal{P}_{j \in J}$ so that $\mathbf{sk} = \sum_{j \in J} (\sum_{I \in \mathcal{P}_j} \mathbf{sk}_I)$. As an example, for $N = 3$ and $T = 2$, we have $\mathbf{sk} = \mathbf{sk}_{01} + \mathbf{sk}_{02} + \mathbf{sk}_{12}$. If users 0, 2 want to sign, then user 0 will use $\mathbf{sk}_{01} + \mathbf{sk}_{12}$ as partial key, while user 2 will simply use $\mathbf{sk}_{12}$.

The key generation procedure is detailed in Fig. 5. While Fig. 5 assumes a trusted dealer for simplicity, a distributed key generation protocol is possible: see Section C.

$\text{RSS}(N, T, \chi) \rightarrow (\mathbf{s}, \mathbf{sk} := (\mathbf{sk}_1, \dots, \mathbf{sk}_N))$
1: for $I \subset [N]$ such that $ I = N - T + 1$ do 2: $\mathbf{s}_I \leftarrow \chi$ 3: $\mathbf{s} := \sum_I \mathbf{s}_I$ 4: for $i \in [N]$ do ▷ Distribute the shares 5: $\mathbf{sk}_i := \{I : \mathbf{s}_I \mid I \subset [N] \text{ s.t. } i \in I \wedge I = N - T + 1\}$ 6: return $(\mathbf{s}, \mathbf{sk} := (\mathbf{sk}_0, \dots, \mathbf{sk}_{N-1}))$
$\text{Keygen}(\kappa, N, T) \rightarrow (\mathbf{vk}, \mathbf{sk})$
1: $\rho \leftarrow \{0, 1\}^{256}$ 2: $\mathbf{A} := \text{ExpandA}(\rho)$ 3: $(\mathbf{s}, (\mathbf{sk}_i)_{i \in [N]}) \leftarrow \text{RSS}(N, T, \chi_s)$ 4: $\mathbf{t} := [\mathbf{A} \ \mathbf{I}] \cdot \mathbf{s}$ 5: $(\mathbf{t}_\top, \mathbf{t}_\perp) := \text{Power2Round}(\mathbf{t}, d)$ 6: $tr \in \{0, 1\}^{256} := H(\rho \parallel \mathbf{t}_\top)$ 7: return $(\mathbf{vk} := (\rho, \mathbf{t}_\top), \mathbf{sk} := (tr, \mathbf{sk}_i)_{i \in [N]})$

Figure 5: Key generation procedure of Threshold ML-DSA.

Signing. The distributed signing protocol is detailed in Fig. 6. Any set act of T signers collectively knows all

the secrets $\mathbf{s}_I$. The core idea is to publicly decide on a partition $\sqcup_{i \in \text{act}} \mathbf{m}_i = \{I\}_I$ of the secrets (or rather their indices) among the T parties using a deterministic function (RSSRecover), and then run T Fiat-Shamir protocols in parallel to produce a partial signature for each partial secret $\mathbf{s}_i^{\text{part}} = \sum_{I \in \mathbf{m}_i} \mathbf{s}_I$: partial commitments $\mathbf{w}_i = [\mathbf{A} \ \mathbf{I}] \cdot \mathbf{r}_i$ and signatures $\mathbf{z}_i = c \cdot \mathbf{s}_i^{\text{part}} + \mathbf{r}_i$.

In particular, each party uses a full MLWE sample as commitment $\mathbf{w}_i$ (i.e. $\mathbf{w}_i = \mathbf{A} \cdot \mathbf{y} + \mathbf{e}$) instead of using $\mathbf{A} \cdot \mathbf{y}$ as in ML-DSA. This change allows for direct reveal of $\mathbf{w}_i$ before rounding, even in case of rejection. To ensure correctness, we verify that the sum of the responses $\mathbf{z} = \sum_{i \in \text{act}} \mathbf{z}_i$ verifies the ML-DSA verification equation $[\mathbf{A} \ \mathbf{I}] \cdot \mathbf{z} = c \cdot \mathbf{t} + \mathbf{w}$: the challenge c can be computed as $c = \text{SampleInBall}(\tilde{c})$, where $\tilde{c} = H(\mu \| \text{HighBits}(\mathbf{w}, 2\gamma_2))$ and $\mathbf{w} = \sum_{i \in \text{act}} \mathbf{w}_i$. SampleInBall maps $\tilde{c}$ to a polynomial c such that $\|c\|_1 = \tau$ and $\|c\|_\infty = 1$.

Following prior works [18, 20], we design a three-round scheme secure against rushing adversaries (who target $\mathbf{w}$):

1. **First round:** parties sample randomness $\mathbf{r}_i \leftarrow \chi_{\mathbf{r}}$ and compute the commitment $\mathbf{w}_i = [\mathbf{A} \ \mathbf{I}] \cdot \mathbf{r}_i$. They publish a hash of the commitment $H_{\text{cmt}}(\text{vk}, i, \mathbf{w}_i)$.
2. **Second round:** parties reveal the values of $(\mathbf{w}_i)_{i \in \text{act}}$.
3. **Third round:** Each party computes the response $\mathbf{z}_i = c \cdot \mathbf{s}_i^{\text{part}} + \mathbf{r}_i$, where c is derived from the aggregated commitment $\mathbf{w} = \sum_{i \in \text{act}} \mathbf{w}_i$, and apply rejection sampling.

In the complete scheme, we omit the second part $\mathbf{z}_i^{(2)} \in \mathcal{R}_q^k$ of the responses since the verifier can reconstruct it from public values. Any public party can then aggregate the partial signatures into $\mathbf{z}^{(1)} = \sum_{i \in \text{act}} \mathbf{z}_i^{(1)} \in \mathcal{R}_q^\ell$ to obtain the signature and compute the corresponding hint $\mathbf{h}$ to output the signature $(c, \mathbf{z}^{(1)}, \mathbf{h})$.

Remark 1 (Balanced partition of the shares for Replicated Secret Sharing). *The sizes of partial secrets $\mathbf{s}_i^{\text{part}}$ in Threshold ML-DSA impact greatly the efficiency of parameters as it directly correlates with the norm 2 of the hyperballs that are used, and it is important to use as few secrets as possible in a session for each party in RSSRecover. To tackle this, we propose in Section B a graph based algorithm that computes a balanced partition of the shares. In practice, we can hardcode the optimal partition produced by our algorithm and include it as part of the public description of the Threshold ML-DSA scheme.*

Combine and Verify. The combine and verify procedures are detailed in Fig. 7. Combine involves computing

RSSRecover(act) $\rightarrow \mathcal{P}(S)^{\text{act}}$ 1: Compute partition $\mathbf{m} = (m_i)_{i \in \text{act}}$ s.t. $\sqcup m_i = \{I \subset [N] \mid I = N - T + 1\}$ and minimizing $\max_{i \in \text{act}} m_i$, using a maximum flow algorithm. See Section B. 2: return $\mathbf{m}$
ShareSign₁(vk, i, sk_i, st_i) 1: $\mathbf{r}_i \leftarrow \chi_{\mathbf{r}}$ 2: $\mathbf{w}_i := [\mathbf{A} \ \mathbf{I}] \cdot \mathbf{r}_i$ 3: $\text{cmt}_i = H_{\text{cmt}}(\text{vk}, i, \mathbf{w}_i) \in \{0, 1\}^{2\kappa}$ 4: $\text{st}_i := \text{st}_i \cup \{(\text{cmt}_i, \mathbf{w}_i, \mathbf{r}_i)\}$ 5: return $(\text{pm}_1^i := \text{cmt}_i, \text{st}_i)$
ShareSign₂(vk, act, msg, pm_1, i, sk_i, st_i) 1: assert $\{\text{act} \subseteq [N] \wedge i \in \text{act}\}$ 2: assert $\{(\text{pm}_1^i, \cdot, \cdot) \in \text{st}_i\}$ 3: Pick $(\text{cmt}_i, \mathbf{w}_i, \mathbf{r}_i)$ from st_i with $\text{pm}_1^i = \text{cmt}_i$ 4: $\text{st}_i := \text{st}_i \setminus \{(\text{cmt}_i, \mathbf{w}_i, \mathbf{r}_i)\}$ 5: $\text{st}_i := \text{st}_i \cup \{(\text{act}, \text{msg}, \text{pm}_1, \mathbf{w}_i, \mathbf{r}_i)\}$ 6: return $(\text{pm}_2^i := \mathbf{w}_i, \text{st}_i)$
ShareSign₃(vk, i, pm_2, i, sk_i, st_i) 1: assert $\{(\cdot, \text{pm}_2^i, \cdot) \in \text{st}_i\}$ 2: Pick $(\text{act}, \text{msg}, \text{pm}_1, \mathbf{w}_i, \mathbf{r}_i)$ from st_i with $\text{pm}_2^i = \mathbf{w}_i$ 3: Parse $\text{pm}_1 = (\text{cmt}_j)_j$ and $\text{pm}_2 = (\mathbf{w}_j)_{j \neq i}$ 4: assert $\{\forall j \in \text{act}, \text{cmt}_j = H_{\text{cmt}}(\text{vk}, j, \mathbf{w}_j)\}$ 5: $\text{st}_i := \text{st}_i \setminus \{(\text{act}, \text{msg}, \text{pm}_1, \mathbf{w}_i, \mathbf{r}_i)\}$ 6: $\mu \in \{0, 1\}^{512} := H(\text{tr} \ \text{msg})$ 7: $\mathbf{w} := \sum_{j \in \text{act}} \mathbf{w}_j$ 8: $\mathbf{w}_\top := \text{HighBits}(\mathbf{w}, 2\gamma_2)$ 9: $\tilde{c} \in \{0, 1\}^{2 \cdot \kappa} := H(\mu \ \mathbf{w}_\top)$ 10: $c := \text{SampleInBall}(\tilde{c})$ ▷ Global challenge 11: $\mathbf{m} := \text{RSSRecover}(\text{act})$ 12: $\mathbf{s}_i^{\text{part}} := \sum_{I \in m_i} \mathbf{s}_I$ 13: $\mathbf{z}_i \leftarrow \text{HRej}(c \cdot \mathbf{s}_i^{\text{part}}, r, r', \nu, M; \mathbf{r}_i)$ ▷ Individual response 14: if $\{\mathbf{z}_i = \perp\}$ then $\{\text{abort}\}$ 15: $\mathbf{z}_i^{(1)}, \mathbf{z}_i^{(2)} := \mathbf{z} \in \mathcal{R}_q^\ell \times \mathcal{R}_q^k$ 16: return $(\text{pm}_3^i := \mathbf{z}_i^{(1)}, \text{st}_i)$

Figure 6: Signing procedure in Threshold ML-DSA.

the hint $\mathbf{h}$ that allows verifiers to recover $\text{HighBits}(\mathbf{w}, 2\gamma_2)$ from the value $\mathbf{A} \cdot \mathbf{z}^{(1)} - 2^d \cdot c \cdot \mathbf{t}_\top$ used as an approximation of $\mathbf{w}$. Similarly to ML-DSA, we compute the difference $\delta = \mathbf{w} - (\mathbf{A} \cdot \mathbf{z}^{(1)} - 2^d \cdot c \cdot \mathbf{t}_\top)$ and use the function MakeHint with δ to compute the hint. The high bits of $\mathbf{w}$ are correctly recovered as long as $\|\delta\|_\infty \leq \gamma_2$; we thus restart the protocol if this is not the case. Verification proceeds just as in ML-DSA: the procedure also restarts if $\|\mathbf{z}^{(1)}\|_\infty \geq \gamma_1 - \beta$ or if $\mathbf{h}$ has more than ω 1's.

While this check ensures security in ML-DSA, it is only used for correctness here. Indeed, the hiding property of the rejection step in ShareSign_3 already ensures that $\mathbf{z}$ does not leak any secret information. Thanks to parallel repetitions we can minimize the need for restarts as we ensure that any execution of the protocol will output a signature with probability at least $1/2$.

Combine($\text{vk} = (\text{seed}, \mathbf{t}_\top), \text{act}, \text{msg}, (\text{pm}_k)_{k \in \{1,2,3\}}$)
1: Parse $\text{pm}_2 = (\mathbf{w}_i)_i$ and $\text{pm}_3 = (\mathbf{z}_i^{(1)})_i$ 2: $\mu \in \{0, 1\}^{512} := H(\text{tr} \parallel \text{msg})$ 3: $\mathbf{w} := \sum_{j \in \text{act}} \mathbf{w}_j \triangleright$ Aggregated commitment in $\mathcal{R}_q^k$ 4: $\mathbf{w}_\top := \text{HighBits}(\mathbf{w}, 2\gamma_2)$ 5: $\tilde{c} := H(\mu \parallel \mathbf{w}_\top) \triangleright \tilde{c} \in \{0, 1\}^{2 \cdot \kappa}$ 6: $c := \text{SampleInBall}(\tilde{c}) \triangleright$ Global challenge 7: $\mathbf{z}^{(1)} = \sum_{i \in \text{act}} \mathbf{z}_i^{(1)}$ 8: $\delta := \mathbf{w} - (\mathbf{A} \cdot \mathbf{z}^{(1)} - 2^d \cdot c \cdot \mathbf{t}_\top) \triangleright$ Recover error in $\mathcal{R}_q^k$ 9: $\mathbf{h} := \text{MakeHint}_q(\delta, \mathbf{A} \cdot \mathbf{z}^{(1)} - 2^d \cdot c \cdot \mathbf{t}_\top, 2\gamma_2)$ 10: if $(\ \mathbf{z}^{(1)}\ _\infty \geq \gamma_1 - \beta)$ or $(\ \delta\ _\infty > \gamma_2)$ or $(\ \mathbf{h}\ _1 > \omega)$ then 11: abort 12: return $\text{sig} := (\tilde{c}, \mathbf{z}^{(1)}, \mathbf{h})$
Verify($\text{vk} := (\rho, \mathbf{t}_\top), \text{msg}, \text{sig}$)
1: Parse $\text{sig} := (\tilde{c}, \mathbf{z}^{(1)}, \mathbf{h})$ 2: $\mathbf{A} := \text{ExpandA}(\rho)$ 3: $\mu \in \{0, 1\}^{512} := H(\text{tr} \parallel \text{msg})$ 4: $c := \text{SampleInBall}(\tilde{c})$ 5: $\mathbf{w}'_\top := \text{UseHint}_q(\mathbf{h}, \mathbf{A} \cdot \mathbf{z}^{(1)} - 2^d \cdot c \cdot \mathbf{t}_\top, 2\gamma_2)$ 6: return $\ \mathbf{z}^{(1)}\ _\infty < \gamma_1 - \beta$ and $\tilde{c} = H(\mu \parallel \mathbf{w}'_\top)$ and $\ \mathbf{h}\ _1 \leq \omega$

Figure 7: Combine and verification of Threshold ML-DSA.

3.2 Correctness and security

We prove that Threshold ML-DSA (i) is unforgeable, and (ii) p -terminates. For (i), Threshold ML-DSA is unforgeable if ML-DSA is unforgeable and the MLWE problem used in ML-DSA is hard. For (ii), we introduce a bound B such that for any $\mathbf{m}_i$ returned by RSSRecover and $(\mathbf{u}_1, \mathbf{u}_2) = \sum_{I \in \mathbf{m}_i} \mathbf{s}_I \in \mathcal{R}_q^{\ell+k}$, $\|(\frac{1}{\nu} \cdot c \cdot \mathbf{u}_1, c \cdot \mathbf{u}_2)\|_2 \leq B$ with overwhelming probability for a uniformly sampled challenge c and over the randomness of the key generation.

Let $\varepsilon > 0$. We assume that (r, r', ε) verify the conditions of Lemma 2.4 for secret vectors of norm B , i.e. $2\varepsilon = I_{1-1/\phi^2} \left(\frac{n(k+\ell)+1}{2}, \frac{1}{2} \right)$ for some ϕ , and $r'^2 \geq$

$r^2 + B^2 + 2rB/\phi$, implying Lemma 3.1.

Lemma 3.1. *For the above constraints, we have for any $\|\mathbf{v}\|_2 \leq B$, $R_\infty^\varepsilon(\chi_{\mathbf{z}} \parallel \chi_{\mathbf{r}} + \mathbf{v}) = M$*

Proof. The equation checks out by applying Lemma 2.4 with the data processing inequality for Rényi divergence. $\square$

Theorem 3.1. *Assume that $\varepsilon = o(1)$ and that $\Pr[\text{HighBits}(\mathbf{w}, 2\gamma_2) = \text{HighBits}(\mathbf{w}', 2\gamma_2)] = o(1)$, where $\mathbf{w}, \mathbf{w}'$ are sampled from $[\mathbf{A} \ \mathbf{I}] \sum_{i=1}^T \chi_{\mathbf{r}}$, and $\mathbf{A} \xleftarrow{\$} \mathcal{R}_q^{k \times \ell}$ – remark that this holds notably under the $\text{MLWE}_{q,k,\ell,\sum_{i=1}^T \chi_{\mathbf{r}}}$ assumption. Threshold ML-DSA p -terminates in the ROM for*

$$p = \mathbb{E}_{k \leftarrow \text{Bin}(K, (1 - \frac{1}{M})^T)}[v_k] + o(1)$$

where v_k , for $k \in [0, K]$, is the probability that in at least one of the k combinations we have $\|\mathbf{z}^{(1)}\|_\infty \leq \gamma_1 - \beta$ and $\|\delta\|_\infty \leq \gamma_2$ and the number of 1's in $\mathbf{h}$ is less than ω , when $\mathbf{z}$ is sampled from $\sum_{i \in [T]} \chi_{\mathbf{z}}$ and $\delta = c \cdot \mathbf{z}^{(2)} - c \cdot \mathbf{t}_\perp$.

Proof. First, observe that by design, any signature produced by the scheme is a valid ML-DSA signature. The signature necessarily (i) verifies $\|\mathbf{z}^{(1)}\|_\infty < \gamma_1 - \beta$, and (ii) $\mathbf{h}$ has at most ω 1's. Additionally, the scheme ensures that $\delta := \mathbf{w} - (\mathbf{A} \cdot \mathbf{z}^{(1)} - 2^d \cdot c \cdot \mathbf{t}_\top)$ has an infinity norm at most γ_2 . By applying Lemma 2.1, we obtain $\text{UseHint}_q(\mathbf{h}, \mathbf{A} \cdot \mathbf{z}^{(1)} - 2^d \cdot c \cdot \mathbf{t}_\top, 2\gamma_2) = \mathbf{w}_\top$ during verification. Thus, the verification bounds are satisfied, and the correct challenge c is successfully recovered.

It remains to ensure that the scheme succeeds with a probability of at least p . We prove this with a series of games starting from $\text{Game}_0 := \text{Game}_{\text{TS}}^{\text{TS-corr}}$. We denote $p_i = \Pr[\text{Game}_i() \neq \perp]$.

Game₁. In this game, we assert that for any signing party and parallel session during the second round, we have $\|(\frac{1}{\nu} \cdot c \cdot \mathbf{s}_1^{\text{part}}, c \cdot \mathbf{s}_2^{\text{part}})\|_2 \leq B$, otherwise the game returns 0. We need to add at most $T \cdot K$ such assertions. As B is an overwhelming bound, we obtain by union-bound:

$$p_0 \geq p_1 - TK \cdot \text{negl}(\kappa) = p_1 - \text{negl}(\kappa)$$

Game₂. In this game, we ensure that the parallel signing sessions use different $\tilde{c}$ and $\mathbf{w}$ with different high bits. Otherwise, the game returns 0. It ensures that different challenges are effectively used in every session. As we have at most K independent sessions, we have from the union bound:

$$p_1 \geq p_2 - K^2 \cdot (\Pr[\text{HighBits}(\mathbf{w}, 2\gamma_2) = \text{HighBits}(\mathbf{w}', 2\gamma_2)] + 2^{-2 \cdot \kappa})$$

Game₃. In this game, we sample the challenge c of each session in advance, sample $\mathbf{z}_i \leftarrow \text{Rej}(c \cdot \mathbf{s}_i^{\text{part}}, \chi_{\mathbf{z}}, \chi_{\mathbf{r}}, M)$ for each party. If $\mathbf{z}_i = \perp$, we instead sample $\mathbf{z}_i$ from the distribution $(\mathbf{z} | \text{rej})_{\mathbf{v} = c \cdot \mathbf{s}_i^{\text{part}}}$ of rejected $\mathbf{z}$. Then, we program the random oracle after computing $\mathbf{w} = [\mathbf{A} \ \mathbf{I}] \cdot \mathbf{z} - c \cdot \mathbf{t}$'s, with $\mathbf{z} = \sum_{i \in \text{act}} \mathbf{z}_i$ in each parallel session. The distribution of $\mathbf{w}$ is identical as in **Game₂**, and **Game₂** ensured that we can program c without affecting the probability of winning as no two $\mathbf{w}$ have the same high bits, and then computing $\mathbf{z}$ first in this way is equivalent: $p_2 = p_3$.

Game₄. In this game, we sample the partial responses $\mathbf{z}_i$ using the *Ideal* rejection sampling from Fig. 3. Recall the **Game₁** ensured that $c \cdot \mathbf{s}_i^{\text{part}}$ has a twisted norm bounded by B . We can thus apply the result Lemma 2.3 on the Rényi divergence of rejection sampling, using the intermediate result on hyperballs of Lemma 2.4, since by assumption r, r', B verify its conditions. The Rényi divergence when replacing a single $\mathbf{z}_i$ is $1 + \frac{\varepsilon}{M-1}$.

Let E the event " $\forall (\mathbf{u}_1, \mathbf{u}_2) = \mathbf{s}^{\text{part}}, \|(\frac{1}{\nu} \cdot c \cdot \mathbf{u}_1, c \cdot \mathbf{u}_2)\|_2 \leq B$ ". Formally, we consider the random variable $X = ((\mathbf{sk}_i)_{i \in [N]}, (\mathbf{z}_i^j)_{i \in \text{act}, j \in [K]})$ conditioned on E , where the $\mathbf{z}_i^j$ are the responses output by the parties in each session. After conditioning on $(\mathbf{sk}_i)_{i \in [N]}$ we observe that the $\mathbf{z}_i^j$ become independent. We can thus apply the multiplicativity of the Rényi divergence (Lemma 2.2) to bound the Rényi divergence between X in **Game₃** and **Game₄** by $\left(1 + \frac{\varepsilon}{M-1}\right)^{KT}$.

We then apply the data processing inequality and probability preservation of the Rényi divergence to obtain:

$$\Pr[\text{Game}_4() = \perp \mid E] \cdot \left(1 + \frac{\varepsilon}{M-1}\right)^{KT} \geq \Pr[\text{Game}_3() = \perp \mid E]$$

As E has the same probability in both games, we have:

$$\Pr[\text{Game}_4() = \perp] \cdot \left(1 + \frac{\varepsilon}{M-1}\right)^{KT} \geq \Pr[\text{Game}_3() = \perp]$$

$$\text{Finally, } p_3 \geq 1 + \left(1 + \frac{\varepsilon}{M-1}\right)^{KT} \cdot (p_4 - 1).$$

Conclusion. At this point, acceptance during the protocol occurs independently of the responses $\mathbf{z}_i$ with probability $(1 - \frac{1}{M})$ for each party. Hence, each session succeeds during the protocol with probability $(1 - \frac{1}{M})^T$.

Additionally, we wish for the combination of responses to succeed. We can combine the above results to evaluate the final success probability of the protocol: $p_4 = \mathbb{E}_{k \leftarrow \text{Bin}(K, (1 - \frac{1}{M})^T)}[v_k]$ $\square$

We now state the unforgeability of Threshold ML-DSA.

Theorem 3.2 (Unforgeability). *For any parameter $\phi > 0$, let $Q_s = 2/(K \cdot I_{1-1/\phi^2}(\frac{n(k+\ell)+1}{2}, \frac{1}{2}))$.*

The TSS from Figs. 5 to 7 is TS-UF secure in the ROM, for up to Q_s calls to the individual signing oracles, under the unforgeability of ML-DSA as well as the hardness of the $\text{MLWE}_{q,k,\ell,\chi}$ assumptions for $\chi \in \{\chi_s, \chi_r, \chi_z\}$.

We deduce from Theorem 3.2 that breaking the unforgeability of Threshold ML-DSA is as hard as breaking ML-DSA. Indeed, the MLWE assumptions over χ_r and χ_z are almost statistically verified due to the large width of the hyperballs we use, and in particular are much harder than the assumptions underlying ML-DSA. We provide below an overview of the proof and defer the formal statement and proof to Section F.

Our proof closely follows the approach of [20, Theorem 4], with two main differences: (i) we adapt the argument to account for the slightly different rounding of commitments $\mathbf{w}$, and (ii) instead of reducing to STMSIS [42], we reduce directly to the unforgeability of ML-DSA. The proof proceeds via a sequence of games, gradually transforming the real attack scenario into one where a forgery against the threshold scheme yields a forgery against single-party ML-DSA, under the hardness of the relevant MLWE problems.

- **Game 2.** We show that honest commitments $\mathbf{w}_i$ have high min-entropy before being revealed, so the adversary cannot guess them in advance. This allows us to program the random oracle H_{cmt} lazily, returning random hashes in round 1 and sampling $\mathbf{w}_i$ only in round 2 for honest parties. This step is justified in the random oracle model.
- **Games 3–6.** We sample the challenges c in advance in round 2. The pre-image and collision resistance of H_{cmt} ensure that the adversary has already chosen its $\mathbf{w}_i$ before round 2. With the lazy sampling from the previous game, honest commitments are sampled last, so the aggregated commitment $\mathbf{w} = \sum_i \mathbf{w}_i$ has high min-entropy. We can thus program the random oracle to return the desired challenge c without the adversary noticing.
- **Game 7.** We compute $\mathbf{z}_i$ first, even in case of rejection, and then derive $\mathbf{w}_i = [\mathbf{A} \ \mathbf{I}] \cdot \mathbf{z}_i - c \cdot \mathbf{t}_i^{\text{part}}$. If $\mathbf{z}_i$ is rejected, we sample it from the distribution of rejected $\mathbf{z}$. This is statically equivalent to the previous game.
- **Game 8.** We ensure that the norm of the secret vector $c \cdot \mathbf{s}_i^{\text{part}}$ is at most B , which holds with overwhelming probability.
- **Game 9.** We replace the commitments $\mathbf{w}_i$ by uniform values when the corresponding $\mathbf{z}_i$ is rejected

and will never be output. This change is indistinguishable to the adversary under the MLWE assumptions for the relevant distributions, following [20, Lemma 5].

- **Games 10–11.** We remove the last dependencies on the secrets by replacing the sampling of $\mathbf{z}_i$ with rejection sampling from the ideal functionality (recalled in Fig. 3). The Rényi divergence argument (see Section 2.7) ensures that, for up to Q_s queries, the adversary’s advantage increases by at most 2 bits.
- **Games 12–13.** We replace the public key of Threshold ML-DSA (composed of $\binom{N}{N-T+1}$ secrets) by an ML-DSA public key (a single secret). Since the signing oracles are now independent of secret values, this is justified by the MLWE assumption.

At the end of this sequence, any forgery output by the adversary yields a valid forgery against an ML-DSA key. Thus, the unforgeability of the threshold scheme reduces to that of ML-DSA, under the stated assumptions and in the ROM.

3.3 A Posteriori Key Sharing

In this section, we describe how a user with an existing ML-DSA signing key can split it into shares without affecting verification. To share a secret key $\mathbf{s}$, we sample a short secret sharing (as in the key generation of 3.1) with the difference that the shares will be conditioned on their sum being $\mathbf{s}$.

Algorithmic overview of key sharing (Section D.2). For $S = \binom{N}{N-T+1}$ shares, we use the root lattice $\mathcal{L} = \{(s_1, \dots, s_S) \in \mathbb{Z}^S, \sum s_i = 0\}$ corresponding to the sharings of 0, and the goal of the user is to sample a Gaussian vector in the appropriate coset of $\mathcal{L}$ to share $\mathbf{s}$. While this can be done using the GPV sampler of Gentry et al. [33], using the parameters prescribed in [33] would result in shares with a significant standard deviation, causing ML-DSA verification to reject with very high probability and increasing communication by over 1000×. We address this by using Rényi divergence instead of statistical distance to obtain more compact parameters, as the sharing is used only once during key generation. We present an analysis of the smoothing parameter with Rényi divergence in Section D.2 which is of independent interest.

Security analysis (Section D.3). A caveat is that this key sharing reveals non-trivial information about the secret $\mathbf{s}$. We show in Section D.3 how this is captured by the hint-MLWE problem. However, this hint-MLWE instance is non-standard: the secret $\mathbf{s}$ comes from a

uniform distribution (instead of Gaussian as in [43]), and the hint standard deviation is below the smoothing parameter of $\mathbb{Z}^n$. We prove that for Gaussian secrets, this still reduces efficiently to standard MLWE (Section D.3). We believe our generalized hint-MLWE problem is of independent interest.

Given this sampling algorithm and hint-MLWE problem, we prove that our threshold signature scheme remains secure with a posteriori key sharing - assuming secrets are Gaussian distributed (for the hint-MLWE reduction to MLWE).

Theorem 3.3 (Unforgeability). *The TSS from Figs. 5 to 7 with a posteriori key sharing is TS-UF secure in the ROM under the same conditions as Theorem 3.2 and if the $\text{MLWE}_{q,k,\ell,\chi_{\text{hint}}}$ assumption holds for χ_{hint} the discrete Gaussian of standard deviation $\sigma = 2^{-1/2} \left(\frac{1}{\sigma_{\text{ML-DSA}}^2} + \frac{S-1}{S\sigma_{\text{share}}^2} \right)^{-1/2}$ with $\sigma_{\text{ML-DSA}}, \sigma_{\text{share}}$ the standard deviation of respectively the ML-DSA secret distribution and the secret shares, $S = \binom{N}{N-T+1}$.*

Proof. The proof is identical to the one of Theorem 3.2, except for the games after Game₁₁.

- **Game 11’.** We replace the shares of the secret obtained with GPV, by the ideal Gaussian distribution. The advantage of the adversary is multiplied at most by the Rényi divergence between the two distributions, cf Section D.2.
- **Game 13.** As previously we replace the public key with an ML-DSA public key. Since one of the secret shares remains unknown the information leaked on the secret corresponds to a hint-MLWE, which we reduce to MLWE in Section D.3.

This concludes the proof. $\square$

Discussion on parameters. Using these techniques, we can construct a secret sharing that reduces the unforgeability of Threshold ML-DSA by at most 7 bits (some loss is inevitable, since a short secret sharing always leaks information about the secret key), while increasing the communication cost by at most a factor 10.

While this result is already notable, we are more interested in estimating the security loss incurred by a posteriori key sharing under the constraint that the share distribution matches the standard deviation of the original key generation (and thus preserves the communication cost). However, this parameter choice falls outside our formal hint-MLWE reduction due to the necessary Gaussian standard deviation $\sqrt{2}$ being below the smoothing parameter of $\mathbb{Z}^n$. Note that this does not necessarily indicate a weakness in the key but rather a bias on all distributions. We provide a heuristic analysis as a sanity check: The leakage from $T - 1$ corruptions

yields a hint $\mathbf{h}$ with conditional entropy $H(\mathbf{s}|\mathbf{h})$. For shares with standard deviation $\sqrt{2}$, the effective entropy (obtained by direct computation) equals that of a secret with standard deviation 0.74. The lattice estimator by Albrecht et al. [2] indicates a security loss of at most 12 bits.

In summary, a posteriori secret sharing preserves all aspects of the original scheme (all algorithms but key generation remain unchanged), enables the reuse of an existing ML-DSA secret key, but incurs a loss between 7 to 12 bits of security.

3.4 Parameter selection

To maintain backward compatibility with ML-DSA, we keep the same public parameters listed in Table 2, introducing only the new parameters K , r' , r , and ν (respectively, the number of parallel repetitions, the respective radii of the randomness $\mathbf{r}'_i$ and rejected randomness $\mathbf{z}_i$, and the expansion factor for the first part of the rejected randomness $\mathbf{z}_i^{(1)}$). Following the security analysis in Section 3.2, we first choose ϕ such that $\varepsilon = I_{1-1/\phi^2} \left(\frac{n(k+\ell)+1}{2}, \frac{1}{2} \right) / 2 \leq 1/(KQ_s)$. Then, given an upper bound B on the norm of partial secrets (holding with overwhelming probability), and choosing $r'^2 \geq r^2 + B^2 + \frac{2rB}{\phi}$, we ensure that Threshold ML-DSA retains the same level of security as ML-DSA for up to Q_s signing queries. Concretely, we take $Q_s = 2^{50}$. We set $B = 1.3 \cdot \sqrt{\tau} \cdot \sqrt{n \cdot (k + \ell/\nu^2)} \cdot \sqrt{\text{Var}(\mathcal{U}(-\eta, \eta))} \cdot \sqrt{\left[\binom{N}{T-1} / T \right]}$ as a heuristic overwhelming bound on the norm of partial secrets¹. Moreover, RSSRecover ensures that each party uses at most $\left\lceil \binom{N}{T-1} / T \right\rceil$ secrets in a session for our selected parameters, c.f. Section B. Finally, we select K, r, r' for each possible pair (T, N) , targeting a total success probability of at least $1/2$ for one protocol execution. We evaluate the success probability following Theorem 3.1, evaluating ν numerically.

To showcase the efficiency of our scheme, we provide the communication cost for each threshold $2 \leq T \leq N \leq 6$ of Threshold ML-DSA-44 in Table 3.

Performance scaling rationale. Figure 8 shows that costs grow sharply once T or N increases, so we focus on $N \leq 6$ for practicality. Two effects drive this. (i) Let p be the probability a party accepts one local rejection step (for fixed r, r'). All T parties must accept together, so one joint attempt succeeds with probability about p^T ; the expected number of sequential retries therefore scales

¹We verify experimentally that $\|\mathbf{sc}\|$ behaves like a Gaussian distribution and take a bound $1.3 \cdot \|\mathbf{s}\| \cdot \|\mathbf{c}\|$ corresponding to 13 standard deviations. If this heuristic were wrong, we would only incur a small loss in the number of queries since the bound on $\|\mathbf{sc}\|$ only affects the Rényi divergence of rejection sampling.

$N \setminus T$	2	3	4	5	6
2	10.5 kB				
3	15.8 kB	21.0 kB			
4	15.8 kB	36.8 kB	42.0 kB		
5	15.8 kB	73.5 kB	157.4 kB	84.0 kB	
6	21.0 kB	99.8 kB	388.4 kB	524.8 kB	194.2 kB

Table 3: Communication costs of Threshold ML-DSA-44 for $2 \leq T \leq N \leq 6$, aiming for a success probability $1/2$. Full parameters are given in Section A.

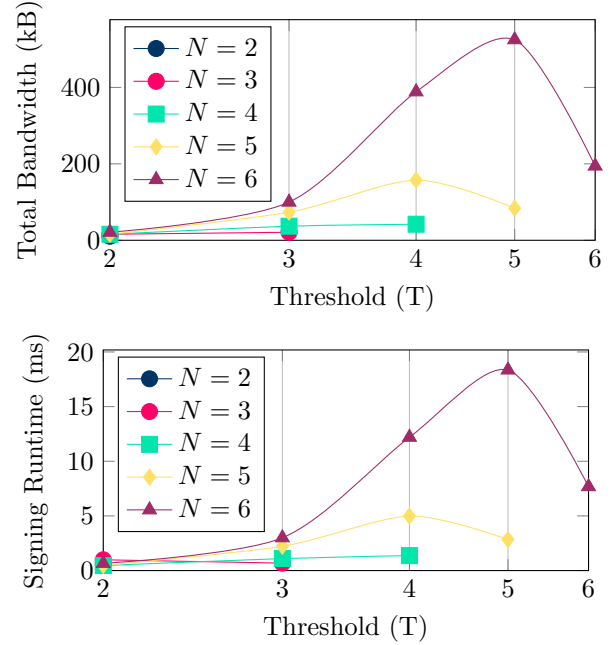


Figure 8: Bandwidth costs (top) and runtime signing costs (bottom) for Threshold ML-DSA-44. Note that the values for $N = 2$ and $N = 3$ overlap for all T .

like p^{-T} . Using $K > 1$ parallel repetitions only trades extra bandwidth for lower latency—it cannot remove the inherent p^T factor. (ii) A partial secret may aggregate up to $\left\lceil \binom{N}{T-1} / T \right\rceil$ base secrets, inflating its norm (captured in B) and forcing larger radii r, r' . This aggregation overhead vanishes at $T = N$, where each party holds exactly one base secret, explaining the mild improvement in that case.

We provide the full parameters of Threshold ML-DSA in Section A, including the parameters (r, r', K, ν) for each security level of ML-DSA as well as the communication costs for each threshold $2 \leq T \leq N \leq 6$. The parameters selected ensure a success probability at least $1/2$ for one protocol execution. For Threshold ML-DSA-44, we include a visual representation of the bandwidth and runtime costs in Fig. 8.

4 Performance Evaluation

We provide implementation details and experimental evaluation of our Threshold ML-DSA scheme. We fully implemented our scheme to ease integration with other systems and libraries. We chose `GoLang` for its popularity in cryptographic implementations and low incidence of security issues [46, 6]. Our implementation of Threshold ML-DSA builds on the code of the CIRCL library [29], that we extended to support our threshold variant, including our secret-sharing functionality.

4.1 Experimental Analysis

We evaluated the performance of Threshold ML-DSA locally, as well as in LAN and WAN settings. All parties ran on a consumer-grade MacBook M3 for local and LAN experiments with 25 GB RAM. We conducted single-threaded local simulations of our scheme to benchmark key generation, signing (across parallel rounds), and verification. Each party’s computation was emulated in a single process, capturing fine-grained timing data for each protocol phase. The runtime (i.e., computational cost) for various (T, N) values is shown in Table 4, for the parameter set 44 of Threshold ML-DSA, as well as visualizations of its bandwidth and runtime in Fig. 8.

(T, N)	KeyGen (ms)	Sign+Combine (ms)	Verify (ms)
(3,3)	0.3669	0.6810	
(2,4)	0.1709	0.4570	
(3,4)	0.2062	1.0961	
(4,4)	0.1655	1.3672	
(3,5)	0.2870	2.2263	0.0306
(4,5)	0.2940	4.9832	
(5,5)	0.1956	2.8453	
(4,6)	0.5016	12.1949	
(6,6)	0.2181	7.6784	

Table 4: Local simulation for the average (mean) costs of Threshold ML-DSA-44. All costs are computed on a MacOS M3 machine and grouped by N . **Green** and **light green** indicate the most and second-most optimal cases per group (N) and per scheme, where applicable.

For our WAN and LAN experiments, we implemented the communication layer of our scheme using the library `libp2p` [56, 55], a modular peer-to-peer networking stack. Each party acts as an independent `libp2p` peer, with its identity embedded as its network address, and communicates via a custom protocol over a direct TCP stream. Parties listen for incoming connections on a specified TCP port, and others connect using multiaddress. After establishing a connection, they exchange messages over a persistent bidirectional `libp2p` stream. This setup ensures execution over a realistic asynchronous, decentralized network, with `libp2p` handling connection man-

agement, peer identity, and stream multiplexing. NAT traversal and relaying were avoided for simplicity, assuming direct TCP connectivity.

LAN experiment We present our results in Table 5. Protocol instances run on separate machines over the same Wi-Fi network. Latency was estimated using an RTT over a TCP connection, yielding 124.166 μ s.

(T, N)	(2,6)	(4,6)	(6,6)
Signing (ms)	0.548	18.216	8.683
TCP packets	≈ 15	≈ 269	≈ 137

Table 5: LAN measurements for Threshold ML-DSA-44. All costs are presented per party.

WAN experiment We present our results in Table 6 for Threshold ML-DSA, using a mesh network topology. The protocol ran on Amazon EC2 ‘‘t2.small’’ instances (Ubuntu 24.04.02, 2.0 GiB RAM), except for a MacOS M3 machine in Taipei, Taiwan, acting as ‘‘leader’’ and initiating connections. Other machines were distributed across Virginia (USA), London (UK), and Seoul (South Korea). Messages were sent via public IPs and in parallel according to best practices in distributed systems.

We report in Table 7 the median latencies (in ms) between the AWS EC2 instances used in our experiments. WAN measurements reflect the slowest communication path (the ‘‘critical path’’ [24]), consistently between Taipei and Virginia, to capture worst-case delays. In real-world cloud deployments, infrastructure optimizations (e.g., load balancing, anycast routing) may reduce delays and overhead.

The measured signing latencies for Threshold ML-DSA are within milliseconds, even in WAN setting over multiple continents, demonstrating our schemes’ practicality in real-world distributed systems. While our scheme require three rounds for a signing operation, these rounds are not executed sequentially in a blocking manner. Instead, messages are sent and processed in parallel across all parties. As a result, the observed latency reflects the most extended communication delay in each round’s critical path—usually between the physically furthest machines—rather than the *sum* of point-to-point delays. Our costs are well under a second, far below a typical TLS handshake timeout of 10-20s [39].

4.2 Applications and Discussion

Our threshold ML-DSA implementation addresses critical security gaps in systems currently using classical threshold signatures that must transition to post-

(T, N)	Locations	Signing (ms)
(2,6)	T – S	27.34
(2,6)	T – V	620.43
(4,6)	T – V – L – L	750.65
(6,6)	T – V – L – L – S – S	659.55

Table 6: WAN signing latency (in ms) for Threshold ML-DSA-44 across different topologies. L = London, S = Seoul, T = Taipei, V = Virginia.

quantum cryptography. The performance characteristics make it particularly suitable for replacing existing ECDSA/RSA threshold deployments.

Post-quantum PKI and TLS infrastructure.

Modern TLS deployments using Content Delivery Networks (CDNs) face critical security challenges when distributing private key material across edge servers. While traditional approaches like Keyless SSL [52] proxy private key operations to origin servers, threshold signatures represent an increasingly attractive alternative that avoids single points of failure [35, 10, 22]. As quantum computers threaten classical cryptography, systems must transition to post-quantum alternatives while maintaining the operational benefits of distributed signing.

Our Threshold ML-DSA scheme addresses this transition by providing ML-DSA-compatible threshold signatures that preserve standardized signature formats—ensuring seamless integration with existing infrastructure. A typical 3-of-5 deployment can tolerate two server failures while maintaining signing capability, with our measurements showing sub-second signing latencies across continents, well within TLS timeout requirements of 10-20s [39].

Cryptocurrency wallet security. Multi-device cryptocurrency wallets [28] represent another application where ML-DSA compatibility is essential for quantum-resistant security. Recent work [48] shows that users prefer fewer, highly reputable parties and lower threshold values for higher availability. Our scheme’s performance characteristics align well with these preferences (typically 2-of-3 or 3-of-5) while providing a desirable stateless design [45, 30] that avoids storage and synchronization issues.

Importantly, our scheme produces standard ML-DSA signatures, ensuring compatibility with blockchain protocols as they transition to post-quantum cryptography without requiring protocol changes.

Critical infrastructure and Tor network. The Tor network [53] serves millions of users daily and requires fresh random values generated by trusted directory au-

thorities². Current distributed random generation techniques are vulnerable to attacks [51], and proposals for threshold-signed consensus documents [37] have stalled due to the absence of practical, standardized threshold schemes.

Our Threshold ML-DSA implementation directly addresses this gap, providing post-quantum security for the long-term integrity of the Tor network. The scheme operates efficiently in the small-threshold setting (we note that the number of directory authorities is approximately 10 [54]; although this number is subject to change over time, it has historically remained small enough for our scheme to be applicable). Moreover, our experimental results demonstrate sub-second signing latencies across continents, comfortably within the requirements for hourly consensus document generation³.

²See <https://spec.torproject.org/rend-spec/shared-random.html>.

³<https://support.torproject.org/glossary/consensus/>

Ethical Considerations

Stakeholder Analysis We identify the following stakeholders impacted by this research:

Research Community: Our work advances the field by providing the first practical ML-DSA-compatible threshold scheme with formal security guarantees and a complete open-source implementation. This research opens new directions in threshold post-quantum cryptography and may inspire innovations in lattice-based distributed systems and standardization efforts.

System Designers: System designers gain access to practical threshold post-quantum signatures, addressing a critical need as distributed systems transition to quantum-resistant cryptography. Our scheme enables secure and efficient distributed signing protocols while maintaining the security properties of classical distributed systems.

End Users: Individuals benefit from improved security and availability of distributed systems. Preserving these properties in the quantum era is essential, though deploying new cryptographic schemes introduces risks of implementation vulnerabilities.

Adversaries: Nation-state actors and cybercriminals will likely analyze our techniques to develop attacks or countermeasures, as occurs with all published cryptographic research.

Standards Bodies: NIST and other standardization organizations may incorporate our findings into future threshold cryptography standards, influencing security infrastructure development.

Ethical Principles Analysis **Beneficence:** This research benefits *researchers* by advancing threshold post-quantum cryptography with formal guarantees; helps *system designers* maintain distributed security during quantum transition; improves security and availability for *end users*; and provides *standards bodies* valuable input for standardization. Although *adversaries* may analyze our techniques, the security improvements for legitimate users far outweigh potential misuse.

Respect for Persons: We support independent verification by the *research community* through transparent publication and open-source release, while respecting *system designers'* autonomy to decide deployment timing.

Justice: Publishing this work ensures equitable access to our results. Future standardization consideration could enhance this access, enabling broader adoption of secure threshold post-quantum signatures.

Respect for Law and Public Interest: This work complies with research standards and regulations while serving public interest through legitimate security improvements and contributing to standards development that protects users.

Risk Assessment and Mitigation Potential Harms:

- *Implementation vulnerabilities:* New cryptographic schemes may contain subtle bugs that lead to security breaches.
- *Cryptanalytic advances:* Future mathematical breakthroughs could compromise our scheme's security.
- *Adversarial misuse:* Attackers may exploit implementation details or side-channel vulnerabilities.

Mitigations Implemented:

- Formal security proofs based on the same assumptions as ML-DSA, ensuring our scheme matches the security of this heavily scrutinized NIST standard.
- Conservative parameter selection based on statistical analysis, reducing the risk of parameter misselection.
- Open-source implementation that enables community audit and verification.

Unmitigated Risks: Residual risks remain that are common to all cryptographic research: potential future cryptanalytic advances and implementation-specific vulnerabilities in production systems.

Decision Rationale We decided to pursue and publish this research after weighing the benefits against the risks. The cryptographic community faces an urgent challenge: existing threshold signatures will become vulnerable to quantum computers, yet these schemes are valuable tools for enhancing user privacy and system security. Organizations seeking to deploy distributed systems—from cryptocurrency platforms to privacy networks like Tor—need quantum-resistant alternatives.

We recognize that publishing novel cryptographic constructions carries risks. Adversaries will scrutinize our techniques, and implementation bugs could lead to vulnerabilities. However, withholding this research would leave critical systems exposed during the quantum transition—a far greater harm. Our formal security proofs, conservative parameters, and open-source implementation provide protection against these risks while enabling community audit and improvement.

Open Science

The implementation artifacts used in this work are available to the community for review and reproducibility. Source code and benchmarking scripts are provided in our public repository at <https://anonymous.4open.science/r/threshold-ml-dsa-4C8A>.

Our repository includes: (i) a complete Threshold ML-DSA implementation based on CIRCL [29], (ii) `libp2p` integration for LAN/WAN experiments, (iii) local benchmarking tools, and (iv) parameter selection scripts. All experimental results can be reproduced using the provided artifact.

Note that our implementations are academic proof-of-concept prototypes suitable for research reproduction but requiring additional security review before production use.

References

- [1] Shweta Agrawal, Damien Stehlé, and Anshu Yadav. Round-optimal lattice-based threshold signatures, revisited. In Mikolaj Bojanczyk, Emanuela Merelli, and David P. Woodruff, editors, *ICALP 2022*, volume 229 of *LIPIcs*, pages 8:1–8:20. Schloss Dagstuhl, July 2022.
- [2] Martin R. Albrecht, Rachel Player, and Sam Scott. On the concrete hardness of learning with errors. Cryptology ePrint Archive, Report 2015/046, 2015.
- [3] Shi Bai, Adeline Langlois, Tancrède Lepoint, Damien Stehlé, and Ron Steinfeld. Improved security proofs in lattice-based cryptography: Using the Rényi divergence rather than the statistical distance. In Tetsu Iwata and Jung Hee Cheon, editors, *ASIACRYPT 2015, Part I*, volume 9452 of *LNCS*, pages 3–24. Springer, Berlin, Heidelberg, November / December 2015.
- [4] Henry Bambury, Hugo Beguinet, Thomas Ricosset, and Éric Sageloli. Polytopes in the fiat-shamir with aborts paradigm. In Leonid Reyzin and Douglas Stebila, editors, *CRYPTO 2024, Part I*, volume 14920 of *LNCS*, pages 339–372. Springer, Cham, August 2024.
- [5] Alexander Bienstock, Leo de Castro, Daniel Escudero, Antigoni Polychroniadou, and Akira Takahashi. Efficient, scalable threshold ml-dsa signatures: An mpc approach, 2025.
- [6] Jenny Blessing, Michael A. Specter, and Daniel J. Weitzner. Cryptography in the wild: An empirical analysis of vulnerabilities in cryptographic libraries. In *Proceedings of the 19th ACM Asia Conference on Computer and Communications Security*, ASIA CCS ’24, page 605–620, New York, NY, USA, 2024. Association for Computing Machinery.
- [7] Katharina Boudgoust and Peter Scholl. Simple threshold (fully homomorphic) encryption from LWE with polynomial modulus. In Jian Guo and Ron Steinfeld, editors, *ASIACRYPT 2023, Part I*, volume 14438 of *LNCS*, pages 371–404. Springer, Singapore, December 2023.
- [8] Ran Canetti, Rosario Gennaro, Steven Goldfeder, Nikolaos Makriyannis, and Udi Peled. UC non-interactive, proactive, threshold ECDSA with identifiable aborts. In Jay Ligatti, Xinming Ou, Jonathan Katz, and Giovanni Vigna, editors, *ACM CCS 2020*, pages 1769–1787. ACM Press, November 2020.
- [9] Guilhem Castagnos, Dario Catalano, Fabien Laguillaumie, Federico Savasta, and Ida Tucker. Bandwidth-efficient threshold EC-DSA revisited: Online/offline extensions, identifiable aborts proactive and adaptive security. *Theor. Comput. Sci.*, 939:78–104, 2023.
- [10] Sofia Celi, Daniel Escudero, and Guilhem Niot. Share the MAYO: thresholdizing MAYO. Cryptology ePrint Archive, Paper 2024/1960, 2024.
- [11] Rutchathon Chairattana-Apirom, Stefano Tessaro, and Chenzhi Zhu. Partially non-interactive two-round lattice-based threshold signatures. In Kai-Min Chung and Yu Sasaki, editors, *ASIACRYPT 2024, Part IV*, volume 15487 of *LNCS*, pages 268–302. Springer, Singapore, December 2024.
- [12] Cloudping. Aws latency monitoring. Cloudping website.
- [13] Daniele Cozzo and Nigel P. Smart. Sharing the LUOV: Threshold post-quantum signatures. In Martin Albrecht, editor, *17th IMA International Conference on Cryptography and Coding*, volume 11929 of *LNCS*, pages 128–153. Springer, Cham, December 2019.
- [14] Elizabeth C. Crites, Chelsea Komlo, and Mary Maller. Fully adaptive Schnorr threshold signatures. In Helena Handschuh and Anna Lysyanskaya, editors, *CRYPTO 2023, Part I*, volume 14081 of *LNCS*, pages 678–709. Springer, Cham, August 2023.
- [15] Luca De Feo and Michael Meyer. Threshold schemes from isogeny assumptions. In Aggelos Kiayias, Markulf Kohlweiss, Petros Wallden, and Vassilis Zikas, editors, *PKC 2020, Part II*, volume 12111 of *LNCS*, pages 187–212. Springer, Cham, May 2020.
- [16] Rafael del Pino, Thomas Espitau, Shuichi Katsumata, Mary Maller, Fabrice Mouhartem, Thomas Prest, Mélissa Rossi, and Markku-Juhani Saarinen. Raccoon. Technical report, National Institute of Standards and Technology, 2023. available at <https://csrc.nist.gov/Projects/pqc-dig-sig/round-1-additional-signatures>.
- [17] Rafael del Pino, Thomas Espitau, Guilhem Niot, and Thomas Prest. Simple and efficient lattice threshold signatures with identifiable aborts. Cryptology ePrint Archive, Paper 2025/871, 2025.
- [18] Rafaël Del Pino, Shuichi Katsumata, Mary Maller, Fabrice Mouhartem, Thomas Prest, and Markku-Juhani O. Saarinen. Threshold raccoon: Practical threshold signatures from standard lattice assumptions. In Marc Joye and Gregor Leander, editors,

- EUROCRYPT 2024, Part II*, volume 14652 of *LNCS*, pages 219–248. Springer, Cham, May 2024.
- [19] Rafael del Pino, Shuichi Katsumata, Guilhem Niot, Michael Reichle, and Kaoru Takemure. Unmasking TRaccoon: A lattice-based threshold signature with an efficient identifiable abort protocol. *Cryptology ePrint Archive*, Paper 2025/849, 2025.
 - [20] Rafael del Pino and Guilhem Niot. Finally! a compact lattice-based threshold signature. In Tibor Jager and Jiaxin Pan, editors, *Public-Key Cryptography – PKC 2025*, pages 169–199, Cham, 2025. Springer Nature Switzerland.
 - [21] Julien Devevey, Omar Fawzi, Alain Passelègue, and Damien Stehlé. On rejection sampling in Lyubashevsky’s signature scheme. In Shweta Agrawal and Dongdai Lin, editors, *ASIACRYPT 2022, Part IV*, volume 13794 of *LNCS*, pages 34–64. Springer, Cham, December 2022.
 - [22] Jack Doerner, Yashvanth Kondi, and Leah Namisa Rosenbloom. Sometimes you can’t distribute random-oracle-based proofs. NIST presentation, 2023.
 - [23] Antonín Dufka, Semjon Kravtšenko, Peeter Laud, and Nikita Snetkov. Trilithium: Efficient and universally composable distributed ML-DSA signing. *Cryptology ePrint Archive*, Paper 2025/675, 2025.
 - [24] Brian Eaton, Jeff Stewart, Jon Tedesco, and N. Cihan Tas. Distributed latency profiling through critical path tracing: Cpt can provide actionable and precise latency analysis. *Queue*, 20(1):40–79, March 2022.
 - [25] Thomas Espitau, Shuichi Katsumata, and Kaoru Takemure. Two-round threshold signature from algebraic one-more learning with errors. In Leonid Reyzin and Douglas Stebila, editors, *CRYPTO 2024, Part VII*, volume 14926 of *LNCS*, pages 387–424. Springer, Cham, August 2024.
 - [26] Thomas Espitau, Guilhem Niot, and Thomas Prest. Flood and submerge: Distributed key generation and robust threshold signature from lattices. In Leonid Reyzin and Douglas Stebila, editors, *CRYPTO 2024, Part VII*, volume 14926 of *LNCS*, pages 425–458. Springer, Cham, August 2024.
 - [27] Thomas Espitau, Alexandre Wallet, and Yang Yu. On gaussian sampling, smoothing parameter and application to signatures. In Jian Guo and Ron Steinfeld, editors, *ASIACRYPT 2023, Part VII*, volume 14444 of *LNCS*, pages 65–97. Springer, Singapore, December 2023.
 - [28] Ittay Eyal. On cryptocurrency wallet design. *Cryptology ePrint Archive*, Report 2021/1522, 2021.
 - [29] Armando Faz-Hernandez and Kris Kwiatkowski. *Introducing CIRCL: An Advanced Cryptographic Library*. Cloudflare, June 2019. Available at <https://github.com/cloudflare/circl>. v1.6.0 Accessed Jan, 2025.
 - [30] Qi Feng, Kang Yang, Kaiyi Zhang, Xiao Wang, Yu Yu, Xiang Xie, and Debiao He. Stateless deterministic multi-party EdDSA signatures with low communication. *Cryptology ePrint Archive*, Report 2024/358, 2024.
 - [31] Marc Fischlin, Aikaterini Mitrokotsa, and Jenit Tomy. BUFFing threshold signature schemes. In Tibor Jager and Jiaxin Pan, editors, *PKC 2025, Part III*, volume 15676 of *LNCS*, pages 137–168. Springer, Cham, May 2025.
 - [32] Rosario Gennaro, Shai Halevi, Hugo Krawczyk, and Tal Rabin. Threshold RSA for dynamic and ad-hoc groups. In Nigel P. Smart, editor, *EUROCRYPT 2008*, volume 4965 of *LNCS*, pages 88–107. Springer, Berlin, Heidelberg, April 2008.
 - [33] Craig Gentry, Chris Peikert, and Vinod Vaikuntanathan. Trapdoors for hard lattices and new cryptographic constructions. In Richard E. Ladner and Cynthia Dwork, editors, *40th ACM STOC*, pages 197–206. ACM Press, May 2008.
 - [34] Kamil Doruk Gür, Jonathan Katz, and Tjrerand Silde. Two-round threshold lattice-based signatures from threshold homomorphic encryption. In Markku-Juhani Saarinen and Daniel Smith-Tone, editors, *Post-Quantum Cryptography - 15th International Workshop, PQCrypto 2024, Part II*, pages 266–300. Springer, Cham, June 2024.
 - [35] Armando Faz Hernandez. Requirements for threshold tls. NIST presentation, 2023.
 - [36] Toke Høiland-Jørgensen, Bengt Ahlgren, Per Hurtig, and Anna Brunstrom. Measuring latency variation in the internet. In *Proceedings of the 12th International on Conference on Emerging Networking EXperiments and Technologies*, CoNEXT ’16, page 473–480, New York, NY, USA, 2016. Association for Computing Machinery.
 - [37] Nicholas Hopper. A threshold signature-based proposal for a shared rng. *Tor Dev Mailing List*, 2014.
 - [38] James Howe, Thomas Prest, Thomas Ricosset, and Mélissa Rossi. Isochronous gaussian sampling: From inception to implementation. In Jintai Ding and

Jean-Pierre Tillich, editors, *Post-Quantum Cryptography - 11th International Conference, PQCrypto 2020*, pages 53–71. Springer, Cham, 2020.

- [39] IBM. Handshake timer, 2023. <https://www.ibm.com/docs/en/zos/3.1.0?topic=considerations-handshake-timer>. Accessed 06/02/24.
- [40] Shuichi Katsumata, Michael Reichle, and Kaoru Takemure. Adaptively secure 5 round threshold signatures from MLWE/MSIS and DL with rewinding. In Leonid Reyzin and Douglas Stebila, editors, *CRYPTO 2024, Part VII*, volume 14926 of *LNCS*, pages 459–491. Springer, Cham, August 2024.
- [41] Irakliy Khaburzaniya, Konstantinos Chalkias, Kevin Lewi, and Harjasleen Malvai. Aggregating and thresholdizing hash-based signatures using STARKs. In Yuji Suga, Kouichi Sakurai, Xuhua Ding, and Kazue Sako, editors, *ASIACCS 22*, pages 393–407. ACM Press, May / June 2022.
- [42] Eike Kiltz, Vadim Lyubashevsky, and Christian Schaffner. A concrete treatment of Fiat-Shamir signatures in the quantum random-oracle model. In Jesper Buus Nielsen and Vincent Rijmen, editors, *EUROCRYPT 2018, Part III*, volume 10822 of *LNCS*, pages 552–586. Springer, Cham, April / May 2018.
- [43] Duhyeong Kim, Dongwon Lee, Jinyeong Seo, and Yongsoo Song. Toward practical lattice-based proof of knowledge from hint-MLWE. In Helena Handschuh and Anna Lysyanskaya, editors, *CRYPTO 2023, Part V*, volume 14085 of *LNCS*, pages 549–580. Springer, Cham, August 2023.
- [44] Chelsea Komlo and Ian Goldberg. FROST: Flexible round-optimized Schnorr threshold signatures. In Orr Dunkelman, Michael J. Jacobson, Jr., and Colin O’Flynn, editors, *SAC 2020*, volume 12804 of *LNCS*, pages 34–65. Springer, Cham, October 2020.
- [45] Yashvanth Kondi, Claudio Orlandi, and Lawrence Roy. Two-round stateless deterministic two-party Schnorr signatures from pseudorandom correlation functions. In Helena Handschuh and Anna Lysyanskaya, editors, *CRYPTO 2023, Part I*, volume 14081 of *LNCS*, pages 646–677. Springer, Cham, August 2023.
- [46] Wenqing Li, Shijie Jia, Limin Liu, Fangyu Zheng, Yuan Ma, and Jingqiang Lin. Cryptogo: Automatic detection of go cryptographic api misuses. In *Proceedings of the 38th Annual Computer Security Applications Conference, ACSAC ’22*, page 318–331, New York, NY, USA, 2022. Association for Computing Machinery.
- [47] Vadim Lyubashevsky. Fiat-Shamir with aborts: Applications to lattice and factoring-based signatures. In Mitsuru Matsui, editor, *ASIACRYPT 2009*, volume 5912 of *LNCS*, pages 598–616. Springer, Berlin, Heidelberg, December 2009.
- [48] Easwar Vivek Mangipudi, Udit Desai, Mohsen Minaei, Mainack Mondal, and Aniket Kate. Uncovering impact of mental models towards adoption of multi-device crypto-wallets. In *Proceedings of the 2023 ACM SIGSAC Conference on Computer and Communications Security, CCS ’23*, page 3153–3167, New York, NY, USA, 2023. Association for Computing Machinery.
- [49] National Institute of Standards and Technology. FIPS 204: Module-Lattice-Based Digital Signature Standard. Federal Information Processing Standard (FIPS) Publication NIST FIPS 204, U.S. Department of Commerce, National Institute of Standards and Technology, August 2024.
- [50] Chris Peikert. An efficient and parallel Gaussian sampler for lattices. In Tal Rabin, editor, *CRYPTO 2010*, volume 6223 of *LNCS*, pages 80–97. Springer, Berlin, Heidelberg, August 2010.
- [51] The Tor Project. Security analysis. Tor Specifications, 2024.
- [52] Douglas Stebila and Nick Sullivan. An Analysis of TLS Handshake Proxying . In *2015 IEEE Trustcom/BigDataSE/IPA*, pages 279–286, Los Alamitos, CA, USA, August 2015. IEEE Computer Society.
- [53] Inc The Tor Project. Tor metrics, 2024. <https://metrics.torproject.org/>.
- [54] Tor Project. Directory authority expectations. https://community.torproject.org/policies/dir-auth/dir_auth_expectations/, 2023. Accessed: 2025-08-25.
- [55] Dennis Trautwein, Aravindh Raman, Gareth Tyson, Ignacio Castro, Will Scott, Moritz Schubotz, Bela Gipp, and Yiannis Psaras. Design and evaluation of ipfs: a storage layer for the decentralized web. In *Proceedings of the ACM SIGCOMM 2022 Conference, SIGCOMM ’22*, page 739–752, New York, NY, USA, 2022. Association for Computing Machinery.
- [56] Dennis Trautwein, Moritz Schubotz, and Bela Gipp. Introducing peer copy - a fully decentralized peer-to-peer file transfer tool. In *2021 IFIP Networking Conference (IFIP Networking)*, pages 1–2, 2021.

A Complete Threshold ML-DSA Parameters

This section provides the complete parameter sets for Threshold ML-DSA, compatible with the different parameter sets of ML-DSA, ML-DSA 44 in Fig. 9, ML-DSA 65 in Fig. 10, and ML-DSA 87 in Fig. 11. These parameter sets are designed to support threshold signatures with up to 6 parties, ensuring efficient signing and verification processes.

(T, N)	r	r'	K	Comm. per party
(2, 2)	252778	252833	2	10.5 kB
(2, 3)	310060	310138	3	15.8 kB
(3, 3)	246490	246546	4	21.0 kB
(2, 4)	305919	305997	3	15.8 kB
(3, 4)	279235	279314	7	36.8 kB
(4, 4)	243463	243519	8	42.0 kB
(2, 5)	285363	285459	3	15.8 kB
(3, 5)	282800	282912	14	73.5 kB
(4, 5)	259427	259526	30	157.4 kB
(5, 5)	239924	239981	16	84.0 kB
(2, 6)	300265	300362	4	21.0 kB
(3, 6)	277014	277139	19	99.8 kB
(4, 6)	268705	268831	74	388.4 kB
(5, 6)	250590	250686	100	524.8 kB
(6, 6)	219245	219301	37	194.2 kB

Figure 9: Parameter sets for Threshold ML-DSA 44, aiming for a success probability $1/2$. All sets use the same multiplicative factor $\nu = 3$.

B Balanced partition of the shares for Replicated Secret Sharing

Recall that we aim at computing a balanced partition of the shares among the party during the signing session, that is for a partition $(m_i)_{i \in \text{act}}$ of the secrets, it minimizes $\max_{i \in \text{act}} |m_i|$. As a general solution when fixing $U = \max_{i \in \text{act}} |m_i|$, we can efficiently look for a solution using a max-flow modelization of the problem. We consider a bipartite graph consisting of (i) the users on one side, (ii) the secrets on the other side, and we add an edge between an user and a secret when said user owns that secret. We can solve the above balanced partition problem as follows:

1. We direct the edges from the users to the secrets.
2. We add a flow U entering the user nodes $i \in \text{act}$.

(T, N)	r	r'	K	Comm. per party
(2, 2)	501495	501613	3	22.9 kB
(2, 3)	540212	540378	5	38.1 kB
(3, 3)	510387	510504	9	68.5 kB
(2, 4)	540212	540378	6	45.7 kB
(3, 4)	506761	506928	20	152.3 kB
(4, 4)	433594	433711	26	198.0 kB
(2, 5)	552371	552575	8	61.0 kB
(3, 5)	552909	553145	62	472.2 kB
(4, 5)	474331	474535	205	1561.3 kB
(5, 5)	425914	426032	78	594.1 kB
(2, 6)	571208	571412	8	61.0 kB
(3, 6)	536793	537058	95	723.5 kB
(4, 6)	488704	488969	804	6123.3 kB
(5, 6)	461324	461529	1200	9139.2 kB
(6, 6)	414896	415013	250	1904.0 kB

Figure 10: Parameter sets for Threshold ML-DSA 65, aiming for a success probability $1/2$. All thresholds (T, N) within a set use the same multiplicative factor $\nu = 6$.

3. We add an exit flow 1 on the secret nodes.

This is represented in Fig. 12. If the max-problem solution covers all the secrets, then we obtain a partition of maximal weight K . We can find the optimal weight K by testing values in increasing order.

For our concrete parameters, we consider $N \leq 6$, for which we can easily verify that the optimal assignments verify $K = \left\lceil \binom{N}{T-1} / T \right\rceil$.

Concrete implementation. For simplicity and efficiency, we do not wish to implement a max-flow solver in the Threshold ML-DSA code. Instead, we first observe that when $T = N$, then the partition is easily obtained as each signing party possesses exactly one secret. For the other cases $2 \leq T < N \leq 6$, we explicit an optimal solution for $\text{act} = \{1, \dots, T\}$ and each values (T, N) . We obtain a partition for all other possible signing set act by symmetry by permuting the index of parties. The idea is formalized in Algorithm 6.

C Distributed Key Generation

The key generation procedure described in Fig. 5 relies on a trusted dealer that samples all secrets $\mathbf{s}_S$ for the RSS scheme. This appendix describes how to distribute the key generation process across multiple parties without compromising security.

C.1 Overview

We describe the distributed key generation protocol at a high level. The main requirements for a secure DKG

(T, N)	r	r'	K	Comm. per party
(2, 2)	503119	503192	3	31.1 kB
(2, 3)	631601	631703	4	41.5 kB
(3, 3)	483107	483180	6	62.2 kB
(2, 4)	632903	633006	4	41.5 kB
(3, 4)	551752	551854	11	114.1 kB
(4, 4)	487958	488031	14	145.2 kB
(2, 5)	607694	607820	5	51.9 kB
(3, 5)	577400	577546	26	269.6 kB
(4, 5)	518384	518510	70	725.8 kB
(5, 5)	468214	468287	35	362.9 kB
(2, 6)	665106	665232	5	51.9 kB
(3, 6)	577541	577704	39	404.4 kB
(4, 6)	517689	517853	208	2156.6 kB
(5, 6)	479692	479819	295	3058.6 kB
(6, 6)	424124	424197	87	902.0 kB

Figure 11: Parameter sets for Threshold ML-DSA 87, aiming for a success probability $1/2$. All thresholds (T, N) within a set use the same multiplicative factor $\nu = 7$.

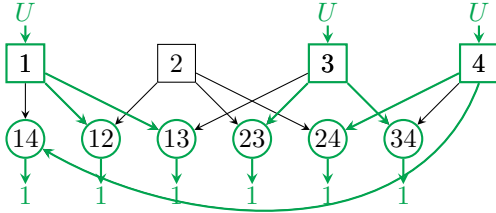


Figure 12: Illustration of the replicated secret sharing with $(N, T) = (4, 3)$: users are on top (rectangles), shares at the bottom (circles). Balanced assignment of shares to active users is performed via a max-flow solving algorithm (in green).

for threshold ML-DSA are: (1) secrets used by honest parties must be sampled from the proper distribution to ensure they remain short enough for safe use in the rejection procedure (c.f. Game 8 of Theorem 3.2), and (2) the final public key must be pseudorandom to maintain unforgeability (c.f. Games 12-13 of Theorem 3.2).

Our distributed key generation protocol eliminates the need for a trusted dealer by having parties collaboratively generate the secrets $\mathbf{s}_S$ through a multi-round commitment-reveal scheme. The key insight is that each subset S of $N - T + 1$ parties can collectively generate their corresponding secret $\mathbf{s}_S$ using a shared random oracle, ensuring that no single party learns secrets for groups they don't belong to while maintaining the secret distributions as the centralized approach for honest parties.

The protocol consists of four communication rounds plus a final aggregation step:

Alg. 6 $\text{RSSRecover}(\text{act}) \rightarrow \mathcal{P}(S)^{\text{act}}$

```

1: if  $T = N$  then
2:   return  $(m_i := (\{i\}))_{i \in \text{act}}$ 
3: if  $T = 2 \wedge N = 3$  then
4:    $\text{sh} := \{0 : \{01, 02\}, 1 : \{12\}\}$ 
5: else if  $T = 2 \wedge N = 4$  then
6:    $\text{sh} := \{0 : \{013, 023\}, 1 : \{012, 123\}\}$ 
7: else if  $T = 3 \wedge N = 4$  then
8:    $\text{sh} := \{0 : \{01, 03\}, 1 : \{12, 13\}, 2 : \{23, 02\}\}$ 
9: else if  $T = 2 \wedge N = 5$  then
10:   $\text{sh} := \{0 : \{0134, 0234, 0124\}, 1 : \{1234, 0123\}\}$ 
11: else if  $T = 3 \wedge N = 5$  then
12:   $\text{sh} := \{0 : \{034, 013, 014, 023\}, 1 : \{012, 123, 124, 134\}, 2 : \{234, 024\}\}$ 
13: else if  $T = 4 \wedge N = 5$  then
14:   $\text{sh} := \{0 : \{01, 03, 04\}, 1 : \{12, 13, 14\}, 2 : \{23, 02, 24\}, 3 : \{34\}\}$ 
15: else if  $T = 2 \wedge N = 6$  then
16:   $\text{sh} := \{0 : \{02345, 01235, 01245\}, 1 : \{12345, 01234, 01345\}\}$ 
17: else if  $T = 3 \wedge N = 6$  then
18:   $\text{sh} := \{0 : \{0134, 0124, 0135, 0345, 0125\}, 1 : \{0145, 1345, 1235, 1234, 1245\}, 2 : \{0235, 0245, 0234, 0123, 2345\}\}$ 
19: else if  $T = 4 \wedge N = 6$  then
20:   $\text{sh} := \{0 : \{014, 023, 015, 012, 045\}, 1 : \{135, 134, 125, 145, 124\}, 2 : \{245, 024, 235, 234, 025\}, 3 : \{034, 013, 123, 345, 035\}\}$ 
21: else if  $T = 5 \wedge N = 6$  then
22:   $\text{sh} := \{0 : \{01, 02, 05\}, 1 : \{12, 13, 15\}, 2 : \{23, 24, 25\}, 3 : \{03, 34, 35\}, 4 : \{45, 04, 14\}\}$ 
23:  $\phi := \{i : i\}_{i \in [N]}$   $\triangleright$  Define a permutation of the user indexes
24:  $i_1 := 0, i_2 := T$ 
25: for  $j \in [N]$  do
26:   if  $j \in \text{act}$  then
27:      $\phi[i_1] = j$ 
28:      $i_1 = i_1 + 1$ 
29:   else
30:      $\phi[i_2] = j$ 
31:      $i_2 = i_2 + 1$ 
32: for  $i \in [T]$  do  $\triangleright$  Translate the ideal sharing for act
33:   $m_{\phi(i)} = \{\phi(u) \mid u \in \text{sh}[i]\}$ 
return  $(m_i)_{i \in \text{act}}$ 

```

1. **Shared secret establishment:** Group leaders establish shared secrets K_S for their respective groups using secure channels, and all parties commit to random strings r_i .
2. **Commitment reveal:** All parties reveal their committed strings to establish global randomness $R = r_1 \parallel \dots \parallel r_N$.

3. **Secret derivation and PK commitment:** Group leaders derive their secrets $\mathbf{s}_S = H_{\text{keygen}}(S, K_S, R)$, compute corresponding partial public keys $\mathbf{t}_S$, and commit to them.
4. **Public key reveal:** Group leaders reveal their partial public keys for verification and aggregation.
5. **Aggregation:** All parties compute the final public key $\mathbf{vk} = (\rho, \mathbf{t}_\top)$ where $\mathbf{t}_\top$ is derived from the sum of all partial public keys.

A key advantage of this approach is that explicit share distribution is unnecessary: each party can directly compute which secrets $\mathbf{s}_S$ they need based on the deterministic group membership structure and the shared secrets they received in round 1.

C.2 Protocol Description

Let $\mathcal{S} = \{S \subset [N] : |S| = N - T + 1\}$ denote the collection of all subsets of size $N - T + 1$. For each $S \in \mathcal{S}$, the parties in S will collectively generate the corresponding secret $\mathbf{s}_S$.

We provide pseudocode in Fig. 13.

C.3 Security Analysis

The security of the distributed key generation protocol relies on the following properties:

Correctness. The protocol produces the same distribution of secret keys as the centralized version for honest parties. Each secret $\mathbf{s}_S$ is generated as $H_{\text{keygen}}(S, K_S, R)$ where K_S is known only to parties in S , and R contains contributions from all parties. As R is sampled with a commit-and-reveal approach, it ensures that $\mathbf{s}_S$ is properly sampled in χ_s by the random oracle, even if K_S is maliciously chosen.

Public-key pseudorandomness. The security of Threshold ML-DSA hinges on the pseudorandomness of the final public key $\mathbf{vk}$. At least one group of parties is composed only of honest members, ensuring that their corresponding secret remains hidden for malicious parties. Thus, the corresponding partial keys appears pseudorandom to them. The commit-and-reveal approach for aggregating the final key combined with transcript verification consequently ensures that the final public key $\mathbf{vk}$ is also pseudorandom and that adversaries cannot bias it to compromise unforgeability.

C.4 Implementation Considerations

Communication Rounds. The protocol requires exactly 4 communication rounds plus a final local aggregation step: (1) shared secret distribution and commitment

broadcast, (2) commitment reveal, (3) transcript signing and partial public key commitment broadcast, (4) transcript verification and partial public key reveal, and (5) local aggregation to compute the final public key. The transcript signing and verification mechanism ensures that all parties agree on the same protocol execution before contributing to the final public key.

Transcript Integrity. Each party signs the full protocol transcript (rounds 1-3) using their long-term signing key in round 4. During aggregation, we verify that everyone signed the same transcript. This prevents attacks where malicious parties would impersonate an honest party and replace their contribution in the protocol to bias the final key.

Leader Selection. Each group S has a deterministic leader (the party with smallest index in S). The leader has the role to sample the shared secret of the group, as well as compute the corresponding partial public key, and distribute them.

Shared Secret Distribution. Leaders distribute shared secrets using secure point-to-point channels. This can be implemented using standard authenticated encryption or by leveraging existing secure communication infrastructure.

This distributed approach maintains the same security guarantees as the trusted dealer version while eliminating single points of failure in key generation, making it suitable for fully decentralized applications.

D A posteriori secret sharing: Details and proofs

D.1 Lattice preliminaries

We denote $\rho_{\sqrt{\Sigma}, \mathbf{c}}$ the mass function over $\mathbb{R}^n$ of the Gaussian distribution centered in $\mathbf{c}$: $\rho_{\sqrt{\Sigma}, \mathbf{c}}(\mathbf{x}) = \exp(-(\mathbf{x} - \mathbf{c})^t \cdot \Sigma^{-1} \cdot (\mathbf{x} - \mathbf{c}))$ for $\Sigma \in \mathbb{R}^{n \times n}$ positive definite. For $\sigma \in \mathbb{R}^+$, we note $\rho_\sigma = \rho_{\sqrt{\sigma^2 I}, \mathbf{0}}$.

A lattice is a discrete subgroup of $\mathbb{R}^n$. The dual of a lattice L , noted $L^\vee$, is the set $L^\vee = \{\mathbf{x} \in \text{Span}_{\mathbb{R}}(L) \mid \forall \mathbf{v} \in L, \langle \mathbf{x}, \mathbf{v} \rangle \in \mathbb{Z}\}$. Discretizing the Gaussian mass function $\rho_{\sqrt{\Sigma}, \mathbf{c}}$ over L and normalizing to 1, we obtain the discrete Gaussian distribution $D_{L, \sqrt{\Sigma}, \mathbf{c}}$. For any lattice L , and any real $\varepsilon > 0$, the smoothing parameter $\eta_\varepsilon(L)$ is the smallest real $s > 0$ such that $\frac{1}{s^n} \rho_{1/(\sqrt{2\pi}s)}(L^\vee \setminus \{0\}) \leq \varepsilon$.

D.2 Sampling the key shares

We first describe in more details how to sample a Gaussian vector from the lattice $\mathcal{L} = \{(\mathbf{s}_1, \dots, \mathbf{s}_S), \sum \mathbf{s}_i = 0\}$ while minimizing its standard deviation.

DKGRound1(N, T, i) $\rightarrow$ ($\text{pm}_1^i, \text{st}_i$) <pre> 1: $\text{st}_i \leftarrow \emptyset$ 2: for $S \in \mathcal{S}$ such that $i = \min(S)$ do 3: Sample $K_S \leftarrow \{0, 1\}^{256}$ 4: for $j \in S \setminus \{i\}$ do 5: Send K_S securely to party j 6: $\text{st}_i \leftarrow \text{st}_i \cup \{(S, K_S)\}$ 7: Sample $r_i \leftarrow \{0, 1\}^{256}$ 8: Compute $c_i = H_{\text{commit}}(i, r_i)$ 9: $\text{st}_i \leftarrow \text{st}_i \cup \{(r_i, c_i)\}$ 10: return ($\text{pm}_1^i := c_i, \text{st}_i$) </pre> DKGRound2($\text{pm}_1, i, \text{st}_i$) $\rightarrow$ ($\text{pm}_2^i, \text{st}_i$) <pre> 1: assert{ $(r_i, c_i) \in \text{st}_i$ for some r_i, c_i } 2: Parse $\text{pm}_1 = (c_j)_{j \in [N]}$ 3: Pick (r_i, c_i) from st_i with $c_i = \text{pm}_1^i$ 4: $\text{st}_i \leftarrow \text{st}_i \cup \{(\text{pm}_1)\}$ 5: return ($\text{pm}_2^i := r_i, \text{st}_i$) </pre> DKGRound3($\text{pm}_2, i, \text{st}_i$) $\rightarrow$ ($\text{pm}_3^i, \text{st}_i$) <pre> 1: assert{ $(\text{pm}_1) \in \text{st}_i$ for some pm_1 } 2: Parse $\text{pm}_1 = (c_j)_{j \in [N]}$ and $\text{pm}_2 = (r_j)_{j \in [N]}$ 3: Assert $\forall j \in [N], c_j = H_{\text{commit}}(j, r_j)$ 4: $R = r_1 \ r_2 \ \dots \ r_N$ 5: $\rho \leftarrow H_{\text{seed}}(R)$ 6: $\mathbf{A} := \text{ExpandA}(\rho)$ 7: $\text{pm}_3^i \leftarrow \emptyset$ 8: for $S \in \mathcal{S}$ such that $i = \min(S)$ do $\triangleright$ Party i is leader of group S 9: Pick (S, K_S) from st_i 10: Compute $\mathbf{s}_S = H_{\text{keygen}}(S, K_S, R)$ where $H_{\text{keygen}} : \mathcal{S} \times \{0, 1\}^{256} \times \{0, 1\}^{256 \cdot N} \rightarrow \chi_{\mathbf{s}}$ 11: Compute $\mathbf{t}_S := [\mathbf{A} \quad \mathbf{I}] \cdot \mathbf{s}_S$ 12: Compute $c_S^{\text{pk}} = H_{\text{commit}}(S, \mathbf{t}_S)$ 13: $\text{pm}_3^i \leftarrow \text{pm}_3^i \cup \{(S, c_S^{\text{pk}})\}$ 14: $\text{st}_i \leftarrow \text{st}_i \cup \{(S, \mathbf{s}_S, \mathbf{t}_S, c_S^{\text{pk}})\}$ 15: return ($\text{pm}_3^i, \text{st}_i$) </pre> DKGRound4($\text{pm}_3, i, \text{st}_i$) $\rightarrow$ ($\text{pm}_4^i, \text{st}_i$) <pre> 1: Parse $\text{pm}_3 = \bigcup_{j \in [N]} \text{pm}_3^j$ 2: Extract $\{(S, c_S^{\text{pk}}) : S \in \mathcal{S}, \min(S) \in [N]\}$ from pm_3 </pre>	<pre> 3: transcript $\leftarrow \text{pm}_1 \ \text{pm}_2 \ \text{pm}_3$ from st_i 4: $\text{pm}_4^i \leftarrow \emptyset$ 5: for $(S, c_S^{\text{pk}}) \in \text{pm}_3$ do 6: Let $j = \min(S)$ 7: if $i = j$ then 8: Pick $(S, \mathbf{s}_S, \mathbf{t}_S, c_S^{\text{pk}})$ from st_i 9: $\text{pm}_4^i \leftarrow \text{pm}_4^i \cup \{(S, \mathbf{t}_S)\}$ 10: else 11: if $i \in S$ then 12: Pick (S, K_S) from st_i 13: Pick (pm_1) from st_i to get R 14: Compute $\mathbf{s}'_S = H_{\text{keygen}}(S, K_S, R)$ 15: Compute $\mathbf{t}'_S = [\mathbf{A} \quad \mathbf{I}] \cdot \mathbf{s}'_S$ 16: Assert $c_S^{\text{pk}} = H_{\text{commit}}(S, \mathbf{t}'_S)$ 17: $\partial_i \leftarrow \{(S, \mathbf{t}_S) : (S, \mathbf{t}_S) \in \text{pm}_4^i\} \triangleright$ Party i's partial output 18: $\sigma_i \leftarrow \text{Sign}(\text{sk}_i^{\text{long}}, \text{transcript} \ \partial_i)$ 19: $\text{pm}_4^i \leftarrow \text{pm}_4^i \cup \{\sigma_i\}$ 20: $\text{st}_i \leftarrow \text{st}_i \cup \{(\text{pm}_3)\}$ 21: return ($\text{pm}_4^i, \text{st}_i$) </pre> DKGAggregate($\text{pm}_2, \text{pm}_3, \text{pm}_4$) $\rightarrow$ vk <pre> 1: assert{ $\forall i, \text{pm}_4^i \neq \perp$ } 2: Extract $\{\sigma_j : j \in [N]\}$ from pm_4 3: transcript $\leftarrow \text{pm}_1 \ \text{pm}_2 \ \text{pm}_3$ 4: Parse $\text{pm}_4 = \bigcup_{j \in [N]} \text{pm}_4^j$ where each pm_4^j contains $(S, \mathbf{t}_S)$ and σ_j 5: for $j \in [N]$ do 6: $\partial_j \leftarrow \{(S, \mathbf{t}_S) : (S, \mathbf{t}_S) \in \text{pm}_4^j, j = \min(S)\}$ 7: if $\text{Verify}(\text{vk}_j^{\text{long}}, \text{transcript} \ \partial_j, \sigma_j) = 0$ then 8: return $\perp$ 9: $\mathbf{t} \leftarrow \mathbf{0}$ 10: for $(S, \mathbf{t}_S) \in \text{pm}_4$ such that $(S, \mathbf{t}_S)$ is not a signature do 11: Find $(S, c_S^{\text{pk}}) \in \text{pm}_3$ 12: Assert $c_S^{\text{pk}} = H_{\text{commit}}(S, \mathbf{t}_S)$ 13: $\mathbf{t} \leftarrow \mathbf{t} + \mathbf{t}_S$ 14: Pick (pm_1) from st_i to get $R = r_1 \ r_2 \ \dots \ r_N$ 15: $\rho \leftarrow H_{\text{seed}}(R)$ 16: $(\mathbf{t}_{\top}, \mathbf{t}_{\perp}) := \text{Power2Round}(\mathbf{t}, d)$ 17: $tr \in \{0, 1\}^{256} := H(\rho \ \mathbf{t}_{\top})$ 18: return ($\text{vk} := (\rho, \mathbf{t}_{\top})$) </pre>
---	---

Figure 13: Distributed key generation for Threshold ML-DSA.

Let $A_S = \{\mathbf{x} \in \mathbb{Z}^S, \sum x_i = 0\}$ be the root lattice of dimension $S - 1$, it is clear that $\mathcal{L} = A_S^m$ where $m = 256(k + \ell)$ is the size of the vector $\mathbf{s}$. We are thus interested in computing the smoothing parameter $\eta_{\varepsilon}(A_n)$. As this is prior work we do not need to use the definition of the

smoothing parameter but only some properties that it verifies:

Lemma D.1 (Adapted from Peikert [50], Theorem 3.1). *Let Σ_1, Σ_2 be definite positive such that $\Sigma_b \geq \eta_{\varepsilon}(\mathbb{Z}^n)$, $b \in \{1, 2\}$, and $\mathbf{c}_1, \mathbf{c}_2 \in \mathbb{Z}^n$. Then for any $\mathbf{x} \in$*

$\mathbb{Z}^n$:

$$\left| \frac{\Pr[X + Y = \mathbf{x}]}{D_{\mathbb{Z}^n, \sqrt{\Sigma_1 + \Sigma_2}, \mathbf{c}_1 + \mathbf{c}_2}(\mathbf{x})} - 1 \right| \leq 3\varepsilon$$

Where $X \leftarrow D_{\mathbb{Z}^n, \sqrt{\Sigma_1}, \mathbf{c}_1}$, and $Y \leftarrow D_{\mathbb{Z}^n, \sqrt{\Sigma_2}, \mathbf{c}_2}$.

Proof. From [50] Theorem 3.1, we have that $\Pr[X + Y = \mathbf{x}] \in \left[\frac{1-\varepsilon}{1+\varepsilon}, \frac{1+\varepsilon}{1-\varepsilon} \right] D_{\mathbb{Z}^n, \sqrt{\Sigma_1 + \Sigma_2}, \mathbf{c}_1 + \mathbf{c}_2}(\mathbf{x})$, and for $\varepsilon \leq 1/3$ we have $1 - 3\varepsilon \leq \frac{1-\varepsilon}{1+\varepsilon} \leq \frac{1+\varepsilon}{1-\varepsilon} \leq 1 + 3\varepsilon$. $\square$

Lemma D.2 (Adapted from [27] Theorem 5). For any $S > 0$, let $q = \lceil (S-1)/9 \rceil$, $s \geq \sqrt{\frac{9}{8}} \eta_\varepsilon(\mathbb{Z}^S)$, and vector $\mathbf{u} \in \text{Span}(A_S)$. The algorithm SampleD_s introduced in [27] is such that for any $\mathbf{v} \in A_S$:

$$\left| \frac{\text{SampleD}_s(\mathbf{v})}{D_{A_S, s, \mathbf{u}}(\mathbf{v})} - 1 \right| \leq 1 + 2(q+1)\varepsilon$$

Lemma D.3 ([50] Lemma 2.5). For any $S > 0$, $\varepsilon > 0$, we have:

$$\eta_\varepsilon(\mathbb{Z}^S) \leq \sqrt{\frac{\log(2S(1+1/\varepsilon))}{\pi}}$$

We will also use some additional properties of the Rényi divergence:

Lemma D.4 ([27] Lemma 3). Let χ and χ' be two distributions with the same support S . If $\exists \delta$ such that $\forall v \in S$:

$$\left| \frac{\chi(v)}{\chi'(v)} - 1 \right| \leq \delta$$

Then $\forall \alpha > 1$:

$$R_\alpha(\chi || \chi') \leq 1 + \frac{\alpha \delta^2}{2}$$

and

$$R_\infty(\chi || \chi') \leq 1 + \delta$$

We now observe that $\mathcal{L} = \{(\mathbf{s}_1, \dots, \mathbf{s}_S) \in \mathcal{R}^{k+\ell}, \sum \mathbf{s}_i = 0\} = A_S^m$ since each coordinate can be considered independently. From all these properties we get:

Lemma D.5. For any $S > 0$, let $q = \lceil (S-1)/9 \rceil$, $s \geq \sqrt{\frac{9}{8}} \sqrt{\frac{\log(2S(1+1/\varepsilon))}{\pi}}$, $\mathcal{L} = \{(\mathbf{s}_1, \dots, \mathbf{s}_S) \in \mathcal{R}^{k+\ell}, \sum \mathbf{s}_i = 0\}$. There exists an efficient algorithm SampleD_s such that, $\forall \mathbf{u} \in \text{Span}(\mathcal{L})$, $\forall \alpha \in (1, \infty)$:

$$R_\alpha(\text{SampleD}_s || D_{\mathcal{L}, s, \mathbf{u}}) \leq 1 + 4\alpha(q+1)^2 \varepsilon^2 \cdot m/2 + o(\varepsilon^2 \cdot m)$$

with $m = 256(k+\ell)$. In particular if we take $\varepsilon = O(\sqrt{m})$ then we get:

$$R_\alpha(\text{SampleD}_s || D_{\mathcal{L}, s, \mathbf{u}}) = O(1)$$

Proof. Using the lemmata from this section we directly obtain a similar statement for the lattice A_S :

$$R_\alpha(\text{SampleD}'_s || D_{A_S, s, \mathbf{u}}) \leq 1 + 4\alpha(q+1)^2 \varepsilon^2/2$$

Where $\text{SampleD}'_s$ is a sampler for A_S . Using the fact that $\mathcal{L} = A_S^m$ and the multiplicativity of Lemma 2.2, we get for SampleD_s the algorithm that executes m instances of $\text{SampleD}'_s$, and $\mathbf{u} = (\mathbf{u}_1, \dots, \mathbf{u}_m)$:

$$\begin{aligned} R_\alpha(\text{SampleD}_s || D_{\mathcal{L}, s, \mathbf{u}}) &\leq \prod R_\alpha(\text{SampleD}'_s || D_{A_S, s, \mathbf{u}_i}) \\ &\leq (1 + 4\alpha(q+1)^2 \varepsilon^2/2)^m \\ &\leq 1 + 4\alpha(q+1)^2 \varepsilon^2 \cdot m/2 + o(\varepsilon^2 \cdot m) \end{aligned}$$

$\square$

We now have a sampler that can sample over any coset of $\mathcal{L}$ (securely only once since we use Rényi divergence)

for any standard deviation $s \simeq \sqrt{\frac{9}{8}} \sqrt{\frac{\log(2S\sqrt{m})}{\pi}}$ which is much better than what we would get with a statistical distance argument.

This sampler can, however, not be used to directly sample from the key share distribution that we desire because the vectors $(\mathbf{s}_1, \dots, \mathbf{s}_S)$ such that $\sum \mathbf{s}_i = \mathbf{s}$ are not in the span of $\mathcal{L}$. To remedy this we will construct a vector $\mathbf{v} = (\mathbf{v}_1, \dots, \mathbf{v}_S)$ such that $\sum \mathbf{v}_i = \mathbf{s}$, and decompose it into $\mathbf{v} = \mathbf{v}^\top + \mathbf{v}^\perp$, where $\mathbf{v}^\top \in \text{Span}(\mathcal{L})$, and $\mathbf{v}^\perp \in \text{Span}(\mathcal{L})^\perp$. We can then sample $\mathbf{x}$ from $D_{\mathcal{L}, s, -\mathbf{v}^\top}$ and output $\mathbf{z} = \mathbf{x} + \mathbf{v}$, which verifies $\sum \mathbf{z}_i = \mathbf{s}$ and $\forall i \in [S], \|\mathbf{z}_i\| \leq \|\mathbf{z}_i + \mathbf{v}_i^\top\| + \|\mathbf{v}_i^\perp\|$. Since the first component comes from $S_{s, \mathcal{L}, 0}$ it is distributed exactly as in our original key generation, to keep compactness we thus need to construct $\mathbf{v}$ such that $\forall i \in [S]$ the norm of $\mathbf{v}_i^\perp$ is negligible compared to the first component.

We can observe that the span of the set S of all vectors $\mathbf{v}$ such that $\sum \mathbf{v}_i = \mathbf{s}$ is an hyperplane parallel to $\text{Span}(\mathcal{L})$ and that the vector $(\mathbf{s}/S, \dots, \mathbf{s}/S)$ is in S and orthogonal to $\text{Span}(\mathcal{L})$. From this we get that for any $\mathbf{v} \in S$ (e.g. $(\mathbf{s}, 0, \dots, 0)$) the norm of $\mathbf{v}_i^\perp$ is $\|\mathbf{s}\|/S$, which quickly becomes negligible as $T = \binom{N}{N-T+1}$ grows.

D.3 Security Analysis

We first introduce a generalization of the hint-MLWE problem from [43, 26], where the hint matrix $\mathbf{B}$ can be non integral, and the hints are Gaussian samples centered around the secret part $\mathbf{B}\mathbf{s}$. We prove that it enjoys a similar reduction to MLWE for Gaussian distributed secrets. In fact the problem and proof of [43, 26] can be viewed as specific instantiations of the one presented here.

Definition D.1 (hint-MLWE). Let k, ℓ, q, m be integers with $m \geq (k+\ell)$, χ be a probability distribution over $\mathcal{R}^{k+\ell}$, $\Sigma \in \mathbb{Z}^{mn \times mn}$ be definite positive, and

$\mathbf{B} \in \mathcal{R}^{m \times (k+\ell)}$. The advantage of an algorithm $\mathcal{A}$ in solving the hint-MLWE $_{q,k,\ell,\chi,\sqrt{\Sigma},\mathbf{B}}$ problem is defined as:

$$|\Pr[\mathcal{A}(\mathbf{A}, [\mathbf{A} \quad \mathbf{I}_k] \cdot \mathbf{s}, \mathbf{z}) = 1] - \Pr[\mathcal{A}(\mathbf{A}, \mathbf{u}, \mathbf{z}) = 1]|$$

where $\mathbf{A} \xleftarrow{\$} \mathcal{R}_q^{k \times \ell}$, $\mathbf{u} \xleftarrow{\$} \mathcal{R}_q^k$, $\mathbf{s} \leftarrow \chi$, $\mathbf{z} \leftarrow D_{\mathcal{R}^m, \sqrt{\Sigma}, \mathbf{B}\mathbf{s}}$. The hint-MLWE assumption states that any efficient adversary $\mathcal{A}$ has a negligible advantage against hint-MLWE.

In particular if we consider $\mathbf{B}$ limited to integer coefficients, we obtain the hint-MLWE instance of [26].

Lemma D.6. Let $\sigma > 0$, for any $0 < l < m$, let $\Sigma \in \mathbb{Z}^{nl \times nl}$ be definite positive, let $B \in \mathcal{R}^{m \times l}$ be a matrix of rank nl . The two following distributions are identical:

$$\left\{ (\mathbf{s}, \mathbf{z}) \mid \mathbf{s} \leftarrow D_{\mathcal{R}^l, \sigma, 0}, \mathbf{z} \leftarrow D_{\mathcal{R}^m, \sqrt{\Sigma}, \mathbf{B}\mathbf{s}} \right\} \\ \left\{ (\hat{\mathbf{s}}, \mathbf{z}) \mid \mathbf{s} \leftarrow D_{\mathcal{R}^l, \sigma, 0}, \mathbf{z} \leftarrow D_{\mathcal{R}^m, \sqrt{\Sigma}, \mathbf{B}\mathbf{s}}, \hat{\mathbf{s}} \leftarrow D_{\mathcal{R}^l, \sqrt{\hat{\Sigma}}, \hat{\mathbf{c}}} \right\}$$

Where $\hat{\Sigma}^{-1} = 1/\sigma^2 + \mathbf{B}^T \Sigma^{-1} \mathbf{B}$, and $\hat{\mathbf{c}} = \hat{\Sigma} \mathbf{B}^T \Sigma^{-1} \mathbf{z}$.

Proof. As in [43] we compute the probability of $(\mathbf{u}, \mathbf{v}) \in \mathcal{R}^k \times \mathcal{R}^m$:

$$\Pr[\mathbf{s} = \mathbf{u}, \mathbf{z} = \mathbf{v}] = \Pr[\mathbf{z} = \mathbf{v} \mid \mathbf{s} = \mathbf{u}] \cdot \Pr[\mathbf{s} = \mathbf{u}] \\ = D_{\mathcal{R}^l, \sigma}(\mathbf{u}) \cdot D_{\mathcal{R}^m, \sqrt{\Sigma}, \mathbf{B}\mathbf{u}}(\mathbf{v}) \\ \propto D_{\mathcal{R}^l, \sigma}(\mathbf{u}) \cdot D_{\mathcal{R}^l, \sqrt{\Sigma'}, \mathbf{c}'}(\mathbf{u}) \\ \propto D_{\mathcal{R}^l, \sqrt{\hat{\Sigma}}, \hat{\mathbf{c}}}(\mathbf{u})$$

Where $\Sigma'^{-1} = \mathbf{B}^T \Sigma^{-1} \mathbf{B}$ and $\mathbf{c}' = \Sigma'^{-1} \mathbf{B} \Sigma^{-1} \mathbf{v}$. For the transition from the second to third line one can observe by expanding that $(\mathbf{u} - \mathbf{c}')^T \Sigma'^{-1} (\mathbf{u} - \mathbf{c}') = (\mathbf{v} - \mathbf{B}\mathbf{u})^T \Sigma (\mathbf{v} - \mathbf{B}\mathbf{u}) + C(\mathbf{v})$ where C is a term that does not depend on $\mathbf{u}$. The fourth line comes from the fact that the product of two Gaussian PDFs of respective center and covariance (μ_1, Σ_1) and (μ_2, Σ_2) is a Gaussian PDF of parameter $(\mu = \Sigma(\Sigma_1^{-1} \mu_1 + \Sigma_2^{-1} \mu_2), \Sigma = (\Sigma_1^{-1} + \Sigma_2^{-1})^{-1})$ up to constant multiplicative factor.

From this we have that the equality on conditional probabilities $\Pr[\hat{\mathbf{s}} = \mathbf{u} \mid \mathbf{z} = \mathbf{v}] = \Pr[\mathbf{s} = \mathbf{u} \mid \mathbf{z} = \mathbf{v}]$ and the result. $\square$

Theorem D.1. Let k, ℓ, q be integers. Let $\Sigma \in \mathbb{Z}^{n(k+\ell) \times n(k+\ell)}$ be definite positive, let $\mathbf{B} \in \mathcal{R}^{m \times k+\ell}$ be of rank $(k+\ell)n$. Let $\sigma_1 > 0$, $0 < \delta \leq 1$, and $\sigma > 0$ be such that $\sigma^{-2} = (1+\delta) \left(\frac{1}{\sigma_1^2} + \frac{\lambda_{\max}(\mathbf{B}^T \mathbf{B})}{\lambda_{\min}(\Sigma)} \right)$ (with $\lambda_{\min}$ and $\lambda_{\max}$ the smallest and largest eigenvalue). For $0 < \varepsilon < 1/3$, if $\delta\sigma > \eta_\varepsilon(\mathbb{Z}^{n(k+\ell)})$ then there exists an efficient reduction from $\text{MLWE}_{q,k,\ell,D_\sigma}$ to $\text{hint-MLWE}_{q,k,\ell,D_\sigma,\Sigma,\mathbf{B}}$ that reduces the advantage by at most $m \cdot 2\varepsilon$.

Proof. Let $(\mathbf{A}, \mathbf{b}) \in \mathcal{R}_q^{(k+\ell) \times k} \times \mathcal{R}_q^k$ be an MLWE instance. We first sample:

$$\mathbf{s}_0 \leftarrow D_{\mathcal{R}^{k+\ell}, \sigma, 0} \\ \mathbf{z}_0 \leftarrow D_{\mathcal{R}^m, \sqrt{\Sigma}, \mathbf{B}\mathbf{s}_0} \\ \hat{\mathbf{s}}_0 \leftarrow D_{\mathcal{R}^{k+\ell}, \sqrt{\hat{\Sigma}-\sigma^2}, \hat{\mathbf{c}}}$$

Where $\hat{\Sigma}^{-1} = 1/\sigma_1^2 + \mathbf{B}^T \Sigma^{-1} \mathbf{B}$, and $\hat{\mathbf{c}} = \hat{\Sigma} \mathbf{B}^T \Sigma^{-1} \mathbf{s}_0$. We then output $(\mathbf{A}, \mathbf{b} + [\mathbf{I}|\mathbf{A}] \cdot \hat{\mathbf{s}}_0, \mathbf{z}_0)$ as a hint-MLWE instance.

First note that since $\sigma^{-2} = \frac{1}{\sigma_1^2} + \frac{\lambda_{\max}(\mathbf{B}^T \mathbf{B})}{\lambda_{\min}(\Sigma)}$ the covariance $\hat{\Sigma} - \sigma^2 \mathbf{I}$ is positive. Now if $\mathbf{b}$ is uniform in $\mathcal{R}_q^k$ then $\mathbf{b} + \hat{\mathbf{s}}_0$ is also uniform. If $\mathbf{b} = \mathbf{A}\mathbf{s}$ for $\mathbf{s} \leftarrow D_{\mathcal{R}^{k+\ell}, \sigma, 0}$ then $\mathbf{s} + \hat{\mathbf{s}}_0$ is close to $D_{\mathcal{R}^{k+\ell}, \sqrt{\hat{\Sigma}}, \hat{\mathbf{c}}}$ by Lemma D.1 and by Lemma D.6 the distributions:

$$\left\{ (\mathbf{A}, [\mathbf{A}|\mathbf{I}]\mathbf{s}, \mathbf{z}) \mid \mathbf{s} \leftarrow D_{\mathcal{R}^k, \sigma_1, 0}, \mathbf{z} \leftarrow D_{\mathcal{R}^m, \sqrt{\Sigma}, \mathbf{B}\mathbf{s}} \right\} \\ \left\{ (\mathbf{A}, [\mathbf{A}|\mathbf{I}]\hat{\mathbf{s}}, \mathbf{z}) \mid \mathbf{s} \leftarrow D_{\mathcal{R}^k, \sigma_1, 0}, \mathbf{z} \leftarrow D_{\mathcal{R}^m, \sqrt{\Sigma}, \mathbf{B}\mathbf{s}}, \hat{\mathbf{s}} \leftarrow D_{\mathcal{R}^k, \sqrt{\hat{\Sigma}}, \hat{\mathbf{c}}} \right\}$$

are identical. Since the second is the distribution output by the adversary and the first is the hint-MLWE distribution we have the desired result. $\square$

From this reduction we can prove that the hint-MLWE corresponding to our posteriori secret sharing is secure. If $T-1$ parties are corrupted then all but one of the S shares are revealed, corresponding to a hint-MLWE instance with $\mathbf{B} = [1/S, \dots, 1/S]^T \in \mathcal{R}^{S-1}$, and $\Sigma = \sigma_{\text{share}}^2 (\mathbf{I}_{(S-1)(k+\ell)n \times (S-1)(k+\ell)n} - \frac{1}{T} \mathbf{J}_{(S-1)(k+\ell)n \times (S-1)(k+\ell)n})$ with $\mathbf{J}$ the all one matrix, and σ_{share} the standard deviation of the shares. From this we have $\lambda_{\max}(\mathbf{B}) = (S-1)/S^2$ and $\lambda_{\min}(\Sigma) = s^2/S$, thus we will reduce to an MLWE instance with parameter

$$\sigma = (1+\delta)^{-1/2} \left(\frac{1}{\sigma_{\text{ML-DSA}}^2} + \frac{S-1}{S\sigma_{\text{share}}^2} \right)^{-1/2}$$

For $\delta > 0$ such that $\delta\sigma \geq \eta_\varepsilon(\mathbb{Z}^{n(k+\ell)}) = \sqrt{\frac{\log(n(k+\ell)(1+1/\varepsilon))}{\pi}}$.

Not that the statement of Theorem D.1 has to use the statistical distance as the Rényi divergence does not work with decision problems. However in the proof of unforgeability we can replace this statistical distance with the Rényi divergence by using it before the reduction. We recall the games of Theorem 3.3 in more detail.

- **Game 11'.** We replace the shares of the secret obtained by using the sampler of Section D.2. For $\sigma_{\text{share}} \geq \sqrt{\frac{9}{8} \sqrt{\frac{\log(2S\sqrt{m})}{\pi}}}$, the advantage of the adversary is multiplied at most by the Rényi divergence between the two distributions.

$$\text{Adv}_{\mathcal{A}}^{\text{Game}_{11'}} \leq (1 + 2\alpha(q+1)^2) \text{Adv}_{\mathcal{A}}^{\text{Game}_{11'}}$$

- **Game 13.** We change the Key generation to use $\mathbf{s} \leftarrow D_{\mathcal{R}^k, \sqrt{\widehat{\Sigma}}, \mathbf{c}}$, by Lemma D.6 this game is identical to the previous one.
- **Game 14.** We replace $\mathbf{s}$ with $\mathbf{s} = \mathbf{s}_A + \mathbf{s}_B$ where $\mathbf{s}_A \leftarrow D_{\mathcal{R}^k, \sqrt{\widehat{\Sigma} - \sigma^2}, \mathbf{c}}$ and $\mathbf{s}_B \leftarrow D_{\mathcal{R}^k, \sigma, 0}$. Since $\sigma \geq \delta\sigma \geq \eta_\varepsilon(\mathbb{Z}^{(\ell+k)n})$ for $\varepsilon^{-1} = (\ell+k)n$. We have:

$$\text{Adv}_{\mathcal{A}}^{\text{Game}_{13}} \leq 4\text{Adv}_{\mathcal{A}}^{\text{Game}_{14}}$$

- **Game 15.** We replace the public key with uniform. Using Theorem D.1:

$$\left| \text{Adv}_{\mathcal{A}}^{\text{Game}_{14}} - \text{Adv}_{\mathcal{A}}^{\text{Game}_{15}} \right| \leq \text{Adv}_{\mathcal{B}}^{\text{MLWE}_{q,k,\ell,D\sigma}}$$

E WAN latencies between AWS EC2 instances

Latencies between the machines used in our experiments from 4 are reported in Table 7, based on median round-trip measurements from [12] (consistent with latency variation over the Internet [36]). WAN measurements reflect the slowest communication path (the "critical path" [24]), consistently between Taipei and Virginia, to capture worst-case delays. In real-world cloud deployments, infrastructure optimizations (e.g., load balancing, anycast routing) may reduce delays and overhead.

	Taipei	London	Virginia	Seoul
Taipei		177.82	232.46	14.22
London	177.82		52.07	163.54
Virginia	232.46	52.07		130.24
Seoul	14.22	163.54	130.24	

Table 7: WAN median latencies (in ms) between different AWS EC2 machines, as noted in [12].

F Omitted proof of unforgeability of Threshold ML-DSA

In this section, we fully formalize the unforgeability of Threshold ML-DSA. We provide a complete formal statement and its proof, adapting the proof from [20]. We limit ourselves to the case $K = 1$, as the general case can be easily derived by considering $Q'_s = K \cdot Q_s$, the number of calls to signing queries.

Theorem F.1 (Unforgeability of Threshold ML-DSA). *Formally, let $\mathcal{A}$ be an adversary against the TS-UF security of our threshold scheme making Q_s calls*

to signing oracles, and at most $Q_{\text{H}_{\text{cmt}}}, Q_{\text{H}}$ queries respectively to the random oracles $\text{H}_{\text{cmt}}, \text{H}$. There exist adversaries $\mathcal{B}_s$ against the $\text{MLWE}_{q,k,\ell,\chi_s}$, $\mathcal{B}_r$ against the $\text{MLWE}_{q,k,\ell,\chi_r}$ game, $\mathcal{B}_z$ against the $\text{MLWE}_{q,k,\ell,\chi_z}$ game, and $\mathcal{B}_{\text{UF}}$ against the unforgeability of ML-DSA running in time $\mathcal{T}_{\mathcal{B}_s} \approx \mathcal{T}_{\mathcal{B}_r} \approx \mathcal{T}_{\mathcal{B}_z} \approx \mathcal{T}_{\mathcal{B}_{\text{UF}}} \approx \mathcal{T}_{\mathcal{A}}$ such that:

$$\begin{aligned} \text{Adv}_{\mathcal{A}}^{\text{TS-UF}} &\leq \frac{Q_s Q_{\text{H}_{\text{cmt}}}}{q^{nk}} + Q_{\text{H}} Q_s^2 \cdot \left(\frac{2\gamma_2 + 1}{q} \right)^{kn} + Q_{\text{H}} Q_s \cdot 2^{-2\cdot\kappa} \\ &\quad + \frac{T \cdot Q_s Q_{\text{H}_{\text{cmt}}} + (Q_{\text{H}_{\text{cmt}}} + Q_s)^2}{2^{2\kappa}} + Q_{\text{H}}^2 \cdot 2^{-512} + \text{negl}(\kappa) \\ &\quad + \left(1 + \frac{2\varepsilon}{1-\varepsilon} \right)^{Q_s} \cdot \left(2\text{Adv}_{\mathcal{B}_s}^{\text{MLWE}} + \text{Adv}_{\mathcal{B}_{\text{UF}}}^{\text{UF}} \right) \\ &\quad + Q_s \cdot \frac{3M-2}{M-1} \cdot \left(\text{Adv}_{\mathcal{B}_r}^{\text{MLWE}} + \text{Adv}_{\mathcal{B}_z}^{\text{MLWE}} + \frac{2\varepsilon}{1-\varepsilon} \right) \end{aligned}$$

For completeness, we include the full formal proof of the unforgeability of our Threshold ML-DSA scheme. This is a straightforward adaptation of the proof and hybrids of [20, Theorem 4].

We first recall results from [20] to tightly simulate rejected transcripts in the Fiat-Shamir with Aborts paradigm, and to evaluate rejection probabilities as a function of ε .

Lemma F.1 (Adaptation of [20, Lemma 4]). *Let any $M > 1$, $B > 0, \varepsilon < 1$, $\mathbf{v} \in \mathcal{R}^{k+\ell}$, distributions χ_r, χ_z over $\mathcal{R}^{k+\ell}$ such that $R_\infty^\varepsilon(\chi_z || \chi_r + \mathbf{v}) \leq M$. We have:*

$$\begin{aligned} \frac{1-\varepsilon}{M} &\leq \Pr[\text{Rej}(\mathbf{v}, \chi_r, \chi_s, M) \neq \perp] \leq \frac{1}{M} \\ \delta &\leq \frac{2\varepsilon}{1-\varepsilon} \end{aligned}$$

where $\delta = \Delta(\mathbf{z}|\text{acc}, \chi_z)$.

Lemma F.2 ([20, Lemma 5]). *Given a secret $\mathbf{s}$, and challenge c , note $(\mathbf{z}|\text{rej})_{\mathbf{v}=\mathbf{c}\cdot\mathbf{s}}$ the distribution of rejected vectors $\mathbf{z}_i$ conditioned on $(\mathbf{s}, c)$. Let $\mathcal{A}$ be an adversary against the $\text{MLWE}_{q,k,\ell,(\mathbf{z}|\text{rej})_{\mathbf{v}=\mathbf{c}\cdot\mathbf{s}}}$ problem. There exist adversaries $\mathcal{B}_1, \mathcal{B}_2$ respectively against the $\text{MLWE}_{q,k,\ell,\chi_r}$ and $\text{MLWE}_{q,k,\ell,\chi_z}$ problems, running in time $\mathcal{T}_{\mathcal{B}_1} \approx \mathcal{T}_{\mathcal{B}_2} \approx \mathcal{T}_{\mathcal{A}}$ such that:*

$$\text{Adv}_{\mathcal{A}}^{\text{MLWE}_{q,k,\ell,(\mathbf{z}|\text{rej})_{\mathbf{v}=\mathbf{c}\cdot\mathbf{s}}}} \leq \frac{1}{p} \cdot \left(\text{Adv}_{\mathcal{B}_1}^{\text{MLWE}_{q,k,\ell,\chi_r}} + \text{Adv}_{\mathcal{B}_2}^{\text{MLWE}_{q,k,\ell,\chi_z}} + \delta \right)$$

where $p = \Pr[\text{Rej}(\mathbf{c}\mathbf{s}, \chi_r, \chi_s, M; \mathbf{r} \leftarrow \chi_r) = \perp]$ is the probability of rejection of $\mathbf{z}$ conditioned on $(\mathbf{s}, c)$ and δ is the statistical distance between $(\mathbf{z}|\text{acc})_{\mathbf{v}=\mathbf{c}\cdot\mathbf{s}}$ and the target distribution χ_z .

Applying Lemma F.1, if $R_\infty^\varepsilon(\chi_z || \chi_r + \mathbf{c} \cdot \mathbf{s}) \leq M$, then $p \geq 1 - \frac{1}{M}$.

Proof. We proceed with a series of hybrids starting from the TS-UF game. Throughout this proof, we denote the advantage of an adversary $\mathcal{A}$ against a search game G by $\text{Adv}_{\mathcal{A}}^G := \Pr[G(\mathcal{A}) \rightarrow 1]$ – in particular, we omit passing κ as input.

Game₁. This is the TS-UF game from Fig. 1.

Game₂. In this hybrid we defer the computation of the honest $\mathbf{w}_i$ to the second round of the protocol, and program the random oracle to be consistent. We also introduce a flag f_{fail} to explicitly mark additional cases where the adversary loses. Notably, f_{fail} is set to $\top$ when the random oracle is programmed on a previously queried value. This is formalized in Fig. 14.

The view of the adversary $\mathcal{A}$ differs if they called the random oracle of one of the $\mathbf{w}_i$ before round 2 was executed, i.e.

$$\left| \text{Adv}_{\mathcal{A}}^{\text{Game}_1} - \text{Adv}_{\mathcal{A}}^{\text{Game}_2} \right| \leq \Pr[E]$$

where E is the event “ H_{cmt} was queried on a honest $\mathbf{w}_i$ before round 2 in **Game₁**”.

By union-bound, we can reduce this to a single call to OSign_1 .

$$\left| \text{Adv}_{\mathcal{A}}^{\text{Game}_1} - \text{Adv}_{\mathcal{A}}^{\text{Game}_2} \right| \leq \sum_{s=1}^{Q_s} \Pr[E'_s] \quad (5)$$

where E'_s is the event “The $\mathbf{w}_i$ produced by the s -th call to OSign_1 is queried on H_{cmt} before round 2 in **Game₁**”.

We denote the above individual probabilities p_s for $s \in [Q_s]$. Formally, we introduce in Fig. 15 an intermediary game Int_1^s in which $\mathcal{A}$ wins with probability exactly p_s . We also introduce a tweak of Int_1^s , named Int_2^s , where we replace the s -th commitment $\mathbf{w}_i$ by a uniform value in $\mathcal{R}_q^k$.

First, the difference in advantage between Int_1^s and Int_2^s reduces to $\text{MLWE}_{q,k,\ell,\chi_r}$. Formally, we can define an adversary $\mathcal{E}^s$ against the $\text{MLWE}_{q,k,\ell,\chi_r}$ problem such that:

$$\left| \text{Adv}_{\mathcal{A}}^{\text{Int}_1^s} - \text{Adv}_{\mathcal{A}}^{\text{Int}_2^s} \right| = \left| p_s - \text{Adv}_{\mathcal{A}}^{\text{Int}_2^s} \right| \leq \text{Adv}_{\mathcal{E}^s}^{\text{MLWE}_{q,k,\ell,\chi_r}} \quad (6)$$

Also, we note that in Int_2^s , the probability that one call of $\mathcal{A}$ to the random oracle queries the s -th $\mathbf{w}_i$ is bounded by the min-entropy of $\mathbf{w}_i \xleftarrow{\$} \mathcal{R}_q^k$, i.e. by q^{-nk} . By union bound, $\text{Adv}_{\mathcal{A}}^{\text{Int}_2^s} \leq Q_{H_{\text{cmt}}} \cdot q^{-nk}$.

By combining the above inequality with Eq. (5) and Eq. (6), we conclude:

$$\left| \text{Adv}_{\mathcal{A}}^{\text{Game}_1} - \text{Adv}_{\mathcal{A}}^{\text{Game}_2} \right| \leq Q_s \cdot Q_{H_{\text{cmt}}} \cdot q^{-nk} + Q_s \cdot \text{Adv}_{\mathcal{B}_1}^{\text{MLWE}_{q,k,\ell,\chi_r}}$$

where $\mathcal{B}_1$ is the adversary $\mathcal{E}^s$ maximizing $\text{Adv}_{\mathcal{E}^s}^{\text{MLWE}_{q,k,\ell,\chi_r}}$.

Remark 2. As an useful remark for following hybrids, we note that a given $\mathbf{w}_i$ can't be used twice by the same party after **Game₂** as then the random oracle programming would fail, and the adversary loses.

Game₃. In this hybrid, we sample a challenge c and its seed $\tilde{c}$ in advance during round 2. When the last honest hash cmt_i obtains a corresponding $\mathbf{w}_i$ programmed, we program the values $H(\mu||\mathbf{w}_{\top}) = \tilde{c}$ and $\text{SampleInBall}(\tilde{c}) = c$. This is formalized in Fig. 16.

The proof for this hybrid is analogous to the previous one. First, these changes do not bias H and SampleInBall as the sampled challenges c and seeds $\tilde{c}$ are used at most once for programming H and SampleInBall , and are not provided to the adversary before programming. The view of the adversary hence differs only if it queried the random oracle (i) H on some $\mu||\mathbf{w}_{\top}$, where $\mathbf{w}$ are the high bits of $\hat{\mathbf{w}} = \sum_{i \in \text{act}} \mathbf{w}_i$ before it is programmed in round 2, or (ii) SampleInBall on $\tilde{c}$ before it is programmed.

First, we can easily bound the probability that SampleInBall is queried on some $\tilde{c}$ before it is programmed by $Q_H Q_s \cdot 2^{-2\kappa}$.

Then, we are interested in the probability that H is queried on some $\mu||\mathbf{w}_{\top}$ before it is programmed. For each $s \in Q_s$, we reexpress as an advantage the probability p_s that the s -th $\mathbf{w}_i$ produced during round 2 is used to compute a $\mathbf{w}_{\top}$ that was previously queried on the random oracle H . Formally, we define the game Int_3^s in Fig. 17, that verifies $p_s = \text{Adv}_{\mathcal{A}}^{\text{Int}_3^s}$. By union bound, we have $\left| \text{Adv}_{\mathcal{A}}^{\text{Game}_2} - \text{Adv}_{\mathcal{A}}^{\text{Game}_3} \right| \leq \sum_{s \in [Q_s]} p_s$.

We additionally define the game Int_4^s in Fig. 17, where we replace the s -th $\mathbf{w}_i$ by a uniform value in $\mathcal{R}_q^k$. The advantage difference between Int_3^s and Int_4^s again reduces to $\text{MLWE}_{q,k,\ell,\chi_r}$: $\left| \text{Adv}_{\mathcal{A}}^{\text{Int}_3^s} - \text{Adv}_{\mathcal{A}}^{\text{Int}_4^s} \right| \leq \text{Adv}_{\mathcal{E}^s}^{\text{MLWE}_{q,k,\ell,\chi_r}}$ for an adversary $\mathcal{E}^s$ against the $\text{MLWE}_{q,k,\ell,\chi_r}$ problem.

Then, we can see that the probability that $\mathcal{A}$ wins in Int_4^s can be bounded using the min-entropy of $\text{HighBits}_q(\mathcal{U}(\mathcal{R}_q^k), 2\gamma_2)$. Indeed, the s -th $\mathbf{w}_i$ is sampled uniformly from $\mathcal{R}_q^k$ and ensures that each $\mathbf{w}$ that needs to be programmed follows the distribution $\text{HighBits}_q(\mathcal{U}(\mathcal{R}_q^k), 2\gamma_2)$. As there are at most Q_s $\mathbf{w}$ to check, we obtain the bound $\text{Adv}_{\mathcal{A}}^{\text{Int}_4^s} \leq Q_H Q_s \cdot \left(\frac{2\gamma_2+1}{q} \right)^{kn}$.

Putting together the different inequalities obtained, we finally deduce the existence of an adversary $\mathcal{B}_2$ against the $\text{MLWE}_{q,k,\ell,\chi_r}$ such that:

$$\left| \text{Adv}_{\mathcal{A}}^{\text{Game}_2} - \text{Adv}_{\mathcal{A}}^{\text{Game}_3} \right| \leq Q_H Q_s^2 \cdot \left(\frac{2\gamma_2+1}{q} \right)^{kn} + Q_H Q_s \cdot 2^{-2\kappa} + Q_s \cdot \text{Adv}_{\mathcal{B}_2}^{\text{MLWE}_{q,k,\ell,\chi_r}}$$

Game₄. In this hybrid, we ensure that H_{cmt} has no collisions and that the adversary cannot find preimages of hashes cmt_i after passing them to the second signing oracle. We introduce a table Cmt that records every cmt sampled in H_{cmt} or in OSign_1 , as well as those passed to the second signing oracle. Whenever a new hash cmt is sampled, we ensure that it was not previously added to

Game₂ 1: $Q_{\text{msg}}, \text{UnopenedCmt}[\cdot] := \emptyset$ 2: $f_{\text{fail}} := \perp$ 3: $\text{pp}, \text{st}_{\mathcal{A}} \leftarrow \text{Setup}(\kappa, N, T)$ 4: $\text{corrupt} \leftarrow \mathcal{A}(\text{pp})$ 5: assert { $\text{corrupt} \subseteq [N]$ } 6: assert { $ \text{corrupt} < T$ } 7: $\text{honest} := [N] \setminus \text{corrupt}$ 8: $(\text{vk}, (\text{sk}_i)_{i \in [N]}) \leftarrow \text{Sign.Keygen}(\text{pp})$ 9: for $i \in \text{honest}$ do 10: $\text{st}_i := \emptyset$ 11: $(\text{msg}, \text{sig}) \leftarrow \mathcal{A}^{\text{H}, (\text{OSign}_i(\cdot))_{i \in [R]}}(\text{st}_{\mathcal{A}}, \text{vk}, (\text{sk}_i)_{i \in \text{corrupt}})$ 12: if $f_{\text{fail}} = \top$ then 13: return 0 14: req { $\text{msg} \notin Q_{\text{msg}}$ } 15: return $\text{Verify}(\text{vk}, \text{msg}, \text{sig})$ 16: return 1	ShareSign₁ ($\text{vk}, i, \text{sk}_i, \text{st}_i$) 1: $\text{cmt}_i \xleftarrow{\$} \{0, 1\}^{2\kappa}$ 2: $\text{UnopenedCmt}[(i, \text{cmt}_i)] = \top$ 3: return $(\text{pm}_1^i := \text{cmt}_i, \text{st}_i)$ ShareSign₂ ($\text{vk}, \text{act}, \text{msg}, \text{pm}_1, i, \text{sk}_i, \text{st}_i$) 1: assert { $\text{act} \subseteq [N] \wedge i \in \text{act}$ } 2: assert { $\text{UnopenedCmt}[(i, \text{pm}_1^i)] = \top$ } 3: $\text{UnopenedCmt}[(i, \text{pm}_1^i)] := \perp$ 4: $\mathbf{r}_i \leftarrow \chi_{\mathbf{r}}$ 5: $\mathbf{w}_i := [\mathbf{A} \quad \mathbf{I}] \cdot \mathbf{r}_i$ 6: $\text{H}_{\text{cmt}}(\text{vk}, \mathbf{w}_i) := \text{pm}_1^i$ ▷ Program random oracle 7: $\text{st}_i := \text{st}_i \cup \{(\text{act}, \text{msg}, \text{pm}_1, \mathbf{w}_i, \mathbf{r}_i)\}$ 8: return $(\text{pm}_2^i := \mathbf{w}_i, \text{st}_i)$
---	--

Figure 14: Second hybrid of the unforgeability proof of Threshold ML-DSA. If the random oracle is programmed on a previously queried index, consider that the adversary loses, i.e. f_{fail} is set to $\top$. Difference with the previous hybrid are highlighted.

Cmt , or the challenger sets the flag f_{fail} to $\top$ in order to abort the game. This is formalized in Fig. 18.

The view of the adversary differs in Game_4 if it was previously able to (i) find a pre-image of a cmt_i after passing it to OSign_2 , or (ii) if a given cmt was sampled twice.

We can bound the probability of (i) due to the pre-image resistance of the hash function H_{cmt} , and with an union bound over all the commits passed by the adversary to OSign_2 which gives a bound $Q_{\text{H}_{\text{cmt}}} \cdot T \cdot Q_s \cdot 2^{-2\kappa}$.

As for (ii), we rely on the collision resistance of H_{cmt} . Since the commitments are sampled uniformly from $\{0, 1\}^{2\kappa}$, we can bound the probability of (ii) by $\frac{(Q_{\text{H}_{\text{cmt}}} + Q_s)^2}{2^{2\kappa}}$.

We conclude that,

$$\left| \text{Adv}_{\mathcal{A}}^{\text{Game}_3} - \text{Adv}_{\mathcal{A}}^{\text{Game}_4} \right| \leq \frac{Q_{\text{H}_{\text{cmt}}} \cdot T \cdot Q_s + (Q_{\text{H}_{\text{cmt}}} + Q_s)^2}{2^{2\kappa}}$$

Game₅. In this hybrid, we ensure that H on $\text{tr}||\text{msg}$ does never return twice the same $\tilde{c}$ for two different messages. This is formalized in Fig. 19.

As for the previous hybrid, this is ensured by the collision resistance of H due to the large space of $\tilde{c}$.

We conclude that,

$$\left| \text{Adv}_{\mathcal{A}}^{\text{Game}_4} - \text{Adv}_{\mathcal{A}}^{\text{Game}_5} \right| \leq Q_{\text{H}}^2 \cdot 2^{-512}$$

Game₆. In this hybrid, we ensure that the challenge used in round 3 is the same as the one sampled in round

2, and we precompute the responses $\mathbf{z}_i$ in advance. This is formalized in Fig. 20.

We want to show that responses are identically distributed in Game_6 . This is the case if programmed responses use the same c in both round 2 and round 3, and if they are used at most once.

Now, we can first observe that if the hash checks in round 3 pass, then all the $\text{cmt}_i := \text{H}_{\text{cmt}}(\text{vk}, \mathbf{w}_i)$ are correctly defined. Recall that Game_4 ensured that H_{cmt} has no collisions, and that pre-images cannot be found after round 2 is called. Hence, if these checks pass, then it means that in round 2, all the hashes cmt_i either: (i) already had pre-images, or (ii) were produced by an honest party in OSign_1 and were waiting for programming. Eventually, all the missing cmt_i from (ii) must thus have been programmed in OSign_2 and as the challenger programs SampleInBall as soon as all the $\mathbf{w}_i$ are defined, then it ensures that the same c is used in round 2 and in round 3.

Additionally, responses are used at most once by a given party due to the collision resistance of H_{cmt} and Remark 2 which ensures that party i will accept cmt_i at most once.

All in all, we conclude that the adversary's view is identically distributed in Game_5 and Game_6 so,

$$\text{Adv}_{\mathcal{A}}^{\text{Game}_6} = \text{Adv}_{\mathcal{A}}^{\text{Game}_5}$$

Game₇. In this hybrid, we no longer sample $\mathbf{r}_i$ out of the rejection sampling. In case the rejection sampling aborts,

Int_1^s	$\text{ShareSign}_1(\text{vk}, i, \text{sk}_i, \text{st}_i)$	$\text{ShareSign}_2(\text{vk}, \text{act}, \text{msg}, \text{pm}_1, i, \text{sk}_i, \text{st}_i)$
1: $Q_{\text{msg}} := \emptyset$ 2: $\text{Commitments}[\cdot] := \emptyset$ 3: $\text{Cnt} := 0$ 4: $\text{wins} := 0$ 5: $\text{pp}, \text{st}_A \leftarrow \text{Setup}(\kappa, N, T)$ 6: $\text{corrupt} \leftarrow \mathcal{A}(\text{pp})$ 7: $\text{assert}\{ \text{corrupt} \subseteq [N] \}$ 8: $\text{assert}\{ \text{corrupt} < T \}$ 9: $\text{honest} := [N] \setminus \text{corrupt}$ 10: $(\text{vk}, (\text{sk}_i)_{i \in [N]}) \leftarrow \text{Sign.Keygen}(\text{pp})$ 11: for $i \in \text{honest}$ do 12: $\text{st}_i := \emptyset$ 13: $(\text{msg}, \text{sig}) \leftarrow \mathcal{A}^{H_i(\text{OSign}_i(\cdot))}_{i \in [R]}(\text{st}_A, \text{vk}, (\text{sk}_i)_{i \in \text{corrupt}})$ 14: return wins	1: $\text{Cnt} := \text{Cnt} + 1$ 2: $\mathbf{r}_i \leftarrow \chi_{\mathbf{r}}$ 3: $\mathbf{w}_i := [\mathbf{A} \ \mathbf{I}] \cdot \mathbf{r}_i$ 4: $\text{Commitments}[\text{Cnt}] := \mathbf{w}_i$ 5: $\text{st}_i := \text{st}_i \cup \{(\text{cmt}_i, \mathbf{w}_i, \mathbf{r}_i)\}$ 6: $\text{cmt}_i = \text{H}_{\text{cmt}}(\text{vk}, \mathbf{w}_i) \in \{0, 1\}^{2\kappa}$ 7: return $(\text{pm}_1^i := \text{cmt}_i, \text{st}_i)$	1: $\text{assert}\{ \text{act} \subseteq [N] \wedge i \in \text{act} \}$ 2: $\text{assert}\{ (\text{pm}_1^i, \cdot, \cdot) \in \text{st}_i \}$ 3: Pick $(\text{cmt}_i, \mathbf{w}_i, \mathbf{r}_i)$ from st_i with $\text{pm}_1^i = \text{cmt}_i$ 4: if $\text{Commitments}[s] = \mathbf{w}_i$ then 5: $\text{Commitments}[s] = \perp$ 6: return $\perp$ 7: $\text{st}_i := \text{st}_i \setminus \{(\text{cmt}_i, \mathbf{w}_i, \mathbf{r}_i)\}$ 8: $\text{st}_i := \text{st}_i \cup \{(\text{act}, \text{msg}, \text{pm}_1, \mathbf{w}_i, \mathbf{r}_i)\}$ 9: return $(\text{pm}_2^i := \mathbf{w}_i, \text{st}_i)$ $\text{H}_{\text{cmt}}(\text{vk}, \mathbf{w})$ 1: if $\text{Commitments}[s] = \mathbf{w}$ then 2: $\text{wins} = 1$ Other lines are identical
Int_2^s	$\text{ShareSign}_1(\text{vk}, i, \text{sk}_i, \text{st}_i)$	
Identical to Int_1^s	1: $\text{Cnt} := \text{Cnt} + 1$ 2: if $\text{Cnt} \neq s$ then 3: $\mathbf{r}_i \leftarrow \chi_{\mathbf{r}}$ 4: $\mathbf{w}_i := [\mathbf{A} \ \mathbf{I}] \cdot \mathbf{r}_i$ 5: $\text{st}_i := \text{st}_i \cup \{(\text{cmt}_i, \mathbf{w}_i, \mathbf{r}_i)\}$ 6: else 7: $\mathbf{w}_i \xleftarrow{\$} \mathcal{R}_q^k$ 8: $\text{st}_i := \text{st}_i \cup \{(\text{cmt}_i, \mathbf{w}_i, \cdot)\}$ 9: $\text{Commitments}[\text{Cnt}] := \mathbf{w}_i$ 10: $\text{cmt}_i = \text{H}_{\text{cmt}}(\text{vk}, \mathbf{w}_i) \in \{0, 1\}^{2\kappa}$ 11: return $(\text{pm}_1^i := \text{cmt}_i, \text{st}_i)$	

Figure 15: Intermediary games for the second hybrid of the unforgeability proof of Threshold ML-DSA. Difference with Game_1 are highlighted.

we sample a fresh $\mathbf{z}_i$ from the aborting distribution $(\mathbf{z}|\text{rej})_{\mathbf{v}=c \cdot \mathbf{s}_i^{\text{part}}}$. Then, we compute $\mathbf{w}_i$ as $[\mathbf{A} \ \mathbf{I}] \cdot \mathbf{z}_i - \mathbf{t}_i^{\text{part}}$, where $\mathbf{t}_i^{\text{part}} = [\mathbf{A} \ \mathbf{I}] \cdot \mathbf{s}_i^{\text{part}}$. This is formalized in Fig. 21.

We can see that the view of the adversary is identically distributed. Indeed, in Game_6 we have $[\mathbf{A} \ \mathbf{I}] \cdot \mathbf{z}_i = c \cdot \underbrace{[\mathbf{A} \ \mathbf{I}] \cdot \mathbf{s}_i^{\text{part}}}_{\mathbf{t}_i^{\text{part}}} + \underbrace{[\mathbf{A} \ \mathbf{I}] \cdot \mathbf{r}_i}_{\mathbf{w}_i}$ for $\mathbf{z}_i = \mathbf{r}_i + c \cdot \mathbf{s}_i^{\text{part}}$ (even in case it is rejected). We can thus equivalently write $\mathbf{w}_i = [\mathbf{A} \ \mathbf{I}] \cdot \mathbf{z}_i - c \cdot \mathbf{t}_i^{\text{part}}$ and sample $\mathbf{z}_i$ first. The sampling of rejected $\mathbf{z}_i$ from $(\mathbf{z}|\text{rej})_{\mathbf{v}=c \cdot \mathbf{s}_i^{\text{part}}}$ finally ensures the same distribution of $\mathbf{z}_i$ in the above equality in case of rejections.

So,

$$\text{Adv}_{\mathcal{A}}^{\text{Game}_7} = \text{Adv}_{\mathcal{A}}^{\text{Game}_6}$$

Game₈. In this hybrid, we ensure that the values $c \cdot \mathbf{s}_i^{\text{part}}$ have a twisted norm at most B . That is, whenever this is

not the case we set the flag f_{fail} to $\top$ to abort the game. This is formalized in Fig. 22.

By assumption on B , for any choice of act this is true with overwhelming probability over the randomness of c and of the secrets.

Since the adversary chooses act , it may not be independent of the secrets and we cannot directly conclude. However, we can simply guess act since there are $\binom{N}{T} = \text{poly}(\kappa)$ of them by assumption.

By union-bound over all the calls to a signing oracle, we conclude:

$$\left| \text{Adv}_{\mathcal{A}}^{\text{Game}_7} - \text{Adv}_{\mathcal{A}}^{\text{Game}_8} \right| \leq Q_s \cdot \binom{N}{T} \cdot \text{negl}(\kappa)$$

Game₉. In this hybrid, we replace the aborting $\mathbf{w}_i$ by a uniform value in $\mathcal{R}_q^k$. This is formalized in Fig. 23.

There are up to Q_s $\mathbf{w}_i$ to replace. We define a family of games G^p for $p \in [Q_s + 1]$, replacing one by one the

Game₃
1: $Q_{\text{msg}}, \text{UnopenedCmt}[\cdot] := \emptyset$ 2: ToProgram, Programmed $[\cdot] := \emptyset$ Other lines are identical to Game ₂
ShareSign₂ (vk, act, msg, pm ₁ , i, sk _i , st _i)
1: assert { act $\subseteq [N] \wedge i \in \text{act}$ } 2: assert { UnopenedCmt $[(i, \text{pm}_1^i)] = \top$ } 3: UnopenedCmt $[(i, \text{pm}_1^i)] := \perp$ 4: if (act, (cmt _j) _{j$\in$act} , msg, $\cdot$) $\notin$ ToProgram then 5: $\tilde{c} \xleftarrow{\$} \{0, 1\}^{2\cdot\kappa}; c \leftarrow \mathcal{C}$ 6: ToProgram := ToProgram $\cup \{(\text{act}, (\text{cmt}_j)_{j \in \text{act}}, \text{msg}, \tilde{c}, c)\}$ 7: Pick (a ₁ , a ₂ , a ₃ , c) 8: from ToProgram s.t. (a ₁ , a ₂ , a ₃) = (act, (cmt _j) _{j$\in$act} , msg) 9: $\mathbf{r}_i \leftarrow \chi_{\mathbf{r}}$ 10: $\mathbf{w}_i := [\mathbf{A} \quad \mathbf{I}] \cdot \mathbf{r}_i$ 11: $\text{H}_{\text{cmt}}(\text{vk}, \mathbf{w}_i) := \text{pm}_1^i$ 12: for (act', (cmt' _j) _{j$\in$act'} , msg', c') $\in$ ToProgram do 13: if (i $\in$ act') $\wedge$ (cmt' _i = pm ₁ ⁱ) $\wedge$ ($\forall j \in \text{act}' \setminus \{i\}, \exists \mathbf{w}_j, \text{cmt}_j = \text{H}_{\text{cmt}}(\text{vk}, j, \mathbf{w}_j)$) then $\triangleright$ Find all the new programmable challenges 14: ToProgram := ToProgram $\setminus \{(\text{act}', (\text{cmt}'_j)_{j \in \text{act}'}, \text{msg}', \tilde{c}', c')\}$ 15: $\mathbf{w} := \sum_{j \in \text{act}'} \mathbf{w}_j$; ($\mathbf{w}_{\top}, \mathbf{w}_{\perp}$) = HighBits _q ($\mathbf{w}$, 2 γ_2) 16: $\text{H}(\mu \mathbf{w}_{\top}) := \tilde{c}'$; SampleInBall($\tilde{c}'$) := c' $\triangleright$ Program random oracle 17: st _i := st _i $\cup \{(\text{act}, \text{msg}, \text{pm}_1, \mathbf{w}_i, \mathbf{r}_i)\}$ 18: return (pm ₂ ⁱ := $\mathbf{w}_i$, st _i)

Figure 16: Third hybrid of the unforgeability proof of Threshold ML-DSA. If the random oracle is programmed on a previously queried index, consider that the adversary loses, i.e. f_{fail} is set to $\top$. Difference with the previous hybrid are highlighted.

$\mathbf{w}_i$ by a uniform vector, which verifies:

$$\left| \text{Adv}_{\mathcal{A}}^{\text{Games}} - \text{Adv}_{\mathcal{A}}^{\text{Game}_9} \right| \leq \sum_{s \in [Q_s]} \left| \text{Adv}_{\mathcal{A}}^{G^{p+1}} - \text{Adv}_{\mathcal{A}}^{G^p} \right|$$

Let $p \in [Q_s + 1]$. We want to bound $\left| \text{Adv}_{\mathcal{A}}^{G^{p+1}} - \text{Adv}_{\mathcal{A}}^{G^p} \right|$.

The core idea is to condition on the value of $v := (\text{act}, i, \mathbf{s}_i^{\text{part}}, c)$ used to define the distribution $(\mathbf{z} | \text{rej})_{\mathbf{v} = c \cdot \mathbf{s}_i^{\text{part}}}$:

$$\left| \text{Adv}_{\mathcal{A}}^{G^{p+1}} - \text{Adv}_{\mathcal{A}}^{G^p} \right| \leq \sum_{s, c} \left| \text{Adv}_{\mathcal{A}}^{F'^v} - \text{Adv}_{\mathcal{A}}^{F^v} \right| \cdot \Pr[v \text{ used by } G^s] \quad (7)$$

where F'^v and F^v are respectively the games G^{p+1} and G^p where we impose the set of signers act, signer i , and

challenge c for the p -th $\mathbf{w}_i$ to replace, and sample the secrets $\mathbf{s}_I$ to be consistent with the value $\mathbf{s}_i^{\text{part}}$.

We can then define adversaries $\mathcal{E}^v$ against the $\text{MLWE}_{q, k, \ell, (\mathbf{z} | \text{rej})_{\mathbf{v} = c \cdot \mathbf{s}_i^{\text{part}}}}$ such that

$$\left| \text{Adv}_{\mathcal{A}}^{F'^v} - \text{Adv}_{\mathcal{A}}^{F^v} \right| = \text{Adv}_{\mathcal{E}^v}^{\text{MLWE}_{q, k, \ell, (\mathbf{z} | \text{rej})_{\mathbf{v} = c \cdot \mathbf{s}_i^{\text{part}}}}} \quad (8)$$

Game₈ ensured that the twisted norm of $\mathbf{v} = c \cdot \mathbf{s}_i^{\text{part}}$ is bounded by B which ensures that $R_{\infty}^{\varepsilon}(\chi_{\mathbf{z}} || \chi_{\mathbf{r}} + \mathbf{v}) \leq M$ by Lemma 2.4. We can thus apply Lemma F.2: there exists adversaries $\mathcal{E}'^v$ and $\mathcal{E}''^v$ such that

$$\text{Adv}_{\mathcal{E}^v}^{\text{MLWE}_{q, k, \ell, (\mathbf{z} | \text{rej})_{\mathbf{v}}}} \leq \frac{1}{1 - \frac{1}{M}} \cdot (\text{Adv}_{\mathcal{E}'^v}^{\text{MLWE}_{q, k, \ell, \chi_{\mathbf{r}}}} + \text{Adv}_{\mathcal{E}''^v}^{\text{MLWE}_{q, k, \ell, \chi_{\mathbf{z}}}} + \delta^v) \quad (9)$$

where $\mathbf{v} = c \cdot \mathbf{s}_i^{\text{part}}$, δ^v is the statistical distance between $\chi_{\mathbf{z}}$ and $(\mathbf{z} | \text{rej})_{\mathbf{v} = c \cdot \mathbf{s}_i^{\text{part}}}$.

We can also apply Lemma F.1 to bound $\delta^v \leq \frac{2\varepsilon}{1 - \varepsilon}$.

By combining Equations 7, 8, and 9, we obtain:

$$\begin{aligned} \left| \text{Adv}_{\mathcal{A}}^{G^{p+1}} - \text{Adv}_{\mathcal{A}}^{G^p} \right| &\leq \frac{M}{M - 1} \max_{\text{act}, i, \|\mathbf{cs}\| \leq B} \left(\text{Adv}_{\mathcal{E}'^v}^{\text{MLWE}_{q, k, \ell, \chi_{\mathbf{r}}}} + \text{Adv}_{\mathcal{E}''^v}^{\text{MLWE}_{q, k, \ell, \chi_{\mathbf{z}}}} \right) \\ &\quad + \frac{M}{M - 1} \cdot \frac{2\varepsilon}{1 - \varepsilon} + \text{negl}(\kappa) \end{aligned}$$

Finally, by summing all the above inequalities, we obtain

$$\begin{aligned} \left| \text{Adv}_{\mathcal{A}}^{\text{Games}} - \text{Adv}_{\mathcal{A}}^{\text{Game}_9} \right| &\leq Q_s \frac{M}{M - 1} \left(\text{Adv}_{\mathcal{B}_3}^{\text{MLWE}_{q, k, \ell, \chi_{\mathbf{r}}}} + \text{Adv}_{\mathcal{B}_4}^{\text{MLWE}_{q, k, \ell, \chi_{\mathbf{z}}}} \right) \\ &\quad + Q_s \frac{M}{M - 1} \cdot \frac{2\varepsilon}{1 - \varepsilon} + \text{negl}(\kappa) \end{aligned}$$

where $\mathcal{B}_3$ is an adversary against the $\text{MLWE}_{q, k, \ell, \chi_{\mathbf{r}}}$ problem, $\mathcal{B}_4$ is an adversary against the $\text{MLWE}_{q, k, \ell, \chi_{\mathbf{z}}}$ problem.

Game₁₀. In this hybrid, we replace the real rejection sampling algorithm $\text{Rej}(c \cdot \mathbf{s}_i^{\text{part}}, \chi_{\mathbf{z}}, \chi_{\mathbf{r}}, M; \mathbf{r}_i)$ by an ideal functionality $\text{Ideal}(\chi_{\mathbf{z}}, M)$ which is independent of the secret. This is formalized in Fig. 24.

We rely on the Rényi divergence to prove that this only slightly increase the probability that $\mathcal{A}$ wins the game. For conciseness, we will abuse Rényi divergence notations to take a random variable as input, implicitly corresponding to the divergence between the marginal distributions from Game₉ and Game₁₀.

We wish to apply a Rényi divergence argument, and its multiplicativity, to the responses $(\mathbf{z}_j)_{j \in [Q_s]}$ produced in signing oracles. However, these responses all depend on the secrets and are not independent, which prevent a simple application of multiplicativity.

To solve that, we will instead apply the Rényi properties from Lemma 2.2 to the tuple $X = ((\mathbf{s}_k)_i)_{i \in [N]}, \mathbf{z}_1, \dots, \mathbf{z}_{Q_s}$.

Int_3^s <pre> 1: $Q_{\text{msg}}, \text{UnopenedCmt}[\cdot], := \emptyset$ 2: $\text{ToProgram} := \emptyset$ 3: $\text{Cnt} := 0$ 4: $\text{wins} := 0$ 5: $f_{\text{fail}} := \perp$ 6: $\text{pp}, \text{st}_{\mathcal{A}} \leftarrow \text{Setup}(\kappa, N, T)$ 7: $\text{corrupt} \leftarrow \mathcal{A}(\text{pp})$ 8: assert{ $\text{corrupt} \subseteq [N]$ } 9: assert{ $\text{corrupt} < T$ } 10: $\text{honest} := [N] \setminus \text{corrupt}$ 11: $(\text{vk}, (\text{sk}_i)_{i \in [N]}) \leftarrow \text{Sign.Keygen}(\text{pp})$ 12: for $i \in \text{honest}$ do 13: $\text{st}_i := \emptyset$ 14: $(\text{msg}, \text{sig}) \leftarrow \mathcal{A}^{\text{H}, (\text{OSign}_i(\cdot))_{i \in [R]}}$ $(\text{st}_{\mathcal{A}}, \text{vk}, (\text{sk}_i)_{i \in \text{corrupt}})$ 15: return wins </pre>	$\text{ShareSign}_2(\text{vk}, \text{act}, \text{msg}, \text{pm}_1, i, \text{sk}_i, \text{st}_i)$ <pre> 1: $\text{Cnt} := \text{Cnt} + 1$ 2: assert{ $\text{act} \subseteq [N] \wedge i \in \text{act}$ } 3: assert{ $\text{UnopenedCmt}[(i, \text{pm}_1^i)] = \top$ } 4: $\text{UnopenedCmt}[(i, \text{pm}_1^i)] := \perp$ 5: if $(\text{act}, (\text{cmt}_j)_{j \in \text{act}}, \text{msg}, \cdot) \notin$ ToProgram then 6: $\text{ToProgram} := \text{ToProgram} \cup$ $\{(\text{act}, (\text{cmt}_j)_{j \in \text{act}}, \text{msg}, \cdot)\}$ 7: $\mathbf{r}_i \leftarrow \chi_{\mathbf{r}}; \mathbf{w}_i := [\mathbf{A} \quad \mathbf{I}] \cdot \mathbf{r}_i$ 8: $\text{H}_{\text{cmt}}(\text{vk}, \mathbf{w}_i) := \text{pm}_1^i$ 9: for $(\text{act}', (\text{cmt}'_j)_{j \in \text{act}'}, \text{msg}', \cdot) \in \text{ToProgram}$ do 10: if $(i \in \text{act}') \wedge (\text{cmt}'_i = \text{pm}_1^i) \wedge (\forall j \in$ $\text{act}' \setminus \{i\}, \exists \mathbf{w}_j, \text{cmt}_j = \text{H}_{\text{cmt}}(\text{vk}, j, \mathbf{w}_j))$ then 11: $\text{ToProgram} := \text{ToProgram} \setminus$ $\{(\text{act}', (\text{cmt}'_j)_{j \in \text{act}'}, \text{msg}', \cdot)\}$ 12: $\mathbf{w} := \sum_{j \in \text{act}'} \mathbf{w}_j; (\mathbf{w}_{\top}, \mathbf{w}_{\perp}) = \text{HighBits}_q(\mathbf{w}, 2\gamma_2)$ 13: if $\text{Cnt} = s \wedge \exists (\mu \mathbf{w}_{\top}) \in \text{H}$ then $\triangleright \text{H already}$ queried on $\mathbf{w}_{\top}$ 14: $\text{wins} = 1$ 15: $\text{st}_i := \text{st}_i \cup \{(\text{act}, \text{msg}, \text{pm}_1, \mathbf{w}_i, \mathbf{r}_i)\}$ 16: return $(\text{pm}_2^i := \mathbf{w}_i, \text{st}_i)$ </pre>
Int_4^s Identical to Int_3^s	$\text{ShareSign}_2(\text{vk}, \text{act}, \text{msg}, \text{pm}_1, i, \text{sk}_i, \text{st}_i)$ <pre> 1: $\text{Cnt} := \text{Cnt} + 1$ 2: assert{ $\text{act} \subseteq [N] \wedge i \in \text{act}$ } 3: assert{ $\text{UnopenedCmt}[(i, \text{pm}_1^i)] = \top$ } 4: $\text{UnopenedCmt}[(i, \text{pm}_1^i)] := \perp$ 5: if $(\text{act}, (\text{cmt}_j)_{j \in \text{act}}, \text{msg}, \cdot) \notin$ ToProgram then 6: $\text{ToProgram} := \text{ToProgram} \cup$ $\{(\text{act}, (\text{cmt}_j)_{j \in \text{act}}, \text{msg}, \cdot)\}$ 7: if $\text{Cnt} = s$ then $\mathbf{w}_i \xleftarrow{\\$} \mathcal{R}_q^k$ 8: else $\mathbf{r}_i \leftarrow \chi_{\mathbf{r}}; \mathbf{w}_i := [\mathbf{A} \quad \mathbf{I}] \cdot \mathbf{r}_i$ Rest is identical </pre>

Figure 17: Intermediary games for the third hybrid of the unforgeability proof of Threshold ML-DSA. Difference with Game_2 are highlighted.

By the data processing inequality of the Rényi divergence, since we can see Game_9 and Game_{10} as functions of X , their Rényi divergence is bounded by the Rényi divergence of X .

We then apply the multiplicativity of Rényi divergence Lemma 2.2:

$$R_{\infty}(\text{Game}_9 || \text{Game}_{10}) \leq \underbrace{R_{\infty}((\text{sk}_i)_{i \in [N]})}_{=1} \cdot \prod_j r_j$$

where $r_j = \max_{(\text{sk}_i)_{i \in [N]}} R_{\infty}(\mathbf{z}_j | (\text{sk}_i)_{i \in [N]}).$

We can apply again the multiplicativity of the Rényi divergence:

$$R_{\infty}(\mathbf{z}_j | (\text{sk}_i)_{i \in [N]}) \leq \max_{\|\mathbf{v}\| \leq B} R_{\infty}(\mathbf{z}_j | c_j \cdot \mathbf{s}_j^{\text{part}} = \mathbf{v})$$

as $\mathbf{z}_j$ only differs between the games when $\|c \cdot \mathbf{s}_j^{\text{part}}\| \leq B$.

We finally apply Lemma F.1 to deduce that,

$$R_{\infty}(\text{Game}_9 || \text{Game}_{10}) \leq \left(1 + \frac{\varepsilon}{M-1}\right)^{Q_s}$$

Applying the probability preservation of the Rényi

Game₄
1: $Q_{\text{msg}}, \text{UnopenedCmt}[\cdot]. \text{Cmt} := \emptyset$ Rest is identical to Game ₃
ShareSign₁(vk, i, sk_i, st_i)
1: $\text{cmt}_i \xleftarrow{\$} \{0, 1\}^{2\kappa}$ 2: $\text{UnopenedCmt}[(i, \text{cmt}_i)] = \top$ 3: if $\text{cmt}_i \in \text{Cmt}$ then 4: $f_{\text{fail}} := \top$ 5: return $\perp$ 6: $\text{Cmt} := \text{Cmt} \cup \{\text{cmt}_i\}$ 7: return $(\text{pm}_1^i := \text{cmt}_i, \text{st}_i)$
ShareSign₂(vk, act, msg, pm₁, i, sk_i, st_i)
1: $\text{Cmt} := \text{Cmt} \cup \{\text{cmt}_j := \text{pm}_1^j\}_{j \in \text{act}}$ The rest is identical
H_{cmt}(vk, w)
1: if $Q_{\text{Hcmt}}[(\text{vk}, \text{w})] = \perp$ then 2: $\text{cmt} \xleftarrow{\$} \{0, 1\}^{2\kappa}$ 3: $Q_{\text{Hcmt}}[(\text{vk}, \text{w})] = \text{cmt}$ 4: if $\text{cmt}_i \in \text{Cmt}$ then 5: $f_{\text{fail}} := \top$ 6: return $\perp$ 7: $\text{Cmt} := \text{Cmt} \cup \{\text{cmt}_i\}$ 8: return $Q_{\text{Hcmt}}[(\text{vk}, \text{w})] = \perp$

Figure 18: Fourth hybrid of the unforgeability proof of Threshold ML-DSA. Difference with the previous hybrid are highlighted.

divergence, we get:

$$\text{Adv}_{\mathcal{A}}^{\text{Game}_9} \leq \left(1 + \frac{\varepsilon}{M-1}\right)^{Q_s} \cdot \text{Adv}_{\mathcal{A}}^{\text{Game}_{10}}$$

Game₁₁. In this hybrid, we remove the abort condition on the norm of $c \cdot \mathbf{s}_i^{\text{part}}$.

This can only increase the winning probability of the adversary, hence

$$\text{Adv}_{\mathcal{A}}^{\text{Game}_{10}} \leq \text{Adv}_{\mathcal{A}}^{\text{Game}_{11}}$$

Game₁₂. In this hybrid, we replace the public key by the rounding of a uniform value, i.e. $\text{Power2Round}(\mathbf{t}, d)$ where $\mathbf{t} \xleftarrow{\$} \mathcal{R}_q^k$. To preserve consistency, we fix a secret index I such that $I_0 \subseteq \text{honest}$, i.e. $\mathbf{s}_{I_0}$ is not given to the adversary, and we replace $\mathbf{t}_{I_0}$ with $\mathbf{t}_{I_0} := \mathbf{t} - \sum_{I \neq I_0} \mathbf{t}_I$. This is formalized in Fig. 25.

Game₅
Identical to Game ₄
H(str)
1: if $\text{str} = \text{tr} \parallel \text{msg}$ and $Q_{\text{H}}[\text{tr} \parallel \text{msg}] = \perp$ then 2: $\tilde{c} \leftarrow \{0, 1\}^{512}$ 3: $Q_{\text{H}}[\text{tr} \parallel \text{msg}] = \tilde{c}$ 4: if $\exists \text{msg}', Q_{\text{H}}[\text{tr} \parallel \text{msg}'] = \tilde{c}$ or $(\cdot, \tilde{c}, \cdot) \in \text{ToProgram}$ then 5: $f_{\text{fail}} := \top$ 6: return $\perp$
Rest is identical

Figure 19: Fifth hybrid of the unforgeability proof of Threshold ML-DSA. Difference with the previous hybrid are highlighted.

As secrets are no longer used by honest parties in the scheme at this stage, we can replace $\mathbf{t}_{I_0}$ by a uniform value by the $\text{MLWE}_{q,k,\ell,\chi_s}$ assumption. Equivalently, we can then sample $\mathbf{t}$ uniformly at random and define $(\mathbf{t}_{\top}, \mathbf{t}_{\perp}) := \text{Power2Round}(\mathbf{t}, d)$ and $\mathbf{t}_{I_0} := \mathbf{t} - \sum_{I \neq I_0} \mathbf{t}_I$.

Formally, there exists an adversary $\mathcal{B}_5$ against the $\text{MLWE}_{q,k,\ell,\chi_s}$ problem such that:

$$\left| \text{Adv}_{\mathcal{A}}^{\text{Game}_{11}} - \text{Adv}_{\mathcal{A}}^{\text{Game}_{12}} \right| \leq \text{Adv}_{\mathcal{B}_5}^{\text{MLWE}_{q,k,\ell,\chi_s}}$$

Game₁₃. In this hybrid, we first sample the high bits of the public key $\mathbf{t}_{\top}$ from an ML-DSA public key, then we sample the full key $\mathbf{t}$ from the distribution $\mathcal{R}_q^k$ conditioned on the value of $\mathbf{t}_{\top}$. This is formalized in Fig. 26.

We can equivalently sample $(\mathbf{t}, \mathbf{t}_{\top})$ in **Game₁₂** by first sampling $(\mathbf{t}_{\top}, \cdot) = \text{Power2Round}(\mathcal{U}(\mathcal{R}_q^k), d)$ and then sampling $\mathbf{t}$ conditioned on $\mathbf{t}_{\top}$. Then, we can see that **Game₁₃** just replaces $\mathcal{U}(\mathcal{R}_q^k)$ in this distribution by an MLWE sample with secrets sampled in χ_s , which is again indistinguishable under the $\text{MLWE}_{q,k,\ell,\chi_s}$ assumption.

Formally, there exists an adversary $\mathcal{B}_6$ against the $\text{MLWE}_{q,k,\ell,\chi_s}$ problem such that:

$$\left| \text{Adv}_{\mathcal{A}}^{\text{Game}_{12}} - \text{Adv}_{\mathcal{A}}^{\text{Game}_{13}} \right| \leq \text{Adv}_{\mathcal{B}_6}^{\text{MLWE}_{q,k,\ell,\chi_s}}$$

Conclusion. Finally, we reduce **Game₁₃** to the unforgeability of ML-DSA. We design an adversary $\mathcal{B}_7$ against the unforgeability of ML-DSA, and simulate the view of the adversary in **Game₁₃**.

$\mathcal{B}_7$ takes as input an ML-DSA public key $\text{pk} = (\rho, \mathbf{t}_{\perp})$, and random oracles **ExpandA**, **H** and **SampleInBall**.

It does the following changes:

- It programs $\text{ExpandA}(\rho) = \mathbf{A}$.

Game₆
1: $Q_{\text{msg}}, \text{UnopenedCmt}[\cdot], \text{Cmt}, \text{Responses}[\cdot] := \emptyset$ Rest is identical to Game ₄
ShareSign ₂ (vk, act, msg, pm ₁ , i, sk _i , st _i)
1: assert { act $\subseteq [N] \wedge i \in \text{act}$ } 2: assert { UnopenedCmt[(i, pm ₁ ⁱ)] = $\top$ } 3: UnopenedCmt[(i, pm ₁ ⁱ)] := $\perp$ 4: if (act, (cmt _j) _{j$\in$act} , msg, $\cdot$) $\notin$ ToProgram then 5: $\tilde{c} \xleftarrow{\$} \{0, 1\}^{2\cdot\kappa}; c \leftarrow \mathcal{C}$ 6: ToProgram := ToProgram $\cup$ { (act, (cmt _j) _{j$\in$act} , msg, $\tilde{c}$, c) } 7: Pick (a ₁ , a ₂ , a ₃ , c) from ToProgram s.t. (a ₁ , a ₂ , a ₃) = (act, (cmt _j) _{j$\in$act} , msg) 8: $\mathbf{r}_i \leftarrow \chi_{\mathbf{r}}$ 9: $\mathbf{w}_i := [\mathbf{A} \ \mathbf{I}] \cdot \mathbf{r}_i$ 10: $\mathbf{m} := \text{RSSRecover}(\text{act})$ 11: $\mathbf{s}_i^{\text{part}} := \sum_{I \in m_i} \mathbf{s}_I$ 12: $\mathbf{z}_i \leftarrow \text{Rej}(c \cdot \mathbf{s}_i^{\text{part}}, \chi_{\mathbf{z}}, \chi_{\mathbf{r}}, M; \mathbf{r}_i)$ 13: Responses[(act, (cmt _j) _{j$\in$act} , msg, i)] := $\mathbf{z}_i$ Rest is identical
ShareSign ₃ (vk, act, msg, i, pm ₂ , i, sk _i , st _i)
1: assert { (act, msg, $\cdot$, pm ₂ ⁱ , $\cdot$) $\in$ st _i } 2: Pick (act, msg, pm ₁ , w _i , $\cdot$) from st _i with pm ₂ ⁱ = w _i 3: Parse pm ₁ = (cmt _j) _j , pm ₂ = (w _j) _{j$\neq$i} 4: for j $\in$ act do 5: if cmt _j $\neq$ H _{cmt} (vk, j, w _j) then 6: abort $\triangleright$ Check hash commit. 7: st _i := st _i \ { (act, msg, pm ₁ , w _i , $\cdot$) } 8: if $\mathbf{z}_i^1, \mathbf{z}_i^2 := \text{Responses}[(\text{act}, (\text{cmt}_j)_{j \in \text{act}}, \text{msg}, i)]$ then return ($\mathbf{z}_i^1$, st _i) 9: else abort

Figure 20: Sixth hybrid of the unforgeability proof of Threshold ML-DSA. Difference with the previous hybrid are highlighted.

- It chooses $\mathbf{t}_{\top}$ and ρ provided by the unforgeability challenger in the threshold Keygen.
- Queries to the random oracles H and SampleInBall are lazily passed to the random oracles provided by the unforgeability challenger instead of honestly sampling their values. Note that in case Game₁₂ programs them otherwise, this behavior is not affected.

$\mathcal{B}_7$ forwards any forgery it receives from $\mathcal{A}$ to the ML-DSA unforgeability challenger.

Game₇
Identical to Game ₆
ShareSign ₂ (vk, act, msg, pm ₁ , i, sk _i , st _i)
1: assert { act $\subseteq [N] \wedge i \in \text{act}$ } 2: assert { UnopenedCmt[(i, pm ₁ ⁱ)] = $\top$ } 3: UnopenedCmt[(i, pm ₁ ⁱ)] := $\perp$ 4: if (act, (cmt _j) _{j$\in$act} , msg, $\cdot$) $\notin$ ToProgram then 5: $\tilde{c} \xleftarrow{\$} \{0, 1\}^{2\cdot\kappa}; c \leftarrow \mathcal{C}$ 6: ToProgram := ToProgram $\cup$ { (act, (cmt _j) _{j$\in$act} , msg, $\tilde{c}$, c) } 7: Pick (a ₁ , a ₂ , a ₃ , c) from ToProgram s.t. (a ₁ , a ₂ , a ₃) = (act, (cmt _j) _{j$\in$act} , msg) 8: $\mathbf{m} := \text{RSSRecover}(\text{act})$ 9: $\mathbf{s}_i^{\text{part}} := \sum_{I \in m_i} \mathbf{s}_I$ 10: $\mathbf{z}_i \leftarrow \text{Rej}(c \cdot \mathbf{s}_i^{\text{part}}, \chi_{\mathbf{z}}, \chi_{\mathbf{r}}, M)$ 11: if $\mathbf{z}_i = \perp$ then $\triangleright$ if reject 12: $\mathbf{z}_i \leftarrow (\mathbf{z} \text{rej})_{v=c \cdot \mathbf{s}_i^{\text{part}}}$ 13: $\mathbf{t}_i^{\text{part}} := \sum_{I \in m_i} \mathbf{t}_I$ 14: $\mathbf{w}_i := [\mathbf{A} \ \mathbf{I}] \cdot \mathbf{z}_i - c \cdot \mathbf{t}_i^{\text{part}}$ 15: if $\mathbf{z}_i \neq \perp$ then 16: Responses[(act, (cmt _j) _{j$\in$act} , msg, i)] := $\mathbf{z}_i$ Rest is identical

Figure 21: Seventh hybrid of the unforgeability proof of Threshold ML-DSA. Difference with the previous hybrid are highlighted.

The view of the adversary $\mathcal{A}$ is identically distributed since the ML-DSA random oracles sample images from the same distributions as Game₁₃, and the ML-DSA public key is sampled as $(\rho, \mathbf{t}_{\top})$ in Game₁₃.

Then, we can see that a valid forgery in Game₁₃ will also be a valid forgery for ML-DSA. Indeed, the verification procedure only depends on ρ and $\mathbf{t}_{\top}$ which are chosen identical for the ML-DSA signature and the threshold signature. Also, the challenge random oracles coincide on messages for which forgeries are valid (i.e. messages for which no signing oracle was called) as a $\tilde{c}$ programmed by Game₁₃ will never be used by the ML-DSA oracles thanks to the condition added in Game₅. Hence,

$$\text{Adv}_{\mathcal{B}_7}^{\text{UF}} = \text{Adv}_{\mathcal{A}}^{\text{Game}_{13}}$$

We finally collect all the bounds to obtain the theorem statement, noting that all the intermediary adversaries have an execution time roughly equal to $\mathcal{T}(\mathcal{A})$. $\square$

Game ₈
Identical to Game ₇
ShareSign ₂ (vk, act, msg, pm ₁ , i, sk _i , st _i)
<pre> 1: assert{ act ⊆ [N] ∧ i ∈ act } 2: assert{ UnopenedCmt[(i, pm₁^{i)] = ⊤ } 3: UnopenedCmt[(i, pm₁^{i)] := ⊥ 4: if (act, (cmt_j)_{j∈act}, msg, ·) ∉ ToProgram then 5: $\tilde{c} \xleftarrow{\\$} \{0, 1\}^{2 \cdot \kappa}; c \leftarrow \mathcal{C}$ 6: ToProgram := ToProgram ∪ {(act, (cmt_j)_{j∈act}, msg, $\tilde{c}$, c)} 7: Pick (a₁, a₂, a₃, c) from ToProgram s.t. (a₁, a₂, a₃) = (act, (cmt_j)_{j∈act}, msg) 8: m := RSSRecover(act) 9: s_i^{part} := ∑_{I∈m_i} s_I; (u₁, u₂) = s_i^{part} 10: if (ν · c · u₁, c · u₂) ₂ > B then 11: f_{fail} := ⊤ 12: return ⊥ 13: z_i ← Rej(c · s_i^{part}, χ_z, χ_r, M) 14: if z_i = ⊥ then ▷ if reject 15: z_i ← (z rej)_{v=c · s_i^{part}} 16: t_i^{part} := ∑_{I∈m_i} t_I 17: w_i := [A I] · z_i - c · t_i^{part} 18: if z_i ≠ ⊥ then 19: Responses[(act, (cmt_j)_{j∈act}, msg, i)] := z_i}}</pre>
Rest is identical

Figure 22: Eighth hybrid of the unforgeability proof of Threshold ML-DSA. Difference with the previous hybrid are highlighted.

Game ₉
Identical to Game ₈
ShareSign ₂ (vk, act, msg, pm ₁ , i, sk _i , st _i)
<pre> 1: assert{ act ⊆ [N] ∧ i ∈ act } 2: assert{ UnopenedCmt[(i, pm₁^{i)] = ⊤ } 3: UnopenedCmt[(i, pm₁^{i)] := ⊥ 4: if (act, (cmt_j)_{j∈act}, msg, ·) ∉ ToProgram then 5: $\tilde{c} \xleftarrow{\\$} \{0, 1\}^{2 \cdot \kappa}; c \leftarrow \mathcal{C}$ 6: ToProgram := ToProgram ∪ {(act, (cmt_j)_{j∈act}, msg, $\tilde{c}$, c)} 7: Pick (a₁, a₂, a₃, c) from ToProgram s.t. (a₁, a₂, a₃) = (act, (cmt_j)_{j∈act}, msg) 8: m := RSSRecover(act) 9: s_i^{part} := ∑_{I∈m_i} s_I; (u₁, u₂) = s_i^{part} 10: if (ν · c · u₁, c · u₂) ₂ > B then 11: f_{fail} := ⊤ 12: return ⊥ 13: z_i ← Rej(c · s_i^{part}, χ_z, χ_r, M) 14: if z_i = ⊥ then 15: w_i ← $\mathcal{R}_q^k$ 16: else 17: t_i^{part} := ∑_{I∈m_i} t_I 18: w_i := [A I] · z_i - c · t_i^{part} 19: Responses[(act, (cmt_j)_{j∈act}, msg, i)] := z_i}}</pre>
Rest is identical

Figure 23: Ninth hybrid of the unforgeability proof of Threshold ML-DSA. Difference with the previous hybrid are highlighted.

Game₁₀
Identical to Game ₆
ShareSign ₂ (vk, act, msg, pm ₁ , i, sk _i , st _i)
<pre> 1: assert{ act ⊆ [N] ∧ i ∈ act } 2: assert{ UnopenedCmt[(i, pm₁^{i)] = ⊤ } 3: UnopenedCmt[(i, pm₁^{i)] := ⊥ 4: if (act, (cmt_j)_{j∈act}, msg, ·) ∉ ToProgram then 5: $\tilde{c} \xleftarrow{\\$} \{0, 1\}^{2 \cdot \kappa}; c \leftarrow \mathcal{C}$ 6: ToProgram := ToProgram ∪ {(act, (cmt_j)_{j∈act}, msg, $\tilde{c}$, c)} 7: Pick (a₁, a₂, a₃, c) from ToProgram s.t. (a₁, a₂, a₃) = (act, (cmt_j)_{j∈act}, msg) 8: m := RSSRecover(act) 9: s_i^{part} := ∑_{I∈m_i} s_I; (u₁, u₂) = s_i^{part} 10: if (ν · c · u₁, c · u₂) ₂ > B then 11: f_{fail} := ⊤ 12: return ⊥ 13: z_i ← Ideal(χ_z, M) 14: if z_i = ⊥ then 15: w_i $\xleftarrow{\\$} \mathcal{R}_q^k$ 16: else 17: t_i^{part} := ∑_{I∈m_i} t_I 18: w_i := [A I] · z_i - c · b_i^{part} 19: Responses[(act, (cmt_j)_{j∈act}, msg, i)] := z_i}}</pre>
Rest is identical

Figure 24: Tenth hybrid of the unforgeability proof of Threshold ML-DSA. Difference with the previous hybrid are highlighted .

Game₁₂
Identical to Game ₈
Keygen(κ, N, T) → (vk, sk)
<pre> 1: ρ ← {0, 1}²⁵⁶ 2: A := ExpandA(ρ) 3: for I ⊂ [N] s.t. I = N - T + 1 and I ≠ I₀ do 4: s_I ← χ_s 5: t_I := [A I] · s_I 6: for i ∈ I do 7: sk_i = sk_i ∪ {s_I} 8: t $\xleftarrow{\\$} \mathcal{R}_q^k$ 9: t_{I₀} := t - ∑_{I≠I₀} t_I 10: (t_⊥, t_⊥) := Power2Round(t, d) 11: vk := (ρ, t_⊥) 12: sk := (tr, sk_i)_{i∈[N]} 13: return (vk, sk) </pre>
▷ Compute partial public keys ▷ Distribute the keys ▷ Verification key

Figure 25: Twelfth hybrid of the unforgeability proof of Threshold ML-DSA. Difference with the previous hybrid are highlighted .

Game₁₃
Identical to Game ₈
Keygen(κ, N, T) → (vk, sk)
<pre> 1: ρ ← {0, 1}²⁵⁶ 2: A := ExpandA(ρ) 3: for I ⊂ [N] s.t. I = N - T + 1 and I ≠ I₀ do 4: s_I ← χ_s 5: t_I := [A I] · s_I 6: for i ∈ I do 7: sk_i = sk_i ∪ {s_I} 8: (s, e) ← χ_s 9: t̂ := A · s + e 10: (t_⊥, ·) := Power2Round(t̂, d) 11: Sample t from U(ℳ_q^k) s.t. (t_⊥, ·) = Power2Round(t̂, d) 12: t_{I₀} := t - ∑_{I≠I₀} t_I 13: vk := (ρ, t_⊥) 14: sk := (tr, sk_i)_{i∈[N]} 15: return (vk, sk) </pre>
▷ Compute partial public keys ▷ Distribute the keys ▷ Verification key

Figure 26: Thirteenth hybrid of the unforgeability proof of Threshold ML-DSA. Difference with the previous hybrid are highlighted .